Universidad Nacional Autónoma de México

Facultad de Ciencias

Aplicación de Redes Neuronales en la Solución de Ecuaciones Diferenciales Ordinarias y Parciales

Presenta:
Santiago Iván Reyes Medina

Asesor:
Prof. Roberto Méndez Méndez

Índice general

Introducción **4**
Contexto General 4
Propósito y Objetivos 6
Descripción del Proyecto 7

1. Configuración del Entorno de Desarrollo **8**
1.1. Creación del Entorno Virtual 9
1.2. Instalación en macOS (Apple Silicon) 9
1.3. Instalación en Windows y Linux 10
1.4. Verificación y Pruebas del Entorno 10

2. Fundamentos de Redes Neuronales **12**
2.1. ¿Qué es una Red Neuronal? 13
2.1.1. Inspiración Biológica 13
2.1.2. El Modelo Matemático de la Neurona Artificial 14
2.1.3. Estructura por Capas 16
2.1.4. Componentes Internos de un Nodo 16
2.1.5. La Red como Grafo Computacional y Notación Matricial 17
2.2. Tipos de Redes Neuronales 19
2.2.1. Redes Neuronales *Feedforward* (FNN) y el Perceptrón Multicapa 19
2.2.2. Redes Neuronales Convolucionales (CNN) 19
2.2.3. Redes Neuronales Recurrentes (RNN) 20
2.2.4. Arquitecturas Basadas en Atención (*Transformers*) 21
2.2.5. Otras Arquitecturas Relevantes 21
2.3. Funciones de Activación 22
2.3.1. Funciones de Tipo Sigmoidal 22
2.3.2. Funciones de Tipo Rectificador 24
2.3.3. Funciones de Activación Avanzadas 26
2.3.4. Función Softmax (Capa de Salida) 27
2.3.5. Criterios de Selección y Resumen Comparativo 28
2.4. Funciones de Pérdida 28
2.4.1. Error Cuadrático Medio (MSE) 29
2.4.2. Entropía Cruzada Binaria 29
2.4.3. Entropía Cruzada Categórica 29
2.5. Técnicas de Regularización 29
2.5.1. Penalización de la Norma de los Pesos 30
2.5.2. Dropout 30

2.5.3. Detención Temprana . . . 30
2.5.4. Normalización por Lotes . . . 30
2.6. Saturación y Normalización de los Datos . . . 31

3. Operación Práctica de una Red Neuronal **33**
3.1. Evaluación Directa (Forward Propagation) . . . 34
3.2. Ajuste de Pesos y Sesgos (Backpropagation) . . . 38
3.3. Implementación Computacional . . . 43
3.4. Comparativa de Resultados . . . 46

4. Machine Learning para Ecuaciones Diferenciales **49**
4.1. Physics-Informed Neural Networks (PINNs) . . . 50
4.1.1. Formulación general del problema y la idea central . . . 50
4.1.2. Construcción de la red informada por la física . . . 51
4.1.3. Función de pérdida compuesta . . . 52
4.1.4. Puntos de colocación y estrategias de muestreo . . . 53
4.1.5. Diferenciación automática: uso dual en PINNs . . . 54
4.1.6. Regularización implícita y eficiencia en datos . . . 55
4.1.7. El problema directo y el problema inverso . . . 55
4.2. PINNs Auto-Adaptativas (SA-PINNs) . . . 56
4.2.1. El problema del desequilibrio en la optimización . . . 56
4.2.2. Pesos auto-adaptativos por punto de entrenamiento . . . 57
4.2.3. Optimización minimax: descenso-ascenso simultáneo . . . 58
4.2.4. Justificación teórica mediante el kernel tangente neuronal . . . 59
4.3. Ecuaciones Diferenciales Ordinarias Neuronales (Neural ODEs) . . . 59
4.3.1. De las redes residuales al límite continuo . . . 59
4.3.2. El método del adjunto para la retropropagación . . . 60
4.3.3. Ventajas computacionales . . . 62
4.3.4. Modelado de series temporales continuas . . . 63
4.4. Perspectivas y métodos complementarios . . . 63

5. Solución de Ecuaciones mediante PINNs **64**
5.1. Ecuaciones Diferenciales Ordinarias No Lineales y Rígidas . . . 65
5.1.1. Planteamiento del problema . . . 65
5.1.2. Formulación PINN para el oscilador de Van der Pol . . . 66
5.1.3. Implementación en PyTorch . . . 68
5.1.4. Resultados y comparación con la solución de referencia . . . 72
5.1.5. Estudios de sensibilidad . . . 74
5.2. Ecuación de Onda en 1D . . . 78
5.2.1. Planteamiento del problema . . . 78
5.2.2. Formulación PINN para la ecuación de onda . . . 80
5.2.3. Implementación en PyTorch . . . 82
5.2.4. Resultados y comparación con la solución de referencia . . . 85
5.2.5. Estudios de sensibilidad . . . 87
5.3. Ecuación del Calor en 1D . . . 91
5.3.1. Planteamiento del problema . . . 91
5.3.2. Formulación PINN para la ecuación del calor . . . 93
5.3.3. Implementación en PyTorch . . . 95

5.3.4. Resultados y comparación con la solución de referencia . . . 96
5.3.5. Estudios de sensibilidad . . . 99
5.4. Ecuación de Laplace (Poisson) en 1D . . . 103
5.4.1. Planteamiento del problema . . . 104
5.4.2. Formulación PINN para la ecuación de Poisson . . . 105
5.4.3. Implementación en PyTorch . . . 106
5.4.4. Resultados y comparación con la solución de referencia . . . 108
5.4.5. Estudios de sensibilidad . . . 111
5.5. Análisis transversal de hiperparámetros y funciones de pérdida . . . 114
5.5.1. Síntesis comparativa de los casos base . . . 114
5.5.2. Hallazgos transversales sobre hiperparámetros . . . 115
5.5.3. Taxonomía de modos de falla . . . 117
5.5.4. Recomendaciones metodológicas . . . 118

Conclusiones Generales **120**
5.6. Cumplimiento de los objetivos . . . 120
5.7. Limitaciones del trabajo . . . 121
5.8. Trabajo futuro . . . 121
5.9. Reflexión final . . . 121

A. Repositorio del Proyecto **122**

Bibliografía **122**

Introducción

Contexto General

Las ecuaciones diferenciales constituyen una de las herramientas matemáticas más relevantes para la modelación de fenómenos físicos, biológicos e ingenieriles. Desde la mecánica clásica de Newton hasta la dinámica de fluidos descrita por las ecuaciones de Navier-Stokes, pasando por los modelos de transferencia de calor y los sistemas de ondas electromagnéticas, la formulación y resolución de ecuaciones diferenciales ordinarias y parciales ha sido central en el avance del conocimiento científico y tecnológico. En muchos de estos problemas, las soluciones analíticas exactas no existen o resultan extremadamente difíciles de obtener, lo cual ha motivado durante décadas el desarrollo de métodos numéricos clásicos como las *diferencias finitas*, los *elementos finitos* y los *métodos espectrales*. Si bien estos métodos han demostrado una enorme eficacia y madurez teórica, también presentan limitaciones intrínsecas: su dependencia de una discretización adecuada del dominio, su costo computacional creciente con la dimensionalidad del problema, y su rigidez frente a geometrías complejas o condiciones de frontera no convencionales.

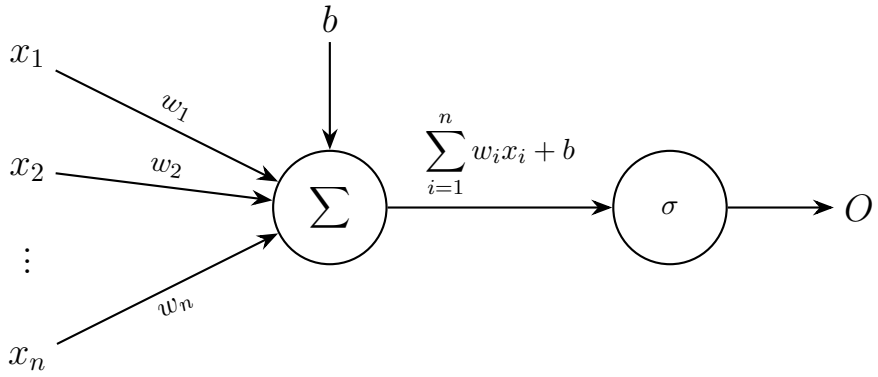
En paralelo al desarrollo de los métodos numéricos, el campo de la inteligencia artificial ha experimentado una evolución profunda y acelerada. Los orígenes formales de la IA se remontan a la conferencia de Dartmouth College en 1956, evento donde se acuñó el término y se sentaron las bases de un campo interdisciplinario convergente con las Ciencias de la Computación, la Teoría de la Información, la Psicología y la Lingüística. Durante sus primeras décadas, la IA enfrentó severas limitaciones tanto teóricas como tecnológicas. Una de ellas fue el uso de funciones de activación estrictamente lineales, lo cual restringía las capacidades de los modelos iniciales e impidió que alcanzaran un impacto práctico más allá del ámbito conceptual y no fue sino hasta la década de 1980 que el campo alcanzó un resurgimiento genuino. Este avance fue impulsado por el aumento en el poder computacional y, de manera crucial, por el desarrollo del algoritmo de retropropagación, el cual permitió el entrenamiento eficiente de modelos de redes neuronales multicapa y superó las limitaciones de los modelos lineales. Gracias a esto renació el interés en la investigación de redes neuronales artificiales en muy diversos ámbitos.

Las redes neuronales artificiales representan una de las líneas de desarrollo más destacadas dentro de la IA, derivando su nombre de su propósito fundamental: simular la estructura y la interacción de las neuronas biológicas. En su formulación más elemental, una neurona artificial funciona como una unidad de procesamiento que recibe un conjunto de entradas, les asigna pesos diferenciados, evalúa la combinación lineal resultante y emite una respuesta a través de una función de activación no lineal. Esta operación puede expresarse matemáticamente como

$$O = \sigma\Big(\sum_{i=1}^{n} w_i\, x_i + b\Big),$$

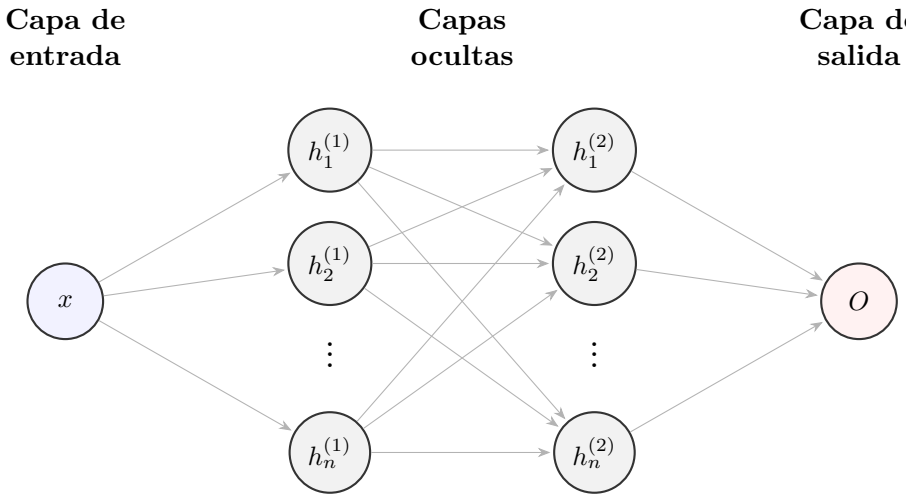
donde $x_1, \ldots, x_n$ son las entradas, $w_1, \ldots, w_n$ los pesos, b el sesgo y σ la función de activación. Precisamente la no linealidad introducida por σ es lo que permite a la red aproximar funciones

complejas; de otro modo, la red se reduciría a una mera transformación lineal, independientemente de la cantidad de capas ocultas que posea.



Kaveh, en su libro "Applications of Artificial Neural Networks and Machine Learning in Civil Engineering" señala que las redes neuronales se componen de al menos tres capas:

- **Capa de entrada:** recibe los datos de entrada sin realizar operaciones de cómputo.
- **Capas ocultas:** llevan a cabo las transformaciones no lineales necesarias para procesar la información. La red puede contener una o más de estas capas.
- **Capa de salida:** produce el resultado final del procesamiento de la red.



Cuando el número de capas ocultas crece, la red adquiere el carácter de red profunda, y el campo que estudia estas arquitecturas se conoce como aprendizaje profundo. En la actualidad, las redes neuronales son reconocidas como una tecnología clave capaz de modelar problemas no lineales y empleándose ampliamente en áreas como el procesamiento de imágenes y voz, la traducción automática y el análisis de datos.

A lo largo de su evolución, se han propuesto múltiples arquitecturas de redes neuronales. Las redes feedforward (FNN), en las que la información fluye en una única dirección sin retroalimentación; las redes neuronales convolucionales (CNN) que están diseñadas para datos con topología de cuadrícula, como las imágenes, y reemplazan la multiplicación matricial global por operaciones de convolución local; las redes neuronales recurrentes (RNN) que introducen conexiones cíclicas que

permiten el procesamiento secuencial con memoria interna y por último las arquitecturas basadas en atención que han revolucionado el panorama del aprendizaje profundo al procesar secuencias de forma simultánea mediante mecanismos de self-attention.

Un aspecto técnico determinante en cualquier arquitectura, es la elección de las funciones de activación. La sigmoide, la tangente hiperbólica, la unidad lineal rectificada (ReLU) y sus variantes (Leaky ReLU, ELU, Swish, entre otras) ofrecen propiedades matemáticas distintas en cuanto a rango de salida, comportamiento del gradiente y velocidad de convergencia. La selección apropiada de estas funciones, junto con la definición de los hiperparámetros de la red, resulta crucial para el buen desempeño del modelo frente al problema específico que se desea resolver.

El actual contexto de madurez tanto en los métodos numéricos para ecuaciones diferenciales como en las técnicas de aprendizaje profundo, ha propiciado un punto de convergencia natural: la aplicación de redes neuronales a la resolución de ecuaciones diferenciales, siendo una de estas familias de métodos las denominadas *Physics-Informed Neural Networks* (PINNs), las cuales han demostrado que es posible incorporar las leyes físicas directamente en la función de pérdida de la red, guiando el proceso de entrenamiento para que la solución aproximada satisfaga la ecuación diferencial, las condiciones iniciales y las condiciones de frontera de manera simultánea. Esta estrategia representa un cambio de paradigma frente a los métodos numéricos tradicionales, ya que no requiere una discretización explícita del dominio y ofrece la flexibilidad de operar sobre dominios irregulares con relativa facilidad.

Propósito y Objetivos

Propósito

El propósito general de este trabajo es doble. Por un lado, se busca realizar una investigación sobre la aplicación de redes neuronales artificiales en la resolución de ecuaciones diferenciales ordinarias y parciales, explorando los alcances y las limitaciones de los métodos basados en aprendizaje profundo —en particular, las PINNs— frente a los métodos numéricos clásicos. Por otro lado, se pretende mostrar elementos de *Redes Neuronales* que pueden integrarse de manera natural en los cursos de Matemáticas para Ciencias Aplicadas que se imparten en la licenciatura en Ciencias de la Computación, enriqueciendo la formación matemática del estudiante con herramientas computacionales modernas.

Objetivos

De acuerdo con el plan de trabajo establecido para este proyecto, los objetivos académicos centrales son:

1. Realizar una investigación estructurada sobre la aplicación de Redes Neuronales en la resolución de Ecuaciones Diferenciales Ordinarias (EDO) y Parciales (EDP).

2. Identificar y mostrar elementos teóricos y prácticos de Redes Neuronales y *Machine Learning* basándonos en el modelo PINN, con el propósito de generar un recurso didáctico que permita a los alumnos del curso de Matemáticas para Ciencias Aplicadas 4 aprender y aplicar los conocimientos vistos en la materia.

Descripción del Proyecto

El presente trabajo, desarrollado en el marco del Servicio Social bajo la asesoría del Prof. Roberto Méndez Méndez, aborda el estudio de las redes neuronales informadas por la física mediante una metodología progresiva. Para dar cumplimiento a los objetivos planteados, el documento y el desarrollo técnico se estructuran de la siguiente manera:

- **Fundamentos Teóricos.** Se establece la base conceptual del aprendizaje profundo, abordando la definición matemática de las redes neuronales, las arquitecturas fundamentales y el análisis de las funciones de activación que dotan al modelo de su capacidad de aproximación universal.
- **Mecánica Computacional.** Se desglosa la operación interna de una red neuronal utilizando funciones reales-valuadas elementales. Mediante la implementación de algoritmos de evaluación directa (*forward propagation*) y retropropagación (*backpropagation*), se ilustra un ciclo completo de aprendizaje, contrastando cuantitativamente las predicciones de la red con funciones analíticas conocidas.
- **Aprendizaje Profundo para Ecuaciones Diferenciales.** Se examinan las técnicas computacionales orientadas a sistemas dinámicos, profundizando en la arquitectura de las *Physics-Informed Neural Networks* (PINN).
- **Experimentación y Análisis de Casos.** Constituye el núcleo aplicado del proyecto. Se instrumentan redes neuronales para resolver problemas matemáticos de complejidad ascendente: desde una ecuación diferencial ordinaria no lineal con propiedades de rigidez, hasta ecuaciones diferenciales parciales clásicas en una dimensión espacial (onda, calor y Laplace). En cada caso, se somete al modelo a un análisis de sensibilidad, variando hiperparámetros clave como la topología de la red y las funciones de pérdida.

El componente práctico de este proyecto está respaldado por la configuración de un entorno de desarrollo reproducible (soportado en TensorFlow y PyTorch con aceleración por hardware), cuyos manuales de instalación se detallan en el Capítulo 1. Asimismo, la totalidad del código fuente generado se encuentra alojada en un repositorio público para garantizar la reproducibilidad de los experimentos.

Capítulo 1

Configuración del Entorno de Desarrollo

Para mantener las dependencias ordenadas y garantizar la reproducibilidad de los experimentos, se utiliza la distribución Anaconda versión 2025.12 (puedes descargar la última versión disponible ya que Anaconda es únicamente el administrador del entorno) para gestionar los paquetes y entornos virtuales. El proceso de descarga e instalación inicial se realiza a través de los instaladores proporcionados en la página oficial de Anaconda (`https://www.anaconda.com/download`), seleccionando el archivo correspondiente a la arquitectura de tu sistema operativo.

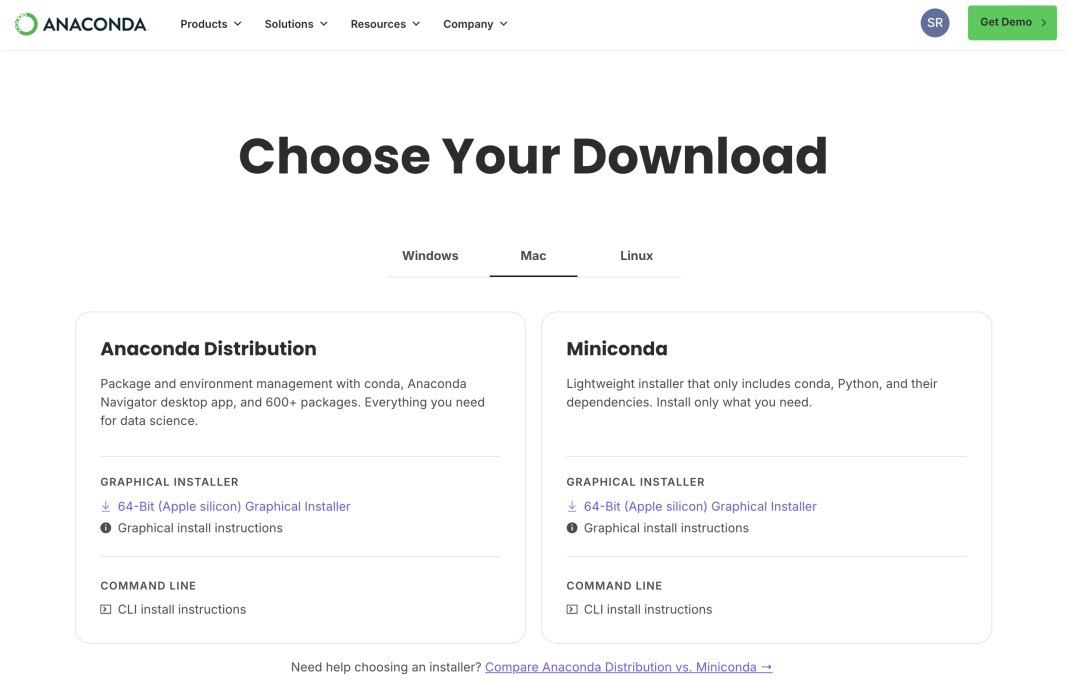


Figura 1.1: Selección del instalador de Anaconda según el sistema operativo.

Una vez descargado el archivo, se ejecuta el asistente de instalación.

1.1. Creación del Entorno Virtual

Una vez completada la instalación de Anaconda, es altamente recomendable crear un nuevo entorno de desarrollo para que, en caso de alguna falla, no se corrompa el entorno raíz (base). Para este proyecto, se requiere la versión 3.12 de Python. Desde la terminal o *Anaconda Prompt*, se ejecutan los siguientes comandos para crear y activar el entorno:

```
# Creamos un nuevo entorno virtual usando Anaconda con el nombre
# Proyecto y especificamos la version de Python
conda create --name Proyecto python=3.12

# Activamos el entorno
conda activate Proyecto
```

Posteriormente, se actualiza el gestor de paquetes `pip` para evitar conflictos de compatibilidad durante la descarga de las dependencias:

```
python -m pip install --upgrade pip
```

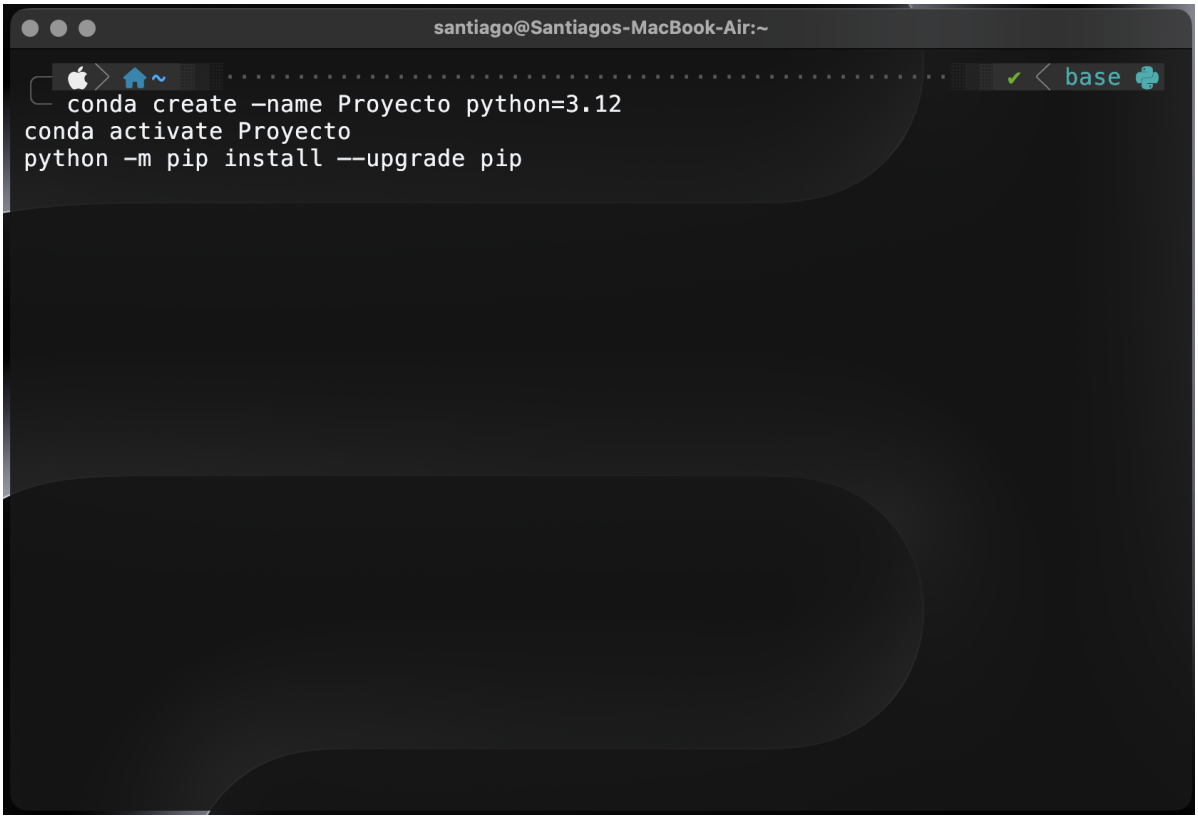


Figura 1.2: Ejecución de los comandos para crear el entorno virtual con Python 3.12.

A partir de este punto, todos los comandos de instalación de dependencias deben ejecutarse en la misma terminal, una vez activado el entorno virtual.

1.2. Instalación en macOS (Apple Silicon)

Para aprovechar la arquitectura de los procesadores de Apple (como la serie M), es necesario instalar versiones específicas de las herramientas. Emplear la memoria unificada acelera considerable-

mente el entrenamiento de los modelos matemáticos a través del backend MPS (*Metal Performance Shaders*).

En el caso de PyTorch, la versión 2.12.1 incluye soporte nativo para los chips de Apple, por lo que la aceleración gráfica se configura automáticamente sin necesidad de librerías adicionales. Se recomienda instalar esta versión junto con sus dependencias base:

```
pip install torch==2.12.1 torchvision torchaudio
```

Adicionalmente, el proyecto requiere librerías científicas y de graficación:

```
pip install numpy scipy matplotlib
```

Aunque el motor principal de este proyecto es PyTorch, si se requiere instalar TensorFlow en el mismo entorno, la versión compatible con Python 3.12 es la 2.21.0. Para esta versión en macOS (arquitectura ARM64), el paquete oficial opera exclusivamente mediante la CPU:

```
pip install tensorflow==2.21.0
```

1.3. Instalación en Windows y Linux

Para entornos Windows y Linux, la instalación base de TensorFlow es directa si el objetivo es ejecutar los cálculos en la CPU:

```
pip install tensorflow==2.21.0
```

Si se cuenta con una GPU NVIDIA y se desea habilitar la aceleración por hardware, el proceso varía. En distribuciones Linux, la instalación de la versión con soporte para CUDA se realiza mediante el siguiente comando, siempre y cuando los drivers correspondientes ya existan en el sistema:

```
python -m pip install tensorflow[and-cuda]
```

Es fundamental destacar que, a partir de la versión 2.11, TensorFlow retiró el soporte nativo de GPU para Windows. Por lo tanto, para habilitar la tarjeta gráfica en este sistema operativo, resulta estrictamente necesario configurar el entorno a través de la arquitectura WSL2 (*Windows Subsystem for Linux*). Para conocer los pasos detallados de esta configuración, se recomienda consultar la documentación oficial de TensorFlow (`https://www.tensorflow.org/install/pip`).

Para instalar PyTorch con soporte de GPU en Windows o Linux, el desarrollador debe especificar explícitamente la versión de CUDA instalada en el equipo. Por ejemplo, para un entorno con CUDA 11.8:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/
    whl/cu118
```

Si el sistema operará exclusivamente con la CPU, el comando estándar sin el parámetro `--index-url` es suficiente.

1.4. Verificación y Pruebas del Entorno

Para comprobar el éxito de la instalación y validar que el entorno virtual reconoce el hardware correctamente (CPU o GPU), se pueden ejecutar pequeños scripts de prueba.

Para verificar la integración de TensorFlow en cualquier sistema operativo, el código es el siguiente:

```python
import tensorflow as tf

# Listar dispositivos físicos (CPU)
print("CPUs disponibles:", tf.config.list_physical_devices('CPU'))

# Listar dispositivos físicos (GPU)
gpus = tf.config.list_physical_devices('GPU')

if gpus:
    print("TensorFlow detectó una GPU.")
    for gpu in gpus:
        print("Detalles:", gpu)
else:
    print("El entorno operará exclusivamente con la CPU.")
```

Para verificar PyTorch, el código difiere en función de la arquitectura del hardware (MPS para Apple o CUDA para NVIDIA).

Para entornos Windows (vía WSL2) y Linux con soporte CUDA:

```python
import torch

# Verifica si CUDA (NVIDIA) está disponible
if torch.cuda.is_available():
    print("PyTorch detectó la GPU (CUDA).")
    print("Nombre del dispositivo:", torch.cuda.get_device_name(0))

    # Asignación de un tensor a la memoria de la GPU
    device = torch.device('cuda')
    x = torch.randn(3, 3, device=device)
    print(x)
else:
    print("CUDA no está disponible. PyTorch usará la CPU.")
```

Para procesadores de Apple (serie M):

```python
import torch

# Verifica que exista una Apple GPU y soporte MPS
if torch.backends.mps.is_available() and torch.backends.mps.is_built():
    print("PyTorch detectó la GPU (MPS).")

    # Asignación de un tensor a la memoria de la GPU
    device = torch.device('mps')
    x = torch.randn(3, 3, device=device)
    print(x)
else:
    print("MPS no está disponible. PyTorch usará la CPU.")
```

Este código evalúa en verdadero si el componente MPS está activo y muestra el tensor asignado al dispositivo “mps:0”.

Capítulo 2

Fundamentos de Redes Neuronales

Una vez configurado el entorno de desarrollo con los frameworks TensorFlow y PyTorch, es posible abordar la construcción y el entrenamiento de redes neuronales. Sin embargo, antes de ejecutar cualquier algoritmo, resulta indispensable comprender, al menos de manera básica, los fundamentos teóricos que subyacen a estas estructuras computacionales. El presente capítulo tiene como objetivo establecer dicha base teórica.

Se parte de la pregunta elemental *¿qué es una red neuronal?*. A continuación, se formalizan las principales arquitecturas de redes neuronales, se estudian las funciones de activación y se enuncia el Teorema de Aproximación Universal, piedra angular que justifica la capacidad de estas redes.

Teorema
APROXIMACIÓN UNIVERSAL Sea $\sigma : \mathbb{R} \to \mathbb{R}$ una función continua, acotada y no constante, y sea $K \subset \mathbb{R}^n$ un conjunto compacto. Si $f : K \to \mathbb{R}$ es una función continua, entonces para todo $\varepsilon > 0$ existen un entero N, vectores $\mathbf{w}_j \in \mathbb{R}^n$ y escalares $b_j, \alpha_j \in \mathbb{R}$, con $j = 1, \ldots, N$, tales que la función

$$F(\mathbf{x}) = \sum_{j=1}^{N} \alpha_j \, \sigma\left(\mathbf{w}_j^\top \mathbf{x} + b_j\right)$$

satisface

$$|f(\mathbf{x}) - F(\mathbf{x})| < \varepsilon \qquad \text{para todo } \mathbf{x} \in K.$$

Se analizan también los problemas del gradiente que surgen durante el entrenamiento, se definen las funciones de pérdida asociadas a distintas tareas y se presentan las técnicas de regularización que permiten controlar el sobreajuste. Finalmente, se discuten la saturación y la normalización de los datos de entrada.

Todos estos elementos constituyen el vocabulario matemático necesario para comprender la operación práctica de una red neuronal (forward propagation y backpropagation) que se desarrollará en el capítulo siguiente.

2.1. ¿Qué es una Red Neuronal?

2.1.1. Inspiración Biológica

La idea de simular el sistema neuronal de los organismos biológicos para construir sistemas inteligentes es anterior a la invención de las computadoras electrónicas. El cerebro humano contiene del orden de 10^{11} neuronas interconectadas mediante unas 10^{14} conexiones sinápticas. Esta red biológica de escala colosal es la responsable de funciones tan diversas como la percepción visual, la comprensión del lenguaje, el razonamiento lógico y el control motor. Comprender, aunque sea de forma simplificada, cómo opera una neurona biológica permite apreciar tanto la motivación como las limitaciones de su contraparte artificial.

Cada neurona biológica está compuesta por tres estructuras principales: el **soma** (cuerpo celular), que contiene el núcleo y la maquinaria metabólica de la célula; el **axón**, una prolongación que puede extenderse desde milímetros hasta más de un metro y que transmite señales eléctricas desde el soma hacia otras neuronas; y las **dendritas**, ramificaciones múltiples que reciben señales provenientes de los axones de otras neuronas. La comunicación entre dos neuronas se produce a través de las **sinapsis**, que son pequeños espacios entre el botón terminal de un axón y la dendrita receptora. Cuando el potencial eléctrico acumulado en el soma de una neurona supera cierto umbral, la neurona genera un impulso eléctrico, denominado potencial de acción, que se propaga a lo largo del axón hasta alcanzar los botones terminales, donde se liberan neurotransmisores que cruzan la sinapsis y alteran el potencial de la neurona receptora. Dependiendo del tipo de neurotransmisor, la sinapsis puede tener un efecto **excitador** (incrementando la probabilidad de que la neurona receptora se active) o **inhibidor** (disminuyéndola).

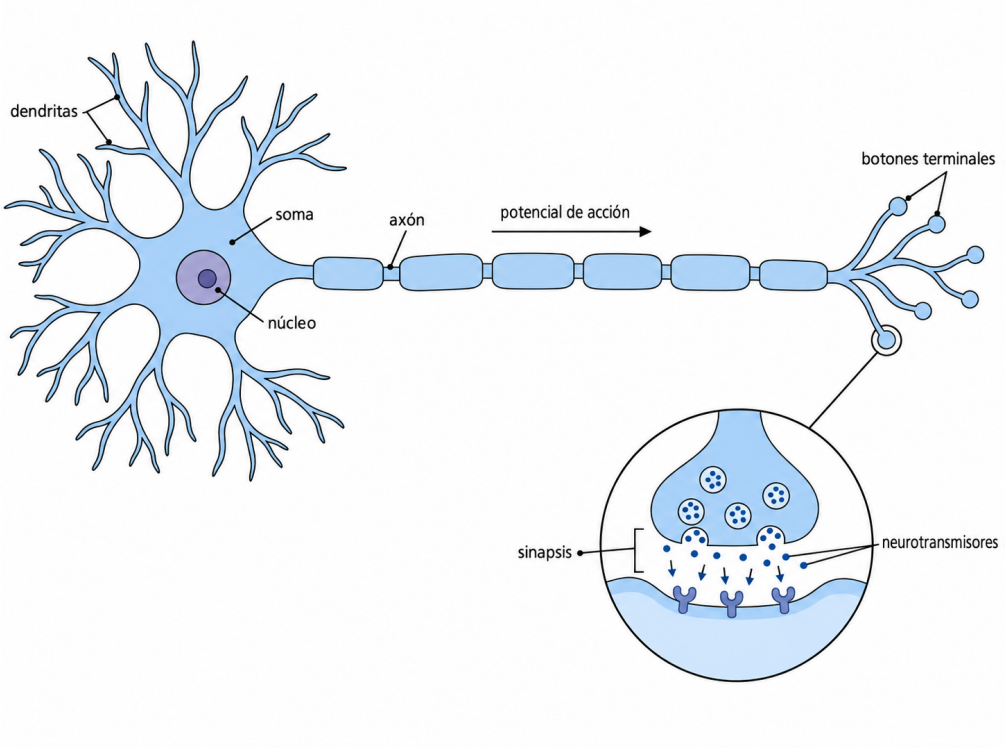


Figura 2.1: Estructura de una neurona biológica. Las dendritas reciben las señales procedentes de otras neuronas, el soma integra el potencial acumulado y el axón conduce el potencial de acción hasta los botones terminales, donde los neurotransmisores cruzan la sinapsis hacia la neurona receptora.

El aprendizaje biológico, en su descripción más elemental, consiste en el fortalecimiento o debilitamiento de estas conexiones sinápticas en función de la experiencia. Cuando un estímulo se presenta repetidamente, las sinapsis involucradas en su procesamiento se refuerzan, facilitando respuestas más rápidas y eficientes ante estímulos futuros similares. Este principio, formalizado por Donald Hebb en 1949, constituye la base conceptual sobre la que se diseñan los algoritmos de aprendizaje de las redes neuronales artificiales.

Es fundamental, no obstante, situar con honestidad las limitaciones de la analogía computacional. Las redes neuronales artificiales utilizadas en la práctica son simplificaciones extremas del sistema neuronal humano. El cerebro posee una complejidad que excede con creces la de cualquier red artificial: sus conexiones contienen ciclos que generan innumerables lazos de retroalimentación; parte de su funcionalidad es heredada genéticamente, no aprendida; el proceso de aprendizaje biológico involucra estímulos multisensoriales a lo largo de meses o años; y los mecanismos de representación del conocimiento en el cerebro son aún poco comprendidos. Las redes artificiales no son inteligentes en el sentido estricto del término: no hay en ellas ninguna representación de su propio funcionamiento, y lo que aprenden queda ligado al dominio de entrenamiento, sin posibilidad de trasladarlo por sí solas a otro distinto. Sin embargo, a pesar de estas limitaciones, su capacidad para resolver tareas complejas de clasificación, regresión y aproximación de funciones las convierte en una herramienta computacional de enorme valor práctico.

2.1.2. El Modelo Matemático de la Neurona Artificial

La unidad computacional básica de una red neuronal artificial es el **perceptrón**, propuesto por Frank Rosenblatt en 1957, inspirado en el modelo previo de la neurona de McCulloch y Pitts. Si la neurona biológica opera sumando las señales eléctricas ponderadas que recibe a través de sus dendritas y disparando un impulso cuando el resultado supera un umbral, la neurona artificial abstrae este proceso en dos operaciones matemáticas bien definidas.

Primera operación: combinación lineal con sesgo.[1] Considérese un nodo k de la red. Sean $x_1, x_2, \ldots, x_n$ las entradas a dicho nodo y $w_{k1}, w_{k2}, \ldots, w_{kn}$ los pesos asociados a cada conexión, donde el primer subíndice identifica al nodo receptor y el segundo a la entrada de la que proviene la conexión. Cada peso w_{ki} modela la intensidad de la conexión sináptica: un peso grande en valor absoluto indica una conexión fuerte; un peso positivo modela una sinapsis excitatoria y uno negativo una sinapsis inhibitoria. El nodo k calcula la suma ponderada de las entradas más un término de sesgo b_k:

$$z_k = \sum_{i=1}^{n} w_{ki}\, x_i + b_k = \mathbf{w}_k^{\top}\mathbf{x} + b_k, \tag{2.1}$$

donde se ha utilizado la notación vectorial $\mathbf{w}_k^{\top} = (w_{k1}, \ldots, w_{kn})$ y $\mathbf{x} = (x_1, \ldots, x_n)$. El papel del sesgo b_k es análogo al de la constante en una función lineal: desplaza el umbral de activación de la neurona, permitiendo que el modelo se ajuste mejor a los datos incluso cuando todas las entradas son cero. Kaveh destaca que los pesos determinan la importancia relativa de cada entrada, mientras que el sesgo permite a la función de activación desplazarse horizontalmente en el espacio vectorial para lograr una predicción más precisa.

Segunda operación: aplicación de la función de activación. El resultado z_k de la combinación lineal se transforma mediante una función no lineal σ, denominada **función de activación**, para

[1] La suma ponderada no es la única forma de integrar las entradas de un nodo: existen también las unidades *sigma-pi* o de alto orden, las de *base radial* (RBF) y las neuronas de *disparo* o *spiking*. En este texto se trabaja exclusivamente con la suma ponderada, por ser la que da origen al perceptrón y a las arquitecturas que de él se derivan.

producir la salida final del nodo:

$$a_k = \sigma(z_k) = \sigma\big(\mathbf{w}_k^\top \mathbf{x} + b_k\big). \tag{2.2}$$

La operación completa del nodo k se interpreta como la composición $\sigma \circ z_k$. La función de activación cumple un papel absolutamente determinante: es la responsable de introducir **no linealidad** en el modelo. Nelson desarrolla una demostración algebraica directa de por qué esta no linealidad es indispensable. Supóngase, para simplificar, que se tiene una red con dos capas ocultas de dos nodos cada una, y que no se aplica ninguna función de activación. Siguiendo la convención anterior, $w_{ki}^{(\ell)}$ denota el peso que va hacia el nodo k de la capa ℓ desde el nodo i de la capa precedente. Entonces la salida de la primera capa oculta sería:

$$\begin{aligned} z_1^{(1)} &= w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}, \\ z_2^{(1)} &= w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}. \end{aligned}$$

Al pasar a la segunda capa oculta, las salidas de la primera se combinan linealmente de nuevo:

$$\begin{aligned} z_1^{(2)} &= w_{11}^{(2)} z_1^{(1)} + w_{12}^{(2)} z_2^{(1)} + b_1^{(2)} \\ &= \big(w_{11}^{(2)} w_{11}^{(1)} + w_{12}^{(2)} w_{21}^{(1)}\big) x_1 + \big(w_{11}^{(2)} w_{12}^{(1)} + w_{12}^{(2)} w_{22}^{(1)}\big) x_2 + \text{constante} \\ &= \tilde{w}_1\, x_1 + \tilde{w}_2\, x_2 + \tilde{b}. \end{aligned}$$

El resultado no es más que una combinación lineal de las entradas originales con pesos y sesgo efectivos. Añadir capas sin activación no aporta nada nuevo: la función de entrenamiento permanece lineal y carece de capacidad para capturar relaciones no lineales en los datos. Esta observación, aparentemente simple, es una de las razones fundamentales por la cual la historia temprana de la IA, cuando solo se disponía de perceptrones lineales, estuvo marcada por severas limitaciones prácticas.

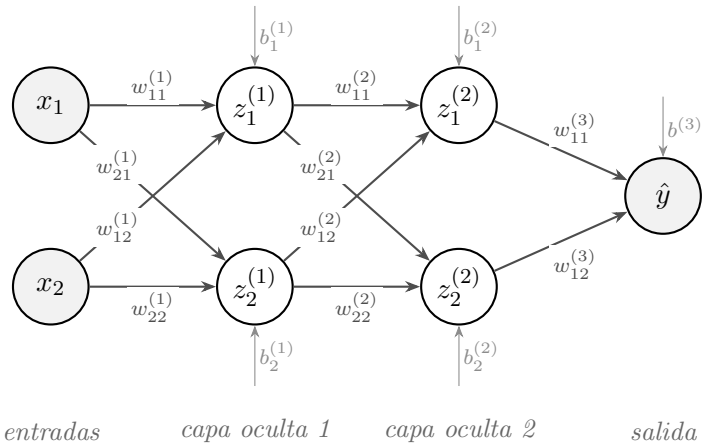


Figura 2.2: Red de dos capas ocultas con dos nodos cada una, sin funciones de activación. Cada arista representa un peso $w_{ki}^{(\ell)}$, que conduce hacia el nodo k de la capa ℓ desde el nodo i de la capa precedente. Al no involucrar no linealidad entre las capas, la composición de todas las transformaciones colapsa en una única combinación lineal de x_1 y x_2.

2.1.3. Estructura por Capas

Las neuronas artificiales no operan de forma aislada, sino que se organizan en **capas** que conforman la estructura global de la red. Toda red neuronal se compone de al menos tres tipos de capas:

Capa de entrada (*input layer*). Recibe los datos sin realizar ninguna operación de cómputo: su única función es transmitir la información hacia las capas siguientes. El número de nodos en esta capa coincide con la dimensión del vector de características de los datos. Si, por ejemplo, se alimenta la red con imágenes de 28×28 píxeles en escala de grises[2], la capa de entrada tendrá 784 nodos, uno por cada píxel. Dado que no realiza cálculos, algunos autores no la contabilizan al reportar la profundidad de la red.

Capas ocultas (*hidden layers*). Situadas entre la capa de entrada y la de salida, son las responsables de aplicar las transformaciones no lineales sucesivas que confieren a la red su poder expresivo. El término *oculta* se debe a que sus salidas no son directamente observables desde el exterior de la red: el usuario solo ve las entradas y la predicción final. El número mínimo de capas ocultas es uno, aunque la complejidad del problema puede exigir que se incrementen. Cada capa oculta recibe las activaciones de la capa anterior, calcula nuevas combinaciones lineales ponderadas, aplica la función de activación y transmite los resultados a la capa siguiente. Este proceso iterativo permite a la red construir representaciones cada vez más abstractas de los datos de entrada: las primeras capas suelen capturar características de bajo nivel (patrones locales, bordes, frecuencias elementales), mientras que las capas más profundas extraen representaciones de alto nivel (formas complejas, relaciones semánticas, estructuras globales).

Una de las primeras redes neuronales, el perceptrón original, contenía únicamente una capa oculta y se consideraba un modelo *superficial*. En la actualidad, se considera que las redes con más de tres capas ocultas se clasifican como **redes profundas** (*deep networks*), y el campo de estudio asociado se denomina **aprendizaje profundo** (*deep learning*).

Capa de salida (*output layer*). Produce el resultado final de la red. Su configuración depende de la tarea:

- **Regresión**: un solo nodo de salida sin función de activación (o con activación lineal), que emite directamente un valor numérico continuo.
- **Clasificación binaria**: un solo nodo con función sigmoide, cuya salida $\hat{y} \in (0, 1)$ se interpreta como la probabilidad de pertenencia a la clase positiva.
- **Clasificación múltiple en C clases**: C nodos de salida con función *softmax*, produciendo un vector de probabilidades que suma uno.

2.1.4. Componentes Internos de un Nodo

En la literatura se descompone la estructura interna de cada nodo de la red en cuatro componentes, cuya comprensión es esencial para el análisis del flujo de información:

1. **Entrada del nodo.** En la primera capa, las entradas son vectores numéricos de la misma dimensión que representan las características de los datos (píxeles de una imagen, valores de una serie temporal, coordenadas espaciales, etc.). En las capas ocultas y en la capa de salida,

[2]Tal es el caso del conjunto de datos MNIST, el cual suele utilizarse como *benchmark* (punto de referencia) para tareas de clasificación en visión por computadora.

las entradas son los valores de activación producidos en nodos de la capa inmediatamente anterior y que comparten una arista con esta neurona.

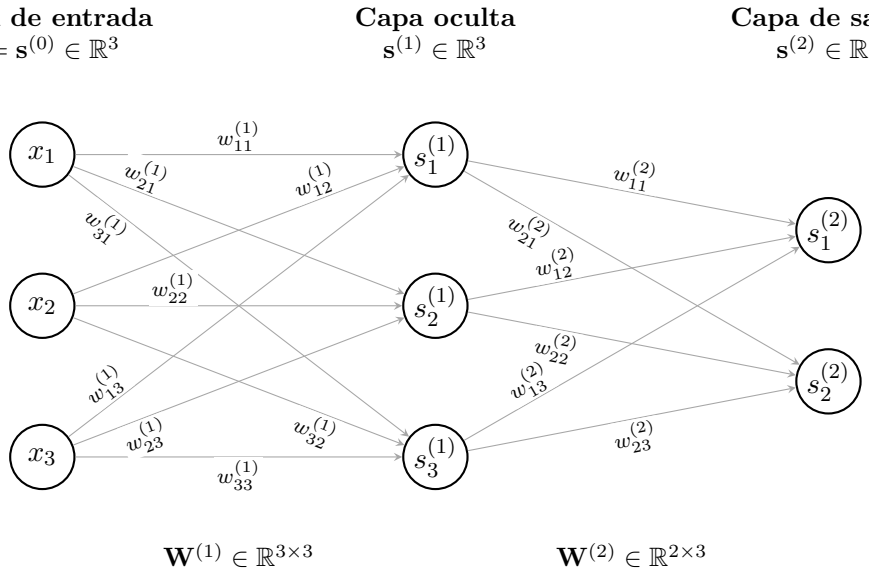
2. **Pesos de las conexiones.** Cada conexión entre dos nodos posee un peso escalar que determina la importancia de la señal transmitida. Los pesos se actualizan iterativamente durante el entrenamiento mediante el algoritmo de retropropagación, con el objetivo de minimizar la función de pérdida. La correcta predicción realizada por el modelo depende directamente de la selección apropiada de estos parámetros (pesos).

3. **Función de activación.** Recibe como argumento la combinación lineal de entradas y pesos más el sesgo, y produce la salida del nodo. Su elección determina el rango de la salida y las propiedades del gradiente, aspectos que se formalizan en la Sección 2.3.

4. **Sesgo (*bias*).** Un parámetro escalar que desplaza el punto de operación de la función de activación. Es análogo a la ordenada al origen de una recta: sin él, toda función de activación estaría centrada forzosamente en el origen, limitando la flexibilidad del modelo. Como el sesgo es un parámetro entrenable, la red aprende automáticamente cuál es el desplazamiento óptimo para cada neurona.

2.1.5. La Red como Grafo Computacional y Notación Matricial

Desde un punto de vista formal, una red neuronal es un **grafo computacional** que representa cómo se relacionan los elementos constitutivos de la red neuronal y de dónde se pueden inferir las operaciones a realizar en el *forward propagation*. El grafo se caracteriza por dos elementos: nodos y aristas. Cada arista que conecta el nodo i de la capa n con el nodo j de la capa $n+1$ posee un peso $\omega_{ji}^{(n+1)}$ asociado. Los pesos se organizan en matrices.



Consideremos, como ejemplo concreto, una red completamente conexa con tres nodos de entrada, una capa oculta de tres nodos y una capa de salida de dos nodos. Los pesos de las aristas que conectan

la entrada con la capa oculta se agrupan en la matriz:

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \end{pmatrix},$$

y los pesos que conectan la capa oculta con la capa de salida en:

$$\mathbf{W}^{(2)} = \begin{pmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} \end{pmatrix}.$$

Las combinaciones lineales que actúan como entrada de cada nodo en la capa oculta en forma matricial se escriben como:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \boldsymbol{b}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \end{pmatrix}, \tag{2.3}$$

donde $\boldsymbol{b}^{(1)} \in \mathbb{R}^3$ es el vector de sesgos de la capa oculta. A partir de este vector, la salida de la capa oculta se obtiene mediante:

$$\mathbf{s}^{(1)} = \sigma\big(\mathbf{z}^{(1)}\big) = \begin{pmatrix} \sigma(z_1^{(1)}) \\ \sigma(z_2^{(1)}) \\ \sigma(z_3^{(1)}) \end{pmatrix}, \tag{2.4}$$

donde la función de activación σ actúa componente a componente sobre $\mathbf{z}^{(1)}$.

Finalmente, para concluir el ejemplo, las combinaciones lineales y la salida correspondiente a la capa de salida se calculan como:

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{s}^{(1)} + \boldsymbol{b}^{(2)} = \begin{pmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} \end{pmatrix} \begin{pmatrix} s_1^{(1)} \\ s_2^{(1)} \\ s_3^{(1)} \end{pmatrix} + \begin{pmatrix} b_1^{(2)} \\ b_2^{(2)} \end{pmatrix}, \tag{2.5}$$

$$\mathbf{s}^{(2)} = \sigma\big(\mathbf{z}^{(2)}\big) = \begin{pmatrix} \sigma(z_1^{(2)}) \\ \sigma(z_2^{(2)}) \end{pmatrix} = \sigma\Big(\mathbf{W}^{(2)}\sigma\big(\mathbf{W}^{(1)}\mathbf{x} + \boldsymbol{b}^{(1)}\big) + \boldsymbol{b}^{(2)}\Big) \in \mathbb{R}^2, \tag{2.6}$$

donde $\boldsymbol{b}^{(2)} \in \mathbb{R}^2$ representa el vector de sesgos de la capa de salida.

En el caso general de una red con h capas ocultas, la salida de la capa k-ésima se expresa de forma recursiva como:

$$\mathbf{s}^{(k)} = \sigma\big(\mathbf{W}^{(k)}\,\mathbf{s}^{(k-1)} + \boldsymbol{b}^{(k)}\big), \qquad k = 1, 2, \ldots, h, \tag{2.7}$$

con $\mathbf{s}^{(0)} = \mathbf{x}$ el vector de entrada. La función de entrenamiento completa de la red se obtiene encadenando estas expresiones de manera telescópica:

$$\mathbf{s}^{(h)} = \sigma\Big(\mathbf{W}^{(h)}\,\sigma\big(\cdots\sigma\big(\mathbf{W}^{(2)}\,\sigma\big(\mathbf{W}^{(1)}\mathbf{x} + \boldsymbol{b}^{(1)}\big) + \boldsymbol{b}^{(2)}\big)\cdots\big) + \boldsymbol{b}^{(h)}\Big). \tag{2.8}$$

Esta formulación matricial es computacionalmente mucho más eficiente que la evaluación nodo por nodo. En la práctica, las matrices de pesos y las entradas se representan como *tensores* (la generalización de matrices a arreglos de dimensión arbitraria) y se operan mediante rutinas de álgebra lineal altamente optimizadas que residen en las unidades de procesamiento gráfico (GPU). De hecho, la denominación del framework TensorFlow proviene precisamente de este concepto: el flujo de tensores a través del grafo computacional que define la red. Asegurar que los tensores caben en la memoria de la GPU constituye, en la práctica, una de las principales restricciones que enfrenta el ingeniero al diseñar redes de gran escala.

2.2. Tipos de Redes Neuronales

A lo largo del desarrollo de la inteligencia artificial, se han propuesto diversas arquitecturas de redes neuronales, cada una diseñada para explotar características específicas de los datos y resolver distintos tipos de problemas. La clasificación principal se basa en el flujo de la información, la estructura de las conexiones y la naturaleza de la tarea. En esta sección se describen las arquitecturas más relevantes.

2.2.1. Redes Neuronales *Feedforward* (FNN) y el Perceptrón Multicapa

La arquitectura *feedforward* es la más elemental y la piedra angular sobre la que se construyen las demás. Su estructura carece de ciclos: la información fluye estrictamente desde la capa de entrada hacia la capa de salida, atravesando secuencialmente las capas ocultas, sin retroalimentación ni recurrencia. Cuando el modelo contiene múltiples capas ocultas completamente conectadas, se denomina **Perceptrón Multicapa** (*Multi-Layer Perceptron*, MLP).

En una red completamente conectada, cada neurona de una capa está conectada a todas las neuronas de la capa siguiente. El proceso que ocurre en cada neurona —combinación lineal, sesgo y activación— se repite de forma idéntica en decenas, cientos, miles o incluso millones de neuronas organizadas en capas, donde la salida de una capa actúa como entrada de la siguiente. De manera análoga a la corteza cerebral, la agregación de procesos simples y similares a lo largo de múltiples neuronas y capas produce funcionalidades mucho más complejas. Esta observación tiene una consecuencia teórica profunda: aunque cada neurona individual sea extremadamente simple —una combinación lineal seguida de una no linealidad elemental—, la composición de muchas de estas operaciones a lo largo de múltiples capas permite representar funciones de complejidad arbitraria.

Precisamente, el aporte teórico central de las FNN es el **Teorema de Aproximación Universal**, que garantiza esta capacidad expresiva. En la práctica, las FNN se utilizan ampliamente en tareas de regresión y clasificación cuando los datos de entrada son vectores de características de dimensión fija, sin estructura espacial o temporal inherente.

2.2.2. Redes Neuronales Convolucionales (CNN)

Mientras que las redes completamente conectadas tratan cada entrada como una característica independiente sin considerar su posición relativa, las **redes neuronales convolucionales** están diseñadas específicamente para procesar datos que presentan una topología de cuadrícula, como las imágenes bidimensionales o las señales unidimensionales. Su motivación surge de una observación práctica: en una imagen de 100×100 píxeles, una capa completamente conectada tendría 10 000 entradas por neurona, y una sola capa oculta de 10 000 neuronas requeriría 10^8 pesos, una cantidad prohibitiva que además propicia el sobreajuste.

La solución que ofrecen las CNN es reemplazar la multiplicación matricial global por la operación de **convolución**. En lugar de conectar cada neurona con todas las entradas, la red aplica un conjunto reducido de pesos —denominados *filtros* o *kernels*— de manera sistemática sobre regiones locales de los datos. Matemáticamente, la convolución discreta bidimensional entre una imagen I y un filtro K de tamaño $k \times k$ se define como:

$$(I * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I[i+m,\, j+n]\, K[m, n]. \tag{2.9}$$

Esta operación confiere a la red dos propiedades fundamentales:

- **Invarianza a la traslación**: un mismo filtro se aplica en toda la extensión de la entrada, por lo que un patrón detectado en una posición (por ejemplo, un borde vertical) será reconocido independientemente de su ubicación exacta en la imagen. Esta propiedad es crucial para tareas de reconocimiento visual.
- **Compartición de parámetros**: al utilizar los mismos filtros en toda la entrada, el número de parámetros entrenables se reduce drásticamente respecto a una capa completamente conectada equivalente. Esto no solo disminuye el costo computacional y de memoria, sino que actúa como una forma implícita de regularización, reduciendo el riesgo de sobreajuste.

La arquitectura típica de una CNN alterna capas convolucionales con capas de ***pooling*** (reducción de dimensionalidad espacial mediante operaciones como el máximo o el promedio sobre regiones locales), seguidas de una o más capas completamente conectadas al final para la tarea de clasificación o regresión. Los filtros de las primeras capas suelen capturar características de bajo nivel —bordes, esquinas, texturas—, mientras que las capas más profundas aprenden representaciones cada vez más abstractas y semánticas. Esta jerarquía emergente de características es una de las propiedades más notables de las CNN y la razón de su éxito en tareas de visión por computadora.

2.2.3. Redes Neuronales Recurrentes (RNN)

Las redes *feedforward* y las CNN procesan cada entrada de forma independiente: no poseen ningún mecanismo para recordar entradas anteriores. Sin embargo, muchas tareas del mundo real involucran datos secuenciales —series temporales, texto, señales de audio— donde el contexto temporal es esencial para producir una predicción correcta. Las **redes neuronales recurrentes** abordan esta limitación al introducir **conexiones cíclicas** en su arquitectura.

En una RNN, la salida que genera un nodo al procesar un elemento de la secuencia se conserva y se reintroduce como parte de la entrada al evaluar el siguiente elemento. Esta retroalimentación le confiere a la red un **estado** o **memoria interna** que evoluciona a lo largo de la secuencia. Formalmente, el estado oculto $\mathbf{h}_t$ en el paso temporal t se actualiza como:

$$\mathbf{h}_t = \sigma\big(\mathbf{W}_h\,\mathbf{h}_{t-1} + \mathbf{W}_x\,\mathbf{x}_t + \mathbf{b}\big), \tag{2.10}$$

donde $\mathbf{W}_h$ es la matriz de pesos recurrentes que conecta el estado anterior con el actual, $\mathbf{W}_x$ es la matriz de pesos de entrada, $\mathbf{x}_t$ es la entrada en el paso t y $\mathbf{b}$ el vector de sesgo. La salida $\mathbf{y}_t$ en cada paso temporal se calcula a partir del estado oculto: $\mathbf{y}_t = g(\mathbf{W}_y\mathbf{h}_t + \mathbf{b}_y)$, donde g es una función de activación apropiada para la tarea.

La principal dificultad de las RNN estándar radica en su capacidad limitada para capturar dependencias a largo plazo. Cuando una secuencia es larga, la información de los primeros pasos se diluye progresivamente al propagarse a través de muchas multiplicaciones matriciales sucesivas, dando lugar al problema del gradiente que se desvanece. Para mitigar esta limitación, se han desarrollado arquitecturas especializadas:

- **LSTM (*Long Short-Term Memory*)**: introduce una *celda de memoria* explícita y tres *compuertas* (de entrada, de olvido y de salida) que regulan selectivamente qué información se almacena, se descarta y se transmite en cada paso temporal. Este mecanismo permite a la red mantener información relevante durante cientos de pasos temporales.
- **GRU (*Gated Recurrent Unit*)**: una versión simplificada de LSTM que combina las compuertas de entrada y de olvido en una sola *compuerta de actualización* y prescinde de la celda de memoria separada. A pesar de su menor complejidad, el rendimiento de las GRU es comparable al de las LSTM en numerosas tareas.

2.2.4. Arquitecturas Basadas en Atención (*Transformers*)

Si bien las LSTM y las GRU mitigaron significativamente el problema de las dependencias a largo plazo, su naturaleza secuencial —procesar un elemento de la secuencia a la vez— limita su paralelización y, en consecuencia, su velocidad de entrenamiento en hardware moderno. Los ***Transformers***, introducidos en 2017, resuelven ambos problemas al descartar por completo la recurrencia en favor de mecanismos de ***self-attention***.

En un *Transformer*, la información de una secuencia completa se procesa de manera simultánea. El mecanismo de atención calcula dinámicamente, para cada elemento de la secuencia, un conjunto de pesos que representan la relevancia de todos los demás elementos con respecto a él, independientemente de la distancia que los separe. Matemáticamente, dado un conjunto de vectores de consulta $\mathbf{Q}$, claves $\mathbf{K}$ y valores $\mathbf{V}$ (obtenidos mediante proyecciones lineales aprendidas de la entrada), la **atención escalada por producto punto** se define como:

$$\text{Atención}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}, \tag{2.11}$$

donde d_k es la dimensión de los vectores de clave y el factor $1/\sqrt{d_k}$ estabiliza numéricamente el producto escalar, evitando que los argumentos de la función softmax se ubiquen en zonas de saturación. La salida de la operación de atención es una combinación ponderada de los vectores de valor, donde los pesos de la combinación reflejan la compatibilidad semántica entre consultas y claves.

Esta formulación permite capturar dependencias e interacciones complejas a largo plazo de forma eficiente y completamente paralelizable. Los *Transformers* han revolucionado no solo el procesamiento de lenguaje natural —dando origen a modelos como BERT, GPT y sus sucesores—, sino también la visión por computadora (Vision Transformers), el modelado de proteínas y, de manera creciente, la resolución de ecuaciones diferenciales, tema que se revisaremos en capítulos posteriores.

2.2.5. Otras Arquitecturas Relevantes

Existe un amplio espectro de arquitecturas adicionales que resulta pertinente mencionar brevemente, ya que aparecen frecuentemente en la literatura y en las aplicaciones prácticas del aprendizaje profundo:

Autocodificadores (*Autoencoders*). Redes diseñadas para aprender representaciones comprimidas (codificación) de los datos de entrada, de las cuales luego se reconstruye la entrada original (decodificación). La capa central, denominada *cuello de botella* (*bottleneck*), tiene una dimensión menor que la entrada, lo que obliga a la red a capturar únicamente la información más esencial. Sus variantes incluyen los autocodificadores variacionales (VAE), que imponen una distribución probabilística sobre el espacio latente, los *denoising autoencoders*, que aprenden a reconstruir datos a partir de versiones corrompidas por ruido, y los *sparse autoencoders*, que penalizan las activaciones densas para promover representaciones dispersas.

Redes Generativas Adversarias (GAN). Arquitectura compuesta por dos redes que compiten en un esquema de juego de suma cero: un *generador* que produce datos sintéticos intentando engañar al *discriminador*, y un discriminador que intenta distinguir los datos reales de los generados. Este entrenamiento adversario produce, en el equilibrio, un generador capaz de crear muestras sintéticas de alta calidad, prácticamente indistinguibles de los datos reales.

Redes Residuales (ResNet). Introducen **conexiones de atajo** (*skip connections*) que permiten al gradiente fluir directamente a través de las capas sin pasar por las transformaciones no lineales

intermedias. Formalmente, en lugar de aprender la función $\mathbf{s}^{(k)} = \sigma(\mathbf{W}^{(k)}\mathbf{s}^{(k-1)}+\boldsymbol{b}^{(k)})$, la red aprende el *residuo* $\mathbf{s}^{(k)} = \sigma(\mathbf{W}^{(k)}\mathbf{s}^{(k-1)} + \boldsymbol{b}^{(k)}) + \mathbf{s}^{(k-1)}$. Esta innovación, aparentemente simple, resolvió el problema de la degradación en redes muy profundas (de 50, 100 o más capas) y constituye uno de los avances arquitectónicos más influyentes de la última década.

Máquinas de Boltzmann Restringidas (RBM). Modelos generativos estocásticos con una capa visible y una capa oculta, sin conexiones internas dentro de una misma capa. Fueron fundamentales en el desarrollo temprano del aprendizaje profundo como bloques constitutivos de las *Deep Belief Networks*.

Máquinas de Aprendizaje Extremo (ELM). Redes *feedforward* de una sola capa oculta en las que los pesos de la primera capa se asignan aleatoriamente y se mantienen fijos; únicamente se entrenan los pesos de la capa de salida. Esto permite un entrenamiento extremadamente rápido mediante técnicas de álgebra lineal, aunque a costa de una menor expresividad.

La elección del tipo de red, así como la definición de sus hiperparámetros, depende directamente de la naturaleza de los datos y del problema a resolver. Sin embargo, en cualquier arquitectura, la capacidad real de la red para modelar fenómenos complejos recae en las transformaciones no lineales aplicadas en cada neurona. Estas transformaciones están definidas por las funciones de activación.

2.3. Funciones de Activación

Como se estableció en las secciones anteriores, el propósito fundamental de la función de activación σ es introducir propiedades no lineales en la red neuronal. Sin este componente, independientemente de la cantidad de capas ocultas que posea el modelo, la red se reduciría matemáticamente a una simple transformación lineal, perdiendo toda capacidad de modelar características complejas.

Se suelen clasificar en tres categorías generales: funciones de escalón binario, funciones lineales y funciones no lineales. Las funciones de escalón binario devuelven únicamente los valores 0 o 1 según el signo de la entrada; aunque modelan el concepto de umbral neuronal, carecen de gradientes continuos y resultan inviables para el entrenamiento por retropropagación. Las funciones lineales, de la forma $f(x) = x$, no añaden no linealidad al modelo y, por las razones algebraicas ya expuestas, resultan insuficientes en redes profundas: la composición de múltiples funciones lineales sigue siendo lineal. Por lo tanto, en la práctica, las funciones que se utilizan en las capas ocultas son exclusivamente **no lineales**.

La elección específica de la función de activación depende de la naturaleza del problema, del rango de salida deseado y, de manera crucial, del comportamiento de su derivada. Mantener un gradiente de error significativo en las regiones de operación es indispensable para que los algoritmos de optimización puedan actualizar los pesos del modelo de manera efectiva. A continuación se definen las funciones de activación más relevantes, agrupadas por familia, acompañadas de sus derivadas y de un análisis de sus ventajas y limitaciones.

2.3.1. Funciones de Tipo Sigmoidal

Las funciones de tipo sigmoidal se inspiran en la neurociencia computacional, donde modelan el umbral de activación de una célula nerviosa. Sus gráficas comparten una forma de "S": producen salidas acotadas en un intervalo fijo, lo cual es útil para interpretar las salidas como probabilidades, pero se saturan para valores de entrada grandes en magnitud. Esta saturación es la causa directa del problema del gradiente que se desvanece.

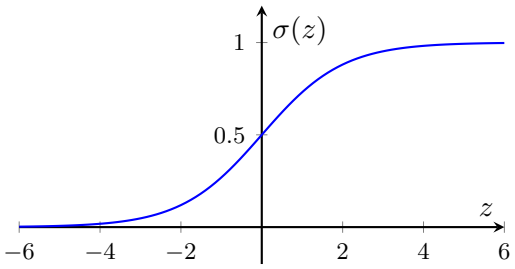
Función Logística (Sigmoide)

La función sigmoide es una de las funciones de activación más antiguas y estudiadas. Es una función monotónica que proyecta cualquier número real en el intervalo abierto $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{2.12}$$

Su derivada presenta una forma particularmente elegante, expresable en términos de la propia función:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big). \tag{2.13}$$



Obsérvese que $\sigma'(z)$ alcanza su valor máximo de 0.25 en $z = 0$, lo que significa que incluso en el mejor caso, cada capa atenúa el gradiente por un factor de al menos $1/4$. Al encadenar n capas, el gradiente se multiplica por factores $\leq 0.25^n$, lo que explica por qué la sigmoide resulta problemática en redes profundas. Además, la salida de la sigmoide siempre es positiva, lo que provoca que los gradientes respecto a los pesos tengan todos el mismo signo, dificultando la convergencia del proceso de optimización.

La sigmoide se sigue utilizando principalmente en la capa de salida de modelos de clasificación binaria, donde su rango $(0, 1)$ es interpretable como la probabilidad de pertenencia a la clase positiva, y donde su efecto de saturación es irrelevante para el cálculo de gradientes en capas anteriores.

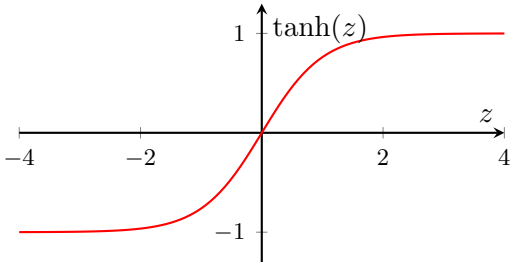
Tangente Hiperbólica (`tanh`)

Es una versión escalada y desplazada de la función sigmoide, con rango $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{2}{1 + e^{-2z}} - 1 = 2\sigma(2z) - 1. \tag{2.14}$$

La última igualdad evidencia la relación directa con la sigmoide. Su derivada es:

$$\tanh'(z) = 1 - \tanh^2(z). \tag{2.15}$$



El valor máximo de $\tanh'(z)$ es 1 (en $z = 0$), mejor que el 0.25 de la sigmoide. Además, al estar centrada en cero, la función tanh produce salidas con media más cercana a cero, lo que reduce

el sesgo en los gradientes respecto a los pesos y favorece la convergencia del entrenamiento. El problema de saturación de salidas se reduce comparado con la sigmoide, aunque no se elimina: para $|z| > 3$, la derivada se aproxima a cero.

Arco Tangente (`arctan`)

Con propiedades similares a la tangente hiperbólica, pero rango $(-\pi/2, \pi/2)$:

$$f(z) = \arctan(z), \qquad f'(z) = \frac{1}{1+z^2}. \tag{2.16}$$

Su derivada es estrictamente positiva en todo $\mathbb{R}$, lo que garantiza que el gradiente nunca sea exactamente cero. Sin embargo, para valores grandes de $|z|$, $f'(z) \approx 1/z^2 \to 0$, por lo que la saturación también se presenta, aunque de forma más suave que en la sigmoide y la tanh. Esta función se utiliza con menor frecuencia en la práctica, pero resulta pertinente mencionarla como otro miembro de la familia sigmoidal.

2.3.2. Funciones de Tipo Rectificador

La segunda familia de funciones de activación aborda directamente el problema de saturación que afecta a las funciones sigmoidales. El principio es simple: si el gradiente se desvanece porque la derivada se acerca a cero en las colas, ¿por qué no utilizar funciones cuya derivada sea constante (y no nula) en al menos una porción del dominio? Las funciones de tipo rectificador implementan esta idea, y su adopción representó un antes y un después en la viabilidad práctica de las redes profundas.

Unidad Lineal Rectificada (ReLU)

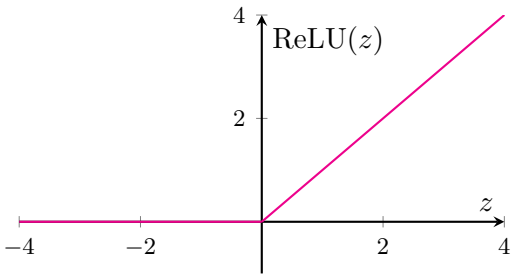
La función ReLU es, por amplio margen, la más utilizada en la práctica actual. Nelson señala que el éxito de la red AlexNet en 2012 —que marcó el inicio de la era del aprendizaje profundo aplicado a la visión por computadora— se atribuye en parte a la adopción de ReLU en lugar de las funciones sigmoidales que eran estándar hasta entonces:

$$\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} 0 & \text{si } z < 0, \\ z & \text{si } z \geq 0. \end{cases} \tag{2.17}$$

Su derivada es:

$$\mathrm{ReLU}'(z) = \begin{cases} 0 & \text{si } z < 0, \\ 1 & \text{si } z > 0, \end{cases} \tag{2.18}$$

(con la derivada indefinida en $z = 0$; en la práctica, se asigna convencionalmente el valor 0 o 1).



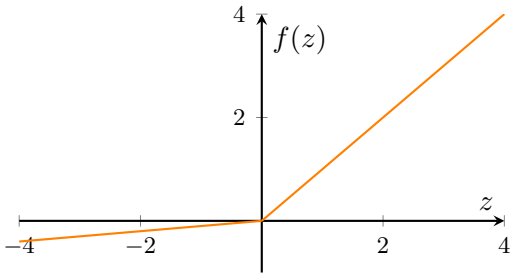
Las ventajas de ReLU son múltiples. Su cálculo es extremadamente eficiente —una simple comparación con cero— y su derivada es constante e igual a 1 en la región activa, lo que permite que el gradiente fluya sin atenuación a través de las capas durante la retropropagación. La función no satura en la región positiva, acelerando significativamente la convergencia del descenso por gradiente. Además, al producir exactamente cero para entradas negativas, ReLU genera activaciones *dispersas*, lo que reduce la carga computacional y puede mejorar la capacidad de generalización.

Aún así, ReLU tiene una limitación, el fenómeno de **neuronas muertas**. Cuando una neurona recibe consistentemente entradas negativas —lo que puede ocurrir si los pesos se inicializan de forma desfavorable o si una actualización de gradiente particularmente grande desplaza los pesos a una región donde la preactivación es siempre negativa—, su salida y su gradiente son permanentemente cero. La neurona deja de contribuir al modelo y sus pesos nunca se actualizan. Si este fenómeno afecta a una fracción significativa de las neuronas, la capacidad efectiva de la red se reduce drásticamente y el aprendizaje se degrada.

Leaky ReLU y PReLU

Para resolver el problema de las neuronas muertas, la Leaky ReLU permite una pequeña pendiente α (típicamente $\alpha = 0.01$) en la región negativa, garantizando que el gradiente nunca sea exactamente cero:

$$f(z) = \begin{cases} \alpha z & \text{si } z < 0, \\ z & \text{si } z \geq 0, \end{cases} \qquad f'(z) = \begin{cases} \alpha & \text{si } z < 0, \\ 1 & \text{si } z > 0. \end{cases} \tag{2.19}$$



La variante **PReLU** (*Parametric ReLU*) trata al coeficiente α como un parámetro entrenable. A diferencia de Leaky ReLU donde α se fija como un hiperparámetro antes del entrenamiento, en PReLU la pendiente de la región negativa se ajusta durante la retropropagación, permitiendo a la red aprender el valor óptimo de α para cada neurona. Si bien esto añade parámetros adicionales al modelo, la mejora en rendimiento puede ser significativa, especialmente en redes donde las entradas a las funciones de activación tienen distribuciones muy variadas.

Unidad Lineal Exponencial (ELU)

Mientras que Leaky ReLU y PReLU utilizan una recta para la región negativa, la ELU define una curva exponencial suave que se aproxima asintóticamente al valor $-\alpha$:

$$f(z) = \begin{cases} \alpha(e^z - 1) & \text{si } z < 0, \\ z & \text{si } z \geq 0, \end{cases} \qquad f'(z) = \begin{cases} f(z) + \alpha & \text{si } z < 0, \\ 1 & \text{si } z \geq 0. \end{cases} \tag{2.20}$$

Podemos destacar dos ventajas de ELU respecto a las variantes lineales por tramos. Primero, la pendiente de la curva en la región negativa es suave y progresiva, lo que evita discontinuidades en la derivada que podrían causar oscilaciones durante la optimización. Segundo, los valores negativos acotados ($f(z) > -\alpha$) producen activaciones con media más cercana a cero, lo que favorece la

normalización natural de las señales a lo largo de la red. Su principal desventaja es el mayor costo computacional derivado del cálculo de la función exponencial.

Softplus

Constituye una aproximación suave y diferenciable en todo $\mathbb{R}$ de la función ReLU:

$$f(z) = \ln(1 + e^z), \qquad f'(z) = \frac{1}{1 + e^{-z}} = \sigma(z). \tag{2.21}$$

Este resultado es notable: la derivada de la función Softplus es exactamente la función sigmoide. Esta conexión pone de manifiesto la relación profunda entre ambas familias de funciones de activación. Softplus comparte las ventajas de ReLU (no satura en la región positiva), pero al ser suave en $z = 0$, evita la discontinuidad de la derivada que ReLU presenta en ese punto. Su uso puede mitigar el problema de gradiente que se desvanece, aunque el costo computacional del logaritmo y la exponencial la hace ligeramente más costosa que ReLU.

2.3.3. Funciones de Activación Avanzadas

La investigación en funciones de activación continúa produciendo propuestas que buscan combinar las mejores propiedades de las familias sigmoidal y rectificadora. Las siguientes funciones, más recientes, han ganado adopción en modelos de vanguardia.

Swish

Propuesta en 2017 por investigadores de Google Brain, se define como el producto de la entrada por su propia función sigmoide:

$$f(z) = z \cdot \sigma(z) = \frac{z}{1 + e^{-z}}. \tag{2.22}$$

A diferencia de ReLU, Swish es una función continua, diferenciable y **no monótona**: para valores ligeramente negativos, la función produce salidas negativas antes de converger a cero, lo que puede resultar beneficioso para el aprendizaje, ya que preserva algo de información de las entradas negativas en lugar de descartarla completamente como hace ReLU. La curva de Swish cambia de dirección de forma suave alrededor de cero, sin el quiebre abrupto que presenta ReLU. Puesto que Swish incorpora la sigmoide en su definición, sus cálculos son más costosos que los de ReLU, pero numerosos experimentos han demostrado un rendimiento comparable o superior en redes profundas, particularmente en tareas de clasificación de imágenes.

GELU (*Gaussian Error Linear Unit*)

Introducida en 2016 y popularizada con los modelos Transformer, combina características de ReLU, del *dropout* y del *zoneout* (regularizador de redes recurrentes). Se define mediante la función de distribución acumulada de la distribución normal estándar $\Phi(z)$:

$$\mathrm{GELU}(z) = z \cdot \Phi(z) \approx 0.5\, z \left(1 + \tanh\!\left[\sqrt{2/\pi}\,(z + 0.044715\, z^3)\right]\right). \tag{2.23}$$

Para valores positivos grandes, $\Phi(z) \approx 1$ y la función se comporta como la identidad; para valores negativos grandes, $\Phi(z) \approx 0$ y la función se anula; en la zona de transición, la ponderación probabilística de $\Phi(z)$ produce una supresión gradual. En la práctica, GELU ha demostrado un rendimiento superior en los modelos de lenguaje más avanzados (BERT, RoBERTa, ALBERT, GPT) y evita el problema de gradiente que se desvanece al no saturar en la región positiva.

SELU (*Scaled Exponential Linear Unit*)

Es una versión escalada de ELU diseñada para producir un efecto **autonormalizador** en la red:

$$f(z) = \begin{cases} \lambda\alpha(e^z - 1) & \text{si } z < 0, \\ \lambda z & \text{si } z \geq 0, \end{cases} \tag{2.24}$$

donde $\lambda \approx 1.0507$ y $\alpha \approx 1.6733$ son constantes derivadas analíticamente.

La propiedad notable de SELU es que, bajo ciertas condiciones sobre la inicialización de los pesos (distribución normal con media cero y varianza $1/n$, donde n es el número de entradas), las activaciones de cada capa mantienen automáticamente media cero y varianza unitaria a medida que se propagan a través de la red. Esta autonormalización elimina la necesidad de técnicas explícitas como la normalización por lotes, y que los problemas de gradiente que se desvanece y gradiente que explota no se presentan en redes que utilizan SELU correctamente. No obstante, dado que es una función relativamente reciente, su rendimiento en todas las arquitecturas posibles aún se encuentra bajo evaluación.

Mish

Es una función de activación suave y autoregularizada definida como:

$$f(z) = z \cdot \tanh\big(\text{softplus}(z)\big) = z \cdot \tanh\big(\ln(1 + e^z)\big). \tag{2.25}$$

Al igual que Swish, Mish es no monótona, continua y diferenciable en todo $\mathbb{R}$. Es una función que combina las propiedades suaves de la sigmoide y la tangente hiperbólica con las características innovadoras de ReLU. Aunque los estudios comparativos son aún limitados, Mish ha mostrado mejoras empíricas sobre ReLU en diversas tareas de visión por computadora, particularmente en redes profundas con residuos.

2.3.4. Función Softmax (Capa de Salida)

A diferencia de todas las funciones anteriores, que se aplican nodo a nodo de forma independiente, la función **softmax** opera sobre un vector completo y se utiliza exclusivamente en la capa de salida para tareas de clasificación múltiple. Dado un vector $\mathbf{z} = (z_1, \ldots, z_C)^\top$ de puntajes crudos para C clases, la función softmax transforma estos puntajes en una distribución de probabilidad válida:

$$\text{softmax}(z_c) = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}, \qquad c = 1, \ldots, C. \tag{2.26}$$

La motivación de softmax parte de una limitación de la sigmoide: si se aplicara la sigmoide de forma independiente a cada nodo de salida, podría ocurrir que varias salidas fueran simultáneamente cercanas a 1 (por ejemplo, 0.8, 0.9, 0.6), y la suma excedería 1, violando la interpretación probabilística. Softmax resuelve este problema al normalizar las salidas exponenciadas para que sumen exactamente 1: $\sum_{c=1}^{C} \text{softmax}(z_c) = 1$ y $\text{softmax}(z_c) > 0$ para todo c. En términos prácticos, cada componente de la salida softmax se interpreta como la probabilidad estimada de que la entrada pertenezca a la clase c. La clase predicha es aquella con la mayor probabilidad.

En términos generales, softmax es una generalización natural de la regresión logística a más de dos clases, y que, junto con la función de pérdida de entropía cruzada categórica, conforma la configuración estándar para la capa de salida de cualquier red de clasificación múltiple.

2.3.5. Criterios de Selección y Resumen Comparativo

Desde el punto de vista teórico, gracias a los teoremas de aproximación universal, la elección de la función de activación no debería importar, pues cualquier función no lineal es suficiente. Sin embargo, en la práctica, la elección tiene un impacto operativo considerable. Las funciones elementales —como ReLU, Leaky ReLU y sus variantes— son preferidas porque tanto su evaluación como el cálculo de sus derivadas involucran operaciones simples y eficientes, lo cual es crítico dado que el entrenamiento de una red implica millones o miles de millones de evaluaciones.

Cada nueva familia de funciones de activación que se introduce resuelve ciertos problemas e introduce otros. Las funciones sigmoidales resolvieron la linealidad del perceptrón original, pero introdujeron el gradiente que se desvanece. ReLU resolvió la saturación, pero introdujo las neuronas muertas. Leaky ReLU y ELU resolvieron las neuronas muertas, pero pueden provocar el gradiente que explota. La lección fundamental es que no existe un truco en la elección: se requiere comprender los principios, experimentar sistemáticamente y mantener una actitud abierta ante mejoras o enfoques radicalmente diferentes.

La siguiente tabla resume las propiedades comparativas de las principales funciones:

Función	**Rango**	**Saturación**	**Centrada en 0**	**Suave**	**Costo**
Sigmoide	$(0, 1)$	Sí	No	Sí	Medio
tanh	$(-1, 1)$	Sí	Sí	Sí	Medio
ReLU	$[0, +\infty)$	Parcial	No	No	Bajo
Leaky ReLU	$(-\infty, +\infty)$	No	No	No	Bajo
ELU	$(-\alpha, +\infty)$	No	Aprox.	Sí	Alto
Softplus	$(0, +\infty)$	No	No	Sí	Medio
Swish	$\approx (-0.28, +\infty)$	No	No	Sí	Medio
GELU	$\approx (-0.17, +\infty)$	No	No	Sí	Alto
SELU	$(-\lambda\alpha, +\infty)$	No	Sí	Sí	Alto
Mish	$\approx (-0.31, +\infty)$	No	No	Sí	Alto

2.4. Funciones de Pérdida

La **función de pérdida** $\mathcal{L}$ es el criterio cuantitativo que mide la discrepancia entre las predicciones de la red $\hat{y}$ y los valores reales y. El proceso de entrenamiento de una red neuronal consiste, en su esencia, en encontrar el conjunto de pesos que minimiza esta función sobre el conjunto de datos de entrenamiento. Su elección adecuada es, por lo tanto, tan importante como la elección de la arquitectura misma.

Normalmente, la estructura matemática del entrenamiento de redes neuronales sigue siempre el mismo patrón: se define una función de entrenamiento (la red), se define una función de pérdida, y se optimiza. Este patrón es idéntico al de los modelos clásicos como la regresión lineal, la regresión logística y las máquinas de soporte vectorial; la diferencia radica en la complejidad de la función de entrenamiento.

2.4.1. Error Cuadrático Medio (MSE)

Para problemas de **regresión**, donde la red predice un valor numérico continuo, la función de pérdida estándar es el error cuadrático medio:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}\big(y^{(i)} - \hat{y}^{(i)}\big)^2, \tag{2.27}$$

donde m es el número de muestras de entrenamiento, $y^{(i)}$ es el valor real de la i-ésima muestra y $\hat{y}^{(i)}$ es la predicción de la red. Esta función es convexa, continuamente diferenciable y tiene un mínimo global claro, lo que simplifica la optimización. Penaliza más severamente los errores grandes (por el efecto cuadrático), lo que la hace sensible a outliers.

2.4.2. Entropía Cruzada Binaria

Para problemas de **clasificación binaria**, donde la red produce una salida sigmoide $\hat{y} \in (0, 1)$ interpretada como la probabilidad de pertenencia a la clase positiva, la función de pérdida es la entropía cruzada binaria:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})\Big]. \tag{2.28}$$

La intuición detrás de esta fórmula es directa: cuando la etiqueta real es $y = 1$, la pérdida es $-\ln \hat{y}$, que es grande cuando $\hat{y}$ es cercano a 0 (predicción incorrecta con alta confianza) y cercano a 0 cuando $\hat{y}$ se aproxima a 1 (predicción correcta). Simétricamente, cuando $y = 0$, la pérdida penaliza las predicciones $\hat{y}$ cercanas a 1.

2.4.3. Entropía Cruzada Categórica

Para **clasificación múltiple en C clases**, la red produce un vector de probabilidades mediante softmax, y la función de pérdida se generaliza a:

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=1}^{C} y_c^{(i)} \ln \hat{y}_c^{(i)}, \tag{2.29}$$

donde $y_c^{(i)}$ es la representación *one-hot* de la etiqueta real (un vector que vale 1 en la posición de la clase correcta y 0 en las demás) y $\hat{y}_c^{(i)}$ es la probabilidad estimada por la red para la clase c.

Desde un punto de vista probabilístico, minimizar la entropía cruzada es equivalente a maximizar la **verosimilitud** del modelo con respecto a los datos observados. Esta equivalencia proporciona una justificación teórica sólida para la elección de la entropía cruzada como función de pérdida en tareas de clasificación.

2.5. Técnicas de Regularización

Las redes neuronales, en virtud de su enorme cantidad de parámetros, son especialmente susceptibles al **sobreajuste**: el modelo memoriza los datos de entrenamiento —incluyendo su ruido inherente— pero generaliza de forma deficiente a datos no vistos. El sobreajuste se manifiesta como una discrepancia creciente entre el error de entrenamiento (que disminuye) y el error de validación (que comienza a aumentar). Las técnicas de regularización imponen restricciones adicionales al modelo para favorecer soluciones más simples, suaves y generalizables.

2.5.1. Penalización de la Norma de los Pesos

La estrategia más directa consiste en añadir un término de penalización a la función de pérdida que castigue la magnitud de los pesos:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \alpha\,\Omega(\mathbf{w}),$$

donde $\alpha > 0$ es el hiperparámetro de regularización y $\Omega(\mathbf{w})$ es la función de penalización.

La **regularización** ℓ^2 (también llamada *Ridge* o *weight decay*) utiliza $\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_j w_j^2$. Esta penalización favorece pesos pequeños y distribuidos uniformemente, produciendo funciones de entrenamiento más suaves. El efecto práctico de la regularización ℓ^2 es "empujar" todos los pesos hacia cero sin anularlos completamente, lo que reduce la varianza del modelo a costa de un ligero aumento en el sesgo.

La **regularización** ℓ^1 (o *Lasso*) utiliza $\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_j |w_j|$. A diferencia de ℓ^2, esta penalización promueve activamente la dispersión del vector de pesos: muchos pesos se anulan exactamente, lo que equivale a eliminar conexiones de la red y constituye una forma implícita de selección de características. El hiperparámetro α controla la intensidad de la regularización en ambos casos: valores grandes producen modelos más simples pero con mayor riesgo de subajuste.

2.5.2. Dropout

Durante el entrenamiento, el *dropout* consiste en desactivar aleatoriamente una fracción p de las neuronas de cada capa oculta en cada paso de actualización de pesos. En cada iteración, se genera una máscara binaria aleatoria que anula las salidas de las neuronas seleccionadas, obligando a las neuronas restantes a aprender representaciones redundantes y robustas que no dependan de la co-activación de neuronas específicas.

Conceptualmente, el *dropout* puede interpretarse como un **ensamble implícito** de múltiples sub-redes entrenadas simultáneamente: en cada paso de entrenamiento, se utiliza una sub-red diferente (definida por la máscara de *dropout*), y la predicción final durante la inferencia se obtiene como un promedio implícito sobre todas las posibles sub-redes. Durante la fase de inferencia, todas las neuronas se mantienen activas, pero sus salidas se escalan por $(1 - p)$ para compensar el hecho de que durante el entrenamiento solo una fracción de las neuronas estuvo activa.

2.5.3. Detención Temprana

Esta técnica monitorea la función de pérdida evaluada sobre un conjunto de **validación** —un subconjunto de datos separado que no se utiliza para actualizar los pesos—. Mientras la pérdida de validación disminuye, el entrenamiento continúa. Cuando la pérdida de validación deja de mejorar durante un número determinado de epochs consecutivas, o cuando comienza a incrementarse de forma sostenida —señal inequívoca de que el modelo está comenzando a memorizar los datos de entrenamiento—, se detiene el entrenamiento y se conservan los pesos correspondientes al punto de mejor rendimiento en validación.

La detención temprana es una de las técnicas de regularización más simples de implementar y más efectivas en la práctica. Su costo computacional adicional es mínimo (solo requiere evaluar la red sobre el conjunto de validación al final de cada época) y no requiere modificar la arquitectura de la red ni la función de pérdida.

2.5.4. Normalización por Lotes

Consiste en normalizar las preactivaciones de cada capa (es decir, los valores $\mathbf{z}^{(k)}$ *antes* de aplicar la función de activación) para que mantengan media cero y varianza unitaria dentro de cada

mini-lote de entrenamiento:

$$\hat{z}_j = \frac{z_j - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

donde μ_B y σ_B^2 son la media y la varianza del mini-lote respectivamente, y ϵ es una constante pequeña que previene la división por cero. La red aprende adicionalmente dos parámetros por neurona, γ y β, que permiten reescalar y desplazar la salida normalizada: $\tilde{z}_j = \gamma \hat{z}_j + \beta$.

La normalización por lotes estabiliza y acelera significativamente el proceso de optimización, reduce la sensibilidad a la inicialización de los pesos, permite el uso de tasas de aprendizaje más grandes, y tiene un efecto regularizador implícito: puesto que la media y la varianza del mini-lote son estimaciones ruidosas de los verdaderos momentos, introducen una perturbación estocástica en las activaciones que actúa de forma similar al *dropout*.

2.6. Saturación y Normalización de los Datos

La utilidad práctica de cada función de activación está condicionada por sus **zonas de saturación**. La saturación ocurre cuando la función alcanza regiones de su dominio donde la derivada es muy cercana a cero; en este estado, la salida apenas cambia aunque la entrada varíe significativamente, y el gradiente transmitido hacia las capas anteriores se atenúa hasta volverse irrelevante para la actualización de pesos. Este fenómeno del gradiente que se desvanece, tiene una manifestación directa y práctica: si los datos de entrada no se preprocesan adecuadamente, las preactivaciones z pueden ubicarse sistemáticamente en las colas de la función de activación, donde la saturación es máxima.

Considérese, por ejemplo, la función sigmoide. Esta presenta un comportamiento óptimo y no saturado cuando sus entradas se encuentran en un rango cercano a $[-1, 1]$. Para $z = 5$, $\sigma'(5) \approx 0.0066$; para $z = 10$, $\sigma'(10) \approx 4.5 \times 10^{-5}$: el gradiente prácticamente desaparece. Un fenómeno análogo ocurre con la tangente hiperbólica: para $|z| > 3$, $\tanh'(z) < 0.01$.

Es por esta razón que, antes de alimentar una red neuronal, los datos de entrada deben someterse a un proceso de **normalización** o **estandarización**. El objetivo es reescalar los valores de las características para que se ubiquen en rangos donde la función de activación opera de forma no saturada y donde el gradiente es suficientemente grande para un aprendizaje efectivo. Las dos estrategias más comunes son:

Escalado min-max. Proyecta los datos al intervalo $[0, 1]$ o $[-1, 1]$ mediante la transformación:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}},$$

donde $x_{\text{mín}}$ y $x_{\text{máx}}$ son los valores mínimo y máximo de la característica en el conjunto de datos. Esta estrategia es natural cuando la función de activación tiene un rango acotado que coincide con el intervalo de normalización: el intervalo $[0, 1]$ para la sigmoide, y el intervalo $[-1, 1]$ para la tanh.

Estandarización (z-score). Centra los datos en media cero y desviación estándar unitaria:

$$x_{\text{std}} = \frac{x - \mu}{\sigma_x},$$

donde μ y σ_x son la media y la desviación estándar de la característica en el conjunto de entrenamiento. Esta transformación no garantiza que los valores queden en un intervalo fijo, pero asegura que la distribución de las entradas tenga propiedades estadísticas favorables para el aprendizaje. En el contexto de redes profundas con ReLU, la estandarización z-score es la práctica más habitual.

Es importante subrayar que los parámetros de la normalización ($x_{\text{mín}}$, $x_{\text{máx}}$, μ, σ_x) deben calcularse **exclusivamente sobre el conjunto de entrenamiento** y aplicarse de forma idéntica al conjunto de validación y al conjunto de prueba. Calcular estos parámetros sobre el conjunto completo (incluyendo datos de prueba) constituye una fuga de información que invalida la evaluación del modelo.

Con los fundamentos teóricos establecidos en este capítulo se dispone del marco conceptual completo para abordar la **operación práctica** de una red neuronal.

Capítulo 3

Operación Práctica de una Red Neuronal

Establecidos los fundamentos teóricos del aprendizaje profundo en el capítulo anterior, abordaremos la mecánica computacional que rige el funcionamiento interno de una red neuronal. El propósito es revisar el ciclo completo mediante el cual una red *feedforward* aprende a aproximar una función real-valuada de variable real. Para ello se utilizará, como caso de estudio recurrente, la aproximación de la función analítica

$$f(x) = \sin(x), \qquad x \in [-2\pi,\, 2\pi]. \tag{3.1}$$

Con nuestra elección: f es no lineal, suave, periódica y posee derivadas acotadas, características que la convierten en un banco de pruebas ideal para ilustrar la capacidad expresiva garantizada por el Teorema de Aproximación Universal. La arquitectura adoptada es deliberadamente sencilla y consta de una neurona en la capa de entrada, dos capas ocultas con n_1 y n_2 neuronas respectivamente, y una neurona en la capa de salida con activación lineal, tal como se ilustra esquemáticamente en la Figura 3.1.

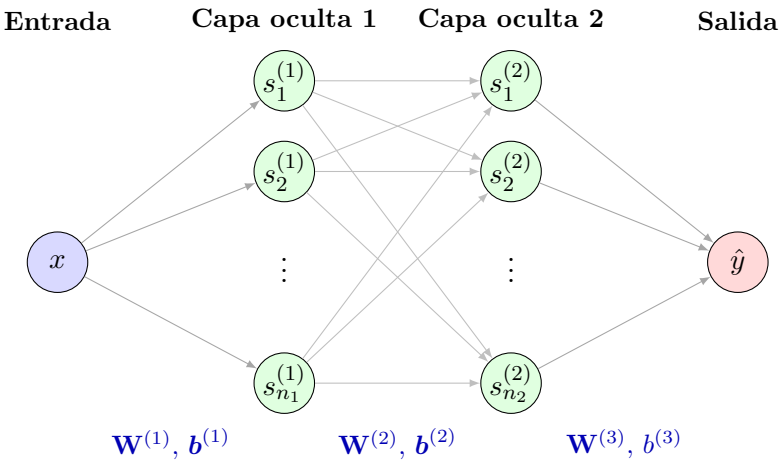


Figura 3.1: Arquitectura general de la red neuronal $1 \to n_1 \to n_2 \to 1$ utilizada en este capítulo. La información fluye de izquierda a derecha; cada flecha representa una conexión ponderada por un elemento de la matriz $\mathbf{W}^{(k)}$ correspondiente.

3.1. Evaluación Directa (Forward Propagation)

La evaluación directa es el procedimiento mediante el cual un dato de entrada se transforma, capa por capa, hasta producir la salida de la red. Bajo la arquitectura descrita, sea $x \in \mathbb{R}$ el dato de entrada. Los pesos y sesgos de la red se organizan de la siguiente manera:

$$\mathbf{W}^{(1)} \in \mathbb{R}^{n_1 \times 1}, \quad \boldsymbol{b}^{(1)} \in \mathbb{R}^{n_1}, \quad \mathbf{W}^{(2)} \in \mathbb{R}^{n_2 \times n_1}, \quad \boldsymbol{b}^{(2)} \in \mathbb{R}^{n_2}, \quad \mathbf{W}^{(3)} \in \mathbb{R}^{1 \times n_2}, \quad b^{(3)} \in \mathbb{R}.$$

Concretamente, $\mathbf{W}^{(1)}$ es un vector columna con los pesos asociados a cada neurona de la primera capa oculta; $\mathbf{W}^{(2)}$ es la matriz que conecta ambas capas ocultas; $\mathbf{W}^{(3)}$ es un vector fila que pondera la salida de la segunda capa oculta; y los vectores $\boldsymbol{b}^{(k)}$ contienen los sesgos correspondientes a cada capa.

Cálculo en la primera capa oculta

Se define en primer lugar el vector de **preactivaciones** de la primera capa oculta como la combinación lineal de la entrada con los pesos y sesgos correspondientes:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} x + \boldsymbol{b}^{(1)} \in \mathbb{R}^{n_1}. \tag{3.2}$$

Componente a componente, la j-ésima preactivación se expresa como

$$z_j^{(1)} = w_j^{(1)}\, x + b_j^{(1)}, \qquad j = 1, 2, \ldots, n_1, \tag{3.3}$$

donde $w_j^{(1)}$ denota el j-ésimo elemento de $\mathbf{W}^{(1)}$ y $b_j^{(1)}$ el j-ésimo elemento de $\boldsymbol{b}^{(1)}$. A continuación se aplica una función de activación no lineal σ componente a componente, obteniendo el vector de **activaciones** de la primera capa oculta:

$$\mathbf{s}^{(1)} = \sigma\big(\mathbf{z}^{(1)}\big), \qquad s_j^{(1)} = \sigma\big(z_j^{(1)}\big), \qquad j = 1, \ldots, n_1. \tag{3.4}$$

Para el caso que se desarrolla, se selecciona $\sigma(z) = \tanh(z)$.

Cálculo en la segunda capa oculta

Las activaciones $\mathbf{s}^{(1)}$ constituyen la entrada de la segunda capa oculta. La preactivación se obtiene mediante la multiplicación matricial

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\, \mathbf{s}^{(1)} + \boldsymbol{b}^{(2)} \in \mathbb{R}^{n_2}, \tag{3.5}$$

o equivalentemente, para la i-ésima componente:

$$z_i^{(2)} = \sum_{j=1}^{n_1} w_{ij}^{(2)}\, s_j^{(1)} + b_i^{(2)}, \qquad i = 1, 2, \ldots, n_2. \tag{3.6}$$

La activación correspondiente se calcula aplicando nuevamente σ componente a componente:

$$\mathbf{s}^{(2)} = \sigma\big(\mathbf{z}^{(2)}\big), \qquad s_i^{(2)} = \sigma\big(z_i^{(2)}\big), \qquad i = 1, \ldots, n_2. \tag{3.7}$$

Cálculo en la capa de salida

Las activaciones $\mathbf{s}^{(2)}$ se propagan finalmente a la capa de salida, cuya preactivación se obtiene como

$$z^{(3)} = \mathbf{W}^{(3)}\,\mathbf{s}^{(2)} + b^{(3)} = \sum_{i=1}^{n_2} w_i^{(3)}\, s_i^{(2)} + b^{(3)} \in \mathbb{R}. \tag{3.8}$$

Puesto que la tarea es de regresión sobre un valor real continuo, la capa de salida emplea una función de activación lineal (es decir, la identidad), de modo que la predicción de la red coincide con su preactivación:

$$\hat{y} = z^{(3)}. \tag{3.9}$$

La elección de una activación lineal en la salida obedece a una razón muy específica: si se utilizara una función acotada como tanh o sigmoide, la red no podría reproducir valores fuera del intervalo correspondiente al rango de dicha función, lo cual restringiría su capacidad de aproximación.

La composición de las ecuaciones (3.2)–(3.9) define la **función de entrenamiento** de la red, que se expresa de manera compacta como la composición

$$\hat{y}(x;\boldsymbol{\theta}) = \mathbf{W}^{(3)}\,\sigma\Big(\mathbf{W}^{(2)}\,\sigma\big(\mathbf{W}^{(1)}x + \boldsymbol{b}^{(1)}\big) + \boldsymbol{b}^{(2)}\Big) + b^{(3)}, \tag{3.10}$$

donde $\boldsymbol{\theta} = \{\mathbf{W}^{(k)}, \boldsymbol{b}^{(k)}\}_{k=1}^{3}$ agrupa la totalidad de los parámetros entrenables. La red define una familia parametrizada de funciones $\hat{y}(\,\cdot\,;\boldsymbol{\theta}) : \mathbb{R} \to \mathbb{R}$ cuyo objetivo es seleccionar, mediante el ajuste de $\boldsymbol{\theta}$, aquel miembro de la familia que mejor aproxime la función (3.1).

Ejemplo numérico de la evaluación directa

Con el propósito de fijar las ideas anteriores en un cálculo concreto, se considera la arquitectura $1 \to 3 \to 3 \to 1$, es decir, $n_1 = n_2 = 3$. Se evalúa la red sobre un único punto. Sea la entrada $x = \pi/2 \approx 1.5708$, cuyo valor objetivo asociado a la función (3.1) es $y = \sin(\pi/2) = 1$. Los parámetros iniciales se fijan en

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0.5 \\ -0.3 \\ 0.8 \end{pmatrix}, \qquad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.1 \\ 0.2 \\ -0.1 \end{pmatrix},$$

$$\mathbf{W}^{(2)} = \begin{pmatrix} 0.4 & -0.2 & 0.3 \\ 0.1 & 0.5 & -0.4 \\ -0.3 & 0.2 & 0.6 \end{pmatrix}, \qquad \boldsymbol{b}^{(2)} = \begin{pmatrix} 0.05 \\ -0.1 \\ 0.15 \end{pmatrix},$$

$$\mathbf{W}^{(3)} = \begin{pmatrix} 0.7 & -0.5 & 0.3 \end{pmatrix}, \qquad b^{(3)} = 0.1.$$

Ejemplo numérico: evaluación directa paso a paso

Paso 1. Preactivaciones de la primera capa oculta. Aplicando la ecuación (3.3):

$$z_1^{(1)} = (0.5)(1.5708) + 0.1 = 0.8854,$$
$$z_2^{(1)} = (-0.3)(1.5708) + 0.2 = -0.2712,$$
$$z_3^{(1)} = (0.8)(1.5708) + (-0.1) = 1.1566.$$

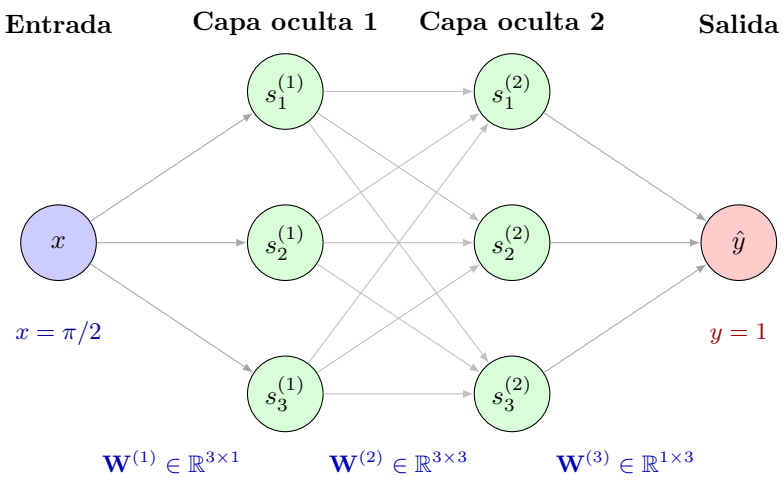


Figura 3.2: Arquitectura $1 \to 3 \to 3 \to 1$ utilizada en el ejemplo numérico. Cada conexión gris representa un peso escalar de la matriz $\mathbf{W}^{(k)}$ correspondiente.

En notación vectorial:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 0.8854 \\ -0.2712 \\ 1.1566 \end{pmatrix}.$$

Paso 2. Activaciones de la primera capa oculta. Se aplica $\sigma(z) = \tanh(z)$ componente a componente:

$$\begin{aligned} s_1^{(1)} &= \tanh(0.8854) \approx 0.7091, \\ s_2^{(1)} &= \tanh(-0.2712) \approx -0.2648, \\ s_3^{(1)} &= \tanh(1.1566) \approx 0.8199. \end{aligned}$$

Por lo tanto,

$$\mathbf{s}^{(1)} = \begin{pmatrix} 0.7091 \\ -0.2648 \\ 0.8199 \end{pmatrix}.$$

Paso 3. Preactivaciones de la segunda capa oculta. Aplicando la ecuación (3.6):

$$\begin{aligned} z_1^{(2)} &= (0.4)(0.7091) + (-0.2)(-0.2648) + (0.3)(0.8199) + 0.05 \\ &= 0.2836 + 0.0530 + 0.2460 + 0.05 = 0.6326, \\ z_2^{(2)} &= (0.1)(0.7091) + (0.5)(-0.2648) + (-0.4)(0.8199) + (-0.1) \\ &= 0.0709 - 0.1324 - 0.3279 - 0.1 = -0.4895, \\ z_3^{(2)} &= (-0.3)(0.7091) + (0.2)(-0.2648) + (0.6)(0.8199) + 0.15 \\ &= -0.2127 - 0.0530 + 0.4919 + 0.15 = 0.3763. \end{aligned}$$

En notación vectorial:

$$\mathbf{z}^{(2)} = \begin{pmatrix} 0.6326 \\ -0.4895 \\ 0.3763 \end{pmatrix}.$$

Paso 4. Activaciones de la segunda capa oculta. Se aplica nuevamente tanh:

$$s_1^{(2)} = \tanh(0.6326) \approx 0.5598,$$
$$s_2^{(2)} = \tanh(-0.4895) \approx -0.4538,$$
$$s_3^{(2)} = \tanh(0.3763) \approx 0.3595.$$

Por lo tanto,

$$\mathbf{s}^{(2)} = \begin{pmatrix} 0.5598 \\ -0.4538 \\ 0.3595 \end{pmatrix}.$$

Paso 5. Preactivación de la capa de salida. Sustituyendo en (3.8):

$$z^{(3)} = (0.7)(0.5598)+(-0.5)(-0.4538)+(0.3)(0.3595)+0.1 = 0.3919+0.2269+0.1078+0.1 = 0.8266.$$

Paso 6. Predicción y discrepancia. Por la ecuación (3.9), $\hat{y} = z^{(3)} = 0.8266$. Comparando con el valor objetivo:

$$\hat{y} - y = 0.8266 - 1 = -0.1734,$$

y el error cuadrático asociado a esta única muestra es

$$\mathcal{L} = \big(\hat{y} - y\big)^2 = (-0.1734)^2 \approx 0.0301.$$

Resumen. Los valores intermedios y finales se concentran en la siguiente tabla:

Magnitud	**Valor**
x	1.5708
$\mathbf{z}^{(1)}$	$(0.8854,\ -0.2712,\ 1.1566)^\top$
$\mathbf{s}^{(1)}$	$(0.7091,\ -0.2648,\ 0.8199)^\top$
$\mathbf{z}^{(2)}$	$(0.6326,\ -0.4895,\ 0.3763)^\top$
$\mathbf{s}^{(2)}$	$(0.5598,\ -0.4538,\ 0.3595)^\top$
$z^{(3)} = \hat{y}$	0.8266
$\mathcal{L}$	0.0301

La discrepancia entre la predicción $\hat{y} = 0.8266$ y el valor objetivo $y = 1$ es esperable: los pesos y sesgos se han fijado de manera arbitraria, sin haber sido ajustados aún. La sección siguiente desarrolla el procedimiento por el cual la red corrige estos parámetros para reducir el error.

3.2. Ajuste de Pesos y Sesgos (Backpropagation)

El ajuste de los parámetros $\boldsymbol{\theta}$ se efectúa minimizando la función de pérdida sobre un conjunto de entrenamiento $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$. Para el problema de regresión que se aborda, se adopta el error cuadrático medio:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\big(\hat{y}^{(i)} - y^{(i)}\big)^2. \tag{3.11}$$

La estrategia de optimización por excelencia es el **descenso por gradiente**, el cual requiere conocer las derivadas parciales $\partial\mathcal{L}/\partial\mathbf{W}^{(k)}$ y $\partial\mathcal{L}/\partial\boldsymbol{b}^{(k)}$ para cada capa $k = 1, 2, 3$. El algoritmo de retropropagación obtiene estas derivadas mediante una aplicación sistemática de la regla de la cadena, recorriendo la red en sentido inverso al de la evaluación directa: partiendo de la salida y avanzando hacia la entrada, tal como se ilustra en la Figura 3.3. A continuación se desarrollará el cálculo para una sola muestra (x, y); el caso de m muestras se obtiene promediando los gradientes resultantes.

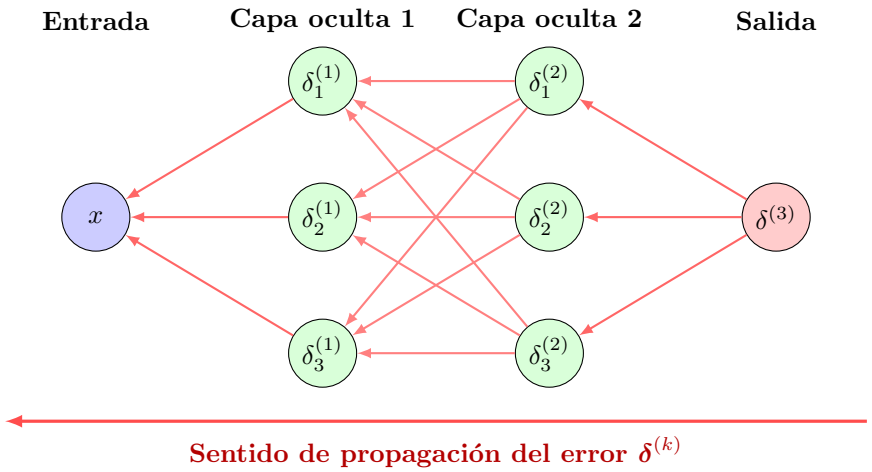


Figura 3.3: Flujo de la retropropagación: la señal de error $\delta^{(3)}$ generada en la salida se propaga hacia las capas anteriores produciendo los vectores $\boldsymbol{\delta}^{(2)}$ y $\boldsymbol{\delta}^{(1)}$, en sentido inverso al del paso directo.

Gradientes de la capa de salida

Se define la **señal de error** de la capa de salida como la derivada de la pérdida respecto a la preactivación correspondiente:

$$\delta^{(3)} \equiv \frac{\partial\mathcal{L}}{\partial z^{(3)}}. \tag{3.12}$$

Aplicando la regla de la cadena a través de la predicción $\hat{y}$:

$$\delta^{(3)} = \frac{\partial\mathcal{L}}{\partial\hat{y}}\,\frac{\partial\hat{y}}{\partial z^{(3)}}.$$

De la definición (3.11) con una única muestra se obtiene $\partial\mathcal{L}/\partial\hat{y} = 2(\hat{y} - y)$, y de la ecuación (3.9) se sigue que $\partial\hat{y}/\partial z^{(3)} = 1$, por lo que

$$\delta^{(3)} = 2\big(\hat{y} - y\big). \tag{3.13}$$

Una vez determinada $\delta^{(3)}$, los gradientes respecto a los parámetros de la capa de salida se obtienen aplicando nuevamente la regla de la cadena. Para cada peso $w_i^{(3)}$:

$$\frac{\partial \mathcal{L}}{\partial w_i^{(3)}} = \frac{\partial \mathcal{L}}{\partial z^{(3)}} \frac{\partial z^{(3)}}{\partial w_i^{(3)}} = \delta^{(3)}\, s_i^{(2)},$$

donde empleamos (3.8) para concluir que $\partial z^{(3)}/\partial w_i^{(3)} = s_i^{(2)}$. En notación matricial:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \delta^{(3)} \big(\mathbf{s}^{(2)}\big)^\top, \qquad \frac{\partial \mathcal{L}}{\partial b^{(3)}} = \delta^{(3)}. \tag{3.14}$$

Gradientes de la segunda capa oculta

Para propagar el error hacia la segunda capa oculta, se define análogamente el vector de señales de error

$$\boldsymbol{\delta}^{(2)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} \in \mathbb{R}^{n_2}. \tag{3.15}$$

La regla de la cadena, aplicada componente a componente, conduce a

$$\delta_i^{(2)} = \frac{\partial \mathcal{L}}{\partial z_i^{(2)}} = \frac{\partial \mathcal{L}}{\partial z^{(3)}} \frac{\partial z^{(3)}}{\partial s_i^{(2)}} \frac{\partial s_i^{(2)}}{\partial z_i^{(2)}}.$$

Cada uno de los tres factores se obtiene de las expresiones del paso directo:

- $\partial \mathcal{L}/\partial z^{(3)} = \delta^{(3)}$ por definición;
- $\partial z^{(3)}/\partial s_i^{(2)} = w_i^{(3)}$ por la ecuación (3.8);
- $\partial s_i^{(2)}/\partial z_i^{(2)} = \sigma'\big(z_i^{(2)}\big)$ por la ecuación (3.7).

Reuniendo los tres factores y reescribiendo en forma vectorial:

$$\boldsymbol{\delta}^{(2)} = \big(\mathbf{W}^{(3)}\big)^\top \delta^{(3)} \odot \sigma'\big(\mathbf{z}^{(2)}\big), \tag{3.16}$$

donde $\odot$ denota el producto componente a componente. Los gradientes respecto a los parámetros de la segunda capa oculta resultan entonces:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \boldsymbol{\delta}^{(2)} \big(\mathbf{s}^{(1)}\big)^\top, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(2)}} = \boldsymbol{\delta}^{(2)}. \tag{3.17}$$

Gradientes de la primera capa oculta

Repitiendo el procedimiento un nivel más, se define el vector de señales de error

$$\boldsymbol{\delta}^{(1)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \in \mathbb{R}^{n_1}, \tag{3.18}$$

cuya forma vectorial es

$$\boldsymbol{\delta}^{(1)} = \big(\mathbf{W}^{(2)}\big)^\top \boldsymbol{\delta}^{(2)} \odot \sigma'\big(\mathbf{z}^{(1)}\big). \tag{3.19}$$

La estructura de la ecuación (3.19) es idéntica a la de (3.16), pero un paso más cerca de la entrada. Esta recursividad —donde la señal de error de cada capa se obtiene a partir de la señal de la capa

siguiente, modulada por la traspuesta de la matriz de pesos correspondiente y por la derivada local de la activación— es lo que da nombre al algoritmo de **retropropagación**.

Conocida $\boldsymbol{\delta}^{(1)}$, los gradientes respecto a los parámetros de la primera capa oculta se obtienen de manera análoga:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)}\, x, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(1)}} = \boldsymbol{\delta}^{(1)}. \tag{3.20}$$

La presencia del factor x en el gradiente de $\mathbf{W}^{(1)}$ es la contraparte del factor $\mathbf{s}^{(2)}$ en (3.14) y del factor $\mathbf{s}^{(1)}$ en (3.17): en todos los casos, la magnitud del ajuste sobre un peso es proporcional al valor de la activación que entra a la conexión correspondiente, lo cual es coherente con la interpretación intuitiva de que un peso debe modificarse más cuanto más activa esté la entrada que pondera.

Regla de actualización por descenso por gradiente

Con la totalidad de los gradientes calculados, los parámetros de la red se actualizan mediante la regla del descenso por gradiente. Para una iteración t y una tasa de aprendizaje $\eta > 0$:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\, \nabla_{\boldsymbol{\theta}}\, \mathcal{L}\big(\boldsymbol{\theta}^{(t)}\big). \tag{3.21}$$

Equivalentemente, parámetro a parámetro:

$$\mathbf{W}^{(k),\,t+1} = \mathbf{W}^{(k),\,t} - \eta\, \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(k)}}, \qquad \boldsymbol{b}^{(k),\,t+1} = \boldsymbol{b}^{(k),\,t} - \eta\, \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(k)}}, \qquad k = 1, 2, 3. \tag{3.22}$$

La tasa η controla el tamaño del paso en el espacio de parámetros: valores muy pequeños producen una convergencia lenta, mientras que valores demasiado grandes pueden inducir oscilaciones o divergencia. La iteración de las ecuaciones (3.21)–(3.22) sobre un número suficiente de epochs constituye el ciclo completo de entrenamiento.

Ejemplo numérico de la retropropagación

Retomando el ejemplo inicial, donde se obtuvieron los valores intermedios $\mathbf{z}^{(1)}$, $\mathbf{s}^{(1)}$, $\mathbf{z}^{(2)}$, $\mathbf{s}^{(2)}$ y la predicción $\hat{y} = 0.8266$. Completaremos ahora una iteración de retropropagación con tasa de aprendizaje $\eta = 0.1$.

Ejemplo numérico: retropropagación y actualización paso a paso

Paso 1. Señal de error en la capa de salida. Por la ecuación (3.13):

$$\delta^{(3)} = 2\big(\hat{y} - y\big) = 2\,(0.8266 - 1) = 2\,(-0.1734) = -0.3468.$$

Paso 2. Gradientes de la capa de salida. Aplicando la ecuación (3.14):

$$\frac{\partial \mathcal{L}}{\partial w_1^{(3)}} = \delta^{(3)}\, s_1^{(2)} = (-0.3468)(0.5598) \approx -0.1941,$$

$$\frac{\partial \mathcal{L}}{\partial w_2^{(3)}} = \delta^{(3)}\, s_2^{(2)} = (-0.3468)(-0.4538) \approx 0.1574,$$

$$\frac{\partial \mathcal{L}}{\partial w_3^{(3)}} = \delta^{(3)}\, s_3^{(2)} = (-0.3468)(0.3595) \approx -0.1247,$$

$$\frac{\partial \mathcal{L}}{\partial b^{(3)}} = \delta^{(3)} = -0.3468.$$

Paso 3. Derivada local de la activación en la segunda capa oculta. Dado que $\sigma'(z) = 1 - \tanh^2(z) = 1 - \big(s^{(2)}\big)^2$:

$$\sigma'\big(z_1^{(2)}\big) = 1 - (0.5598)^2 = 1 - 0.3134 = 0.6866,$$
$$\sigma'\big(z_2^{(2)}\big) = 1 - (-0.4538)^2 = 1 - 0.2059 = 0.7941,$$
$$\sigma'\big(z_3^{(2)}\big) = 1 - (0.3595)^2 = 1 - 0.1292 = 0.8708.$$

Paso 4. Propagación del error a la segunda capa oculta. Primero se calcula el producto $\big(\mathbf{W}^{(3)}\big)^\top \delta^{(3)}$:

$$\big(\mathbf{W}^{(3)}\big)^\top \delta^{(3)} = \begin{pmatrix} 0.7 \\ -0.5 \\ 0.3 \end{pmatrix} (-0.3468) = \begin{pmatrix} -0.2427 \\ 0.1734 \\ -0.1040 \end{pmatrix}.$$

A continuación se aplica el producto componente a componente con $\sigma'\big(\mathbf{z}^{(2)}\big)$, según la ecuación (3.16):

$$\boldsymbol{\delta}^{(2)} = \begin{pmatrix} -0.2427 \\ 0.1734 \\ -0.1040 \end{pmatrix} \odot \begin{pmatrix} 0.6866 \\ 0.7941 \\ 0.8708 \end{pmatrix} \approx \begin{pmatrix} -0.1667 \\ 0.1377 \\ -0.0906 \end{pmatrix}.$$

Paso 5. Gradientes de la segunda capa oculta. El gradiente $\partial \mathcal{L}/\partial \mathbf{W}^{(2)}$ es una matriz de dimensión 3×3 obtenida mediante el producto exterior $\boldsymbol{\delta}^{(2)}\big(\mathbf{s}^{(1)}\big)^\top$, cuyas entradas (i, j) se calculan como $\delta_i^{(2)}\, s_j^{(1)}$. Sustituyendo los valores:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} \approx \begin{pmatrix} (-0.1667)(0.7091) & (-0.1667)(-0.2648) & (-0.1667)(0.8199) \\ (0.1377)(0.7091) & (0.1377)(-0.2648) & (0.1377)(0.8199) \\ (-0.0906)(0.7091) & (-0.0906)(-0.2648) & (-0.0906)(0.8199) \end{pmatrix},$$

operación que produce

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} \approx \begin{pmatrix} -0.1182 & 0.0441 & -0.1367 \\ 0.0976 & -0.0365 & 0.1129 \\ -0.0642 & 0.0240 & -0.0743 \end{pmatrix}.$$

Y para los sesgos:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(2)}} = \boldsymbol{\delta}^{(2)} \approx \begin{pmatrix} -0.1667 \\ 0.1377 \\ -0.0906 \end{pmatrix}.$$

Paso 6. Derivada local de la activación en la primera capa oculta.

$$\sigma'\big(z_1^{(1)}\big) = 1 - (0.7091)^2 = 0.4972,$$
$$\sigma'\big(z_2^{(1)}\big) = 1 - (-0.2648)^2 = 0.9299,$$
$$\sigma'\big(z_3^{(1)}\big) = 1 - (0.8199)^2 = 0.3277.$$

Paso 7. Propagación del error a la primera capa oculta. Se calcula el producto $\big(\mathbf{W}^{(2)}\big)^\top \boldsymbol{\delta}^{(2)}$:

$$\big(\mathbf{W}^{(2)}\big)^\top \boldsymbol{\delta}^{(2)} = \begin{pmatrix} 0.4 & 0.1 & -0.3 \\ -0.2 & 0.5 & 0.2 \\ 0.3 & -0.4 & 0.6 \end{pmatrix} \begin{pmatrix} -0.1667 \\ 0.1377 \\ -0.0906 \end{pmatrix} \approx \begin{pmatrix} -0.0257 \\ 0.0841 \\ -0.1594 \end{pmatrix}.$$

Aplicando el producto componente a componente con $\sigma'\big(\mathbf{z}^{(1)}\big)$, según la ecuación (3.19):

$$\boldsymbol{\delta}^{(1)} = \begin{pmatrix} -0.0257 \\ 0.0841 \\ -0.1594 \end{pmatrix} \odot \begin{pmatrix} 0.4972 \\ 0.9299 \\ 0.3277 \end{pmatrix} \approx \begin{pmatrix} -0.0128 \\ 0.0782 \\ -0.0522 \end{pmatrix}.$$

Paso 8. Gradientes de la primera capa oculta. Aplicando la ecuación (3.20) con $x = 1.5708$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1^{(1)}} &= \delta_1^{(1)}\, x = (-0.0128)(1.5708) \approx -0.0201, \\ \frac{\partial \mathcal{L}}{\partial w_2^{(1)}} &= \delta_2^{(1)}\, x = (0.0782)(1.5708) \approx 0.1228, \\ \frac{\partial \mathcal{L}}{\partial w_3^{(1)}} &= \delta_3^{(1)}\, x = (-0.0522)(1.5708) \approx -0.0821, \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(1)}} &= \boldsymbol{\delta}^{(1)} \approx (-0.0128,\, 0.0782,\, -0.0522)^{\top}. \end{aligned}$$

Paso 9. Actualización de los parámetros. Aplicando la regla (3.22) con $\eta = 0.1$, los nuevos pesos y sesgos resultan:

$$\mathbf{W}^{(1),\,\text{nuevo}} \approx \begin{pmatrix} 0.5020 \\ -0.3123 \\ 0.8082 \end{pmatrix}, \qquad \boldsymbol{b}^{(1),\,\text{nuevo}} \approx \begin{pmatrix} 0.1013 \\ 0.1922 \\ -0.0948 \end{pmatrix},$$

$$\mathbf{W}^{(2),\,\text{nuevo}} \approx \begin{pmatrix} 0.4118 & -0.2044 & 0.3137 \\ 0.0902 & 0.5036 & -0.4113 \\ -0.2936 & 0.1976 & 0.6074 \end{pmatrix}, \qquad \boldsymbol{b}^{(2),\,\text{nuevo}} \approx \begin{pmatrix} 0.0667 \\ -0.1138 \\ 0.1591 \end{pmatrix},$$

$$\mathbf{W}^{(3),\,\text{nuevo}} \approx \begin{pmatrix} 0.7194 & -0.5157 & 0.3125 \end{pmatrix}, \qquad b^{(3),\,\text{nuevo}} \approx 0.1347.$$

Verificación. Al efectuar nuevamente la evaluación directa con los parámetros actualizados sobre la misma entrada $x = \pi/2$, se obtiene una nueva predicción $\hat{y}_{\text{nuevo}} \approx 0.9289$, considerablemente más cercana al valor objetivo $y = 1$ que la predicción inicial $\hat{y} = 0.8266$. La pérdida correspondiente desciende de

$$\mathcal{L}_{\text{inicial}} \approx 0.0301 \qquad \text{a} \qquad \mathcal{L}_{\text{nuevo}} \approx 0.0051,$$

es decir, una reducción de aproximadamente $83\,\%$ en una única iteración del algoritmo.

El ejemplo anterior evidencia que el procedimiento de retropropagación, complementado con el descenso por gradiente, mueve efectivamente los parámetros en la dirección que reduce el error. Es importante observar también una propiedad estructural del algoritmo: las señales de error decrecen en magnitud conforme se propagan hacia capas más cercanas a la entrada —en el ejemplo, $\|\delta^{(3)}\| \approx 0.35$, $\|\boldsymbol{\delta}^{(2)}\| \approx 0.23$, $\|\boldsymbol{\delta}^{(1)}\| \approx 0.09$—, esto es un ejemplo a pequeña escala del fenómeno de gradiente que se desvanece estudiado anteriormente. Naturalmente, en un caso real no se entrena la red sobre un único punto, sino sobre un conjunto de muestras del dominio $[-2\pi,\, 2\pi]$, y se itera el ciclo durante miles de epochs hasta alcanzar una aproximación uniformemente precisa.

3.3. Implementación Computacional

A continuación se traduce la formulación matemática anterior a una implementación operativa en Python. Con el propósito de exhibir con la mayor transparencia posible la correspondencia entre las ecuaciones desarrolladas en las dos secciones precedentes y su realización computacional, se opta por una implementación **manual** del ciclo completo: las matrices $\mathbf{W}^{(k)}$ y los vectores $\boldsymbol{b}^{(k)}$ se inicializan explícitamente como tensores, la evaluación directa se programa línea por línea reproduciendo los cálculos, los gradientes se calculan a mano siguiendo las fórmulas y los parámetros se actualizan mediante la regla del descenso por gradiente en su forma más elemental. No se recurre, por tanto, ni al motor de diferenciación automática (`autograd`) ni a optimizadores avanzados como Adam: el código que se presenta a continuación es una traducción directa del procedimiento matemático.

Se utiliza la biblioteca `PyTorch` únicamente como motor de álgebra tensorial y por su compatibilidad con la aceleración por hardware MPS; ninguna de las funciones de alto nivel de la biblioteca (capas predefinidas, optimizadores, retropropagación automática) interviene en el cálculo. Para vectorizar las operaciones sobre las m muestras del conjunto de entrenamiento de manera simultánea, los datos se organizan en matrices $\mathbf{X} \in \mathbb{R}^{1\times m}$ y $\mathbf{Y} \in \mathbb{R}^{1\times m}$, donde cada columna corresponde a una muestra distinta. Bajo esta convención, las ecuaciones (3.2)–(3.9) se generalizan reemplazando los escalares x y los vectores columna $\mathbf{z}^{(k)}$, $\mathbf{s}^{(k)}$ por matrices cuyas columnas representan, respectivamente, las preactivaciones y activaciones de cada muestra. La pérdida MSE sobre el conjunto completo, conforme a la ecuación (3.11), se obtiene promediando los cuadrados de las diferencias columna a columna.

```python
import torch
import numpy as np
import matplotlib.pyplot as plt

# Reproducibilidad de los experimentos
torch.manual_seed(42)
np.random.seed(42)

# ------------------------------------------------------------
# 1. Generación del conjunto de entrenamiento
# ------------------------------------------------------------
# Se muestrea f(x) = sin(x) en [-2*pi, 2*pi] con m = 200 puntos.
# X y Y se organizan como matrices fila para vectorizar las
# operaciones sobre todas las muestras simultáneamente.
m = 200
x = torch.linspace(-2*np.pi, 2*np.pi, steps=m).reshape(1, m)
y = torch.sin(x)
X, Y = x, y                       # X, Y en R^{1 x m}

# ------------------------------------------------------------
# 2. Inicialización manual de W^(k) y b^(k)
# ------------------------------------------------------------
n1, n2 = 32, 32                   # Neuronas por capa oculta
W1 = torch.randn(n1, 1)  * np.sqrt(1.0/1)
b1 = torch.zeros(n1, 1)
W2 = torch.randn(n2, n1) * np.sqrt(1.0/n1)
b2 = torch.zeros(n2, 1)
W3 = torch.randn(1, n2)  * np.sqrt(1.0/n2)
b3 = torch.zeros(1, 1)

# ------------------------------------------------------------
# 3. Hiperparámetros del entrenamiento
# ------------------------------------------------------------
```

```
eta      = 0.03                    # Tasa de aprendizaje
n_epocas = 20000
historial_perdida = []

# ------------------------------------------------------------
# 4. Ciclo de entrenamiento manual
# ------------------------------------------------------------
for epoca in range(n_epocas):

    # ----- Evaluación directa (forward propagation) -----
    Z1 = W1 @ X + b1
    S1 = torch.tanh(Z1)
    Z2 = W2 @ S1 + b2
    S2 = torch.tanh(Z2)
    Z3 = W3 @ S2 + b3
    Y_hat = Z3

    # ----- Pérdida MSE -----
    L = ((Y_hat - Y)**2).mean()
    historial_perdida.append(L.item())

    # ----- Retropropagación manual -----
    # Capa de salida
    dZ3 = (2.0/m) * (Y_hat - Y)
    dW3 = dZ3 @ S2.T
    db3 = dZ3.sum(dim=1, keepdim=True)

    # Segunda capa oculta:
    dS2 = W3.T @ dZ3
    dZ2 = dS2 * (1 - S2**2)
    dW2 = dZ2 @ S1.T
    db2 = dZ2.sum(dim=1, keepdim=True)

    # Primera capa oculta:
    dS1 = W2.T @ dZ2
    dZ1 = dS1 * (1 - S1**2)
    dW1 = dZ1 @ X.T
    db1 = dZ1.sum(dim=1, keepdim=True)

    # ----- Actualización de los parámetros -----
    W1 -= eta * dW1;  b1 -= eta * db1
    W2 -= eta * dW2;  b2 -= eta * db2
    W3 -= eta * dW3;  b3 -= eta * db3

    if (epoca + 1) % 2000 == 0:
        print(f"Época {epoca+1:5d}/{n_epocas} | "
              f"Pérdida MSE: {L.item():.6e}")

# ------------------------------------------------------------
# 5. Predicción final con la red entrenada
# ------------------------------------------------------------
Z1 = W1 @ X + b1;  S1 = torch.tanh(Z1)
Z2 = W2 @ S1 + b2; S2 = torch.tanh(Z2)
Y_pred = (W3 @ S2 + b3).numpy()

# ------------------------------------------------------------
# 6. Gráfica 1: aproximación vs función analítica
# ------------------------------------------------------------
x_np    = X.numpy().flatten()
```

```
y_np    = Y.numpy().flatten()
y_pred  = Y_pred.flatten()

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(x_np, y_np,
        color='blue',  linewidth=2.0,
        label=r'$f(x) = \sin(x)$ (analítica)')
ax.plot(x_np, y_pred,
        color='red',   linewidth=1.2, linestyle='--',
        label=r'$\hat{y}(x;\theta)$ (red entrenada)')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$f(x), \; \hat{y}(x;\theta)$')
ax.set_xlim(-2*np.pi, 2*np.pi)
ax.set_ylim(-1.3, 1.3)
ax.set_xticks([-2*np.pi, -np.pi, 0, np.pi, 2*np.pi])
ax.set_xticklabels([r'$-2\pi$', r'$-\pi$', r'$0$', r'$\pi$', r'$2\pi$'])
ax.grid(True, linestyle='--', alpha=0.4)
ax.legend(loc='lower right', framealpha=0.9)
fig.tight_layout()
plt.close(fig)

# ------------------------------------------------------------
# 7. Gráfica 2: curva de decaimiento de la pérdida
# ------------------------------------------------------------
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.semilogy(range(1, n_epocas + 1), historial_perdida,
            color='blue', linewidth=1.2,
            label='Pérdida de entrenamiento')
ax.set_xlabel('Época')
ax.set_ylabel(r'$\mathcal{L}_{\mathrm{MSE}}$')
ax.set_xlim(0, n_epocas)
ax.grid(True, which='both', linestyle='--', alpha=0.4)
ax.legend(loc='upper right', framealpha=0.9)
fig.tight_layout()
plt.close(fig)

# ------------------------------------------------------------
# 8. Métrica final: error máximo absoluto
# ------------------------------------------------------------
error_max = np.max(np.abs(y_np - y_pred))
error_l2  = np.sqrt(np.mean((y_np - y_pred)**2))
print(f"Error máximo |y - y_hat|: {error_max:.6e}")
print(f"Error RMSE sobre el conjunto: {error_l2:.6e}")
```

Aproximación de $f(x) = \sin(x)$ con descenso por gradiente manual

El bloque numerado como `1` construye las matrices fila $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{1\times 200}$, donde cada columna corresponde a un par muestra–objetivo.

El bloque `2` inicializa los seis arreglos de parámetros: cada entrada de $\mathbf{W}^{(k)}$ se muestrea de una distribución normal estándar y se reescala por un factor $1/\sqrt{n_{k-1}}$, donde n_{k-1} es el número de neuronas en la capa anterior; los sesgos se inicializan en cero. Esta inicialización mantiene la varianza de las activaciones aproximadamente constante a lo largo de las capas, atenuando los problemas de saturación.

El bloque `4` constituye el núcleo del programa. La sección *forward* reproduce, en cinco líneas, la cadena completa de las ecuaciones (3.2)–(3.9), donde el operador `@` denota la multiplicación matricial. La sección de *retropropagación manual* implementa explícitamente las fórmulas desarrolladas en la Sección 3.2: el cálculo de `dZ3` corresponde a la ecuación (3.13); las multiplicaciones `dZ3 @ S2.T`,

`dZ2 @ S1.T` y `dZ1 @ X.T` realizan los productos exteriores que aparecen en las ecuaciones (3.14), (3.17) y (3.20), respectivamente; y los términos `(1 - S1**2)` y `(1 - S2**2)` representan la derivada $\sigma'(z) = 1 - \tanh^2(z)$ del producto componente a componente en las ecuaciones (3.16) y (3.19). La suma `dZ.sum(dim=1)` agrega las contribuciones de las m muestras al gradiente del sesgo, equivalente al promedio implícito en la ecuación (3.11).

La sección de *actualización* aplica la regla del descenso por gradiente (3.22) de la manera más directa: a cada parámetro se le resta su gradiente multiplicado por la tasa de aprendizaje η. Conviene resaltar dos elecciones de hiperparámetros. La tasa $\eta = 0.03$ resulta de un compromiso entre velocidad de convergencia y estabilidad: valores significativamente mayores ($\eta \gtrsim 0.1$) producen divergencia numérica del entrenamiento, en tanto que valores menores prolongan innecesariamente el cómputo. El número de epochs, 20 000, es considerablemente superior al que requeriría un optimizador adaptativo como Adam ($\sim$ 2 000 epochs para alcanzar una pérdida comparable); este contraste ilustra de manera cuantitativa la diferencia de eficiencia entre el descenso por gradiente y los algoritmos de optimización modernos.

El bloque `5` ejecuta una última evaluación directa con los parámetros entrenados para obtener las predicciones.

El bloque `6` produce la primera gráfica comparativa (Figura 3.4), superponiendo la curva analítica $f(x) = \sin(x)$ con la predicción $\hat{y}(x; \boldsymbol{\theta})$ generada por la red entrenada sobre los mismos 200 puntos del dominio.

El bloque `7` construye, en escala semilogarítmica, la curva de decaimiento de la pérdida (Figura 3.5) a partir del arreglo `historial_perdida` acumulado durante el entrenamiento.

El bloque `8` computa dos métricas escalares útiles para cuantificar la calidad de la aproximación: el error máximo absoluto $\max_i |y^{(i)} - \hat{y}^{(i)}|$ y el error cuadrático medio raíz (RMSE) sobre el conjunto completo.

3.4. Comparativa de Resultados

Una vez finalizado el ciclo de entrenamiento, se contrasta la salida de la red contra la función analítica original con dos propósitos: validar cualitativamente la capacidad de aproximación de la arquitectura adoptada y verificar la convergencia del proceso de optimización.

La Figura 3.4 muestra la superposición entre la función objetivo $f(x) = \sin(x)$ y la predicción $\hat{y}(x; \boldsymbol{\theta})$ generada por la red tras 20 000 epochs de descenso por gradiente. Se observa que ambas curvas son prácticamente indistinguibles a simple vista, lo cual confirma que una red de dos capas ocultas con 32 neuronas cada una es suficiente para aproximar de manera precisa una función trigonométrica sobre dos períodos completos. Pese a la sencillez relativa de la arquitectura y al uso del optimizador más elemental —el descenso por gradiente sin modificaciones—, el modelo logra reproducir la curvatura, los puntos de inflexión y los extremos locales de la función con un error visualmente despreciable.

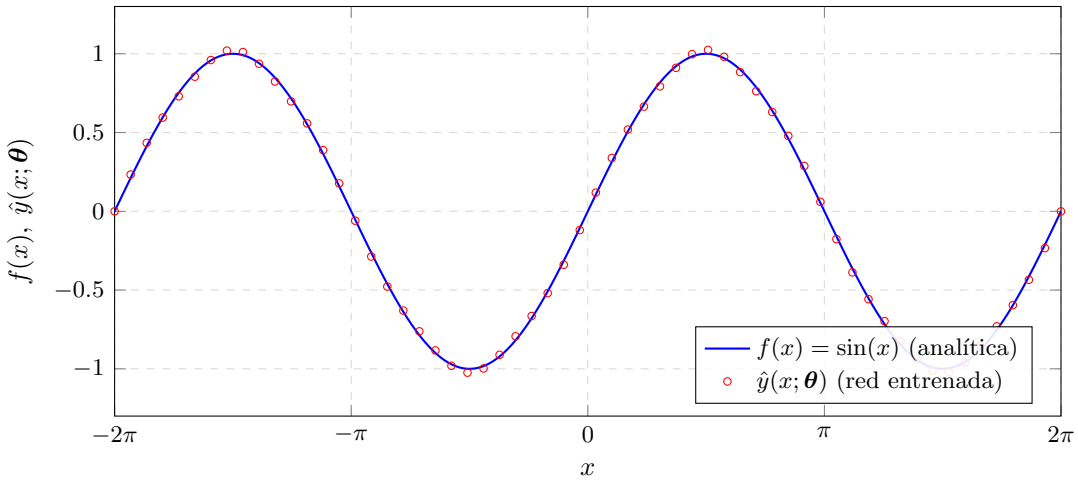


Figura 3.4: Comparación entre la función analítica $f(x) = \sin(x)$ y la aproximación $\hat{y}(x;\boldsymbol{\theta})$ producida por la red neuronal de arquitectura $1 \to 32 \to 32 \to 1$ tras 20 000 epochs de entrenamiento mediante descenso por gradiente con $\eta = 0.03$.

La Figura 3.5 presenta la evolución de la función de pérdida $\mathcal{L}_{\text{MSE}}$ a lo largo del entrenamiento, representada en escala semilogarítmica para apreciar la dinámica completa de la optimización. Se distinguen dos regímenes claramente diferenciados: Durante las primeras 10 000 epochs, la pérdida desciende de manera aproximadamente exponencial, este comportamiento se manifiesta como una caída suave y prácticamente lineal, característica del descenso por gradiente cuando la trayectoria del optimizador atraviesa una región del paisaje de pérdida con curvatura moderada; A partir de la época $\approx 10\,000$, la curva entra en una fase de estabilización: la pérdida se mantiene esencialmente plana, con variaciones imperceptibles a la escala de la gráfica. Este comportamiento indica que el algoritmo se aproxima a un mínimo local del paisaje de pérdida y que las correcciones que el descenso por gradiente puede aún introducir sobre los parámetros se han vuelto marginales.

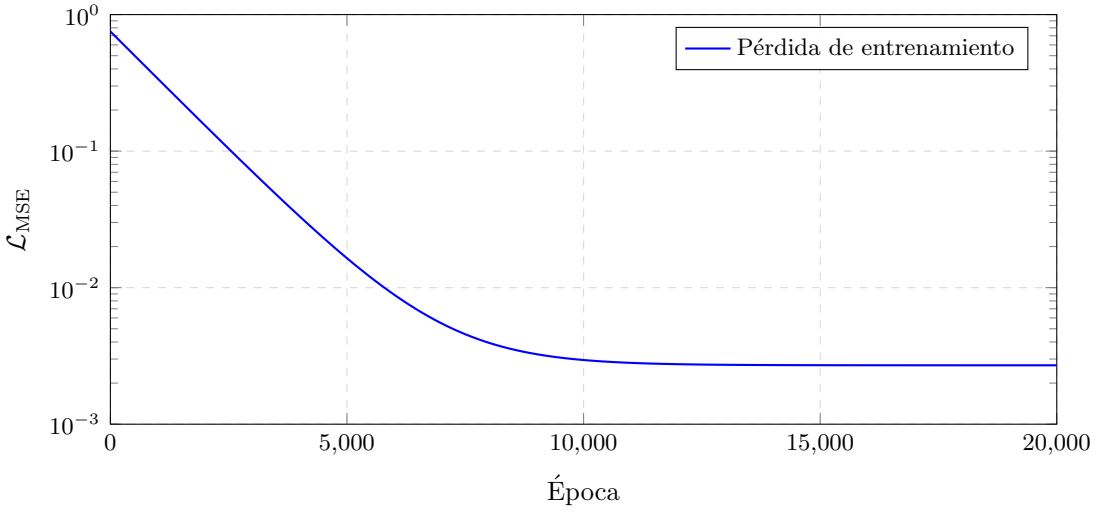


Figura 3.5: Curva de decaimiento de la función de pérdida $\mathcal{L}_{\text{MSE}}$ durante las 20 000 epochs de entrenamiento mediante descenso por gradiente, representada en escala semilogarítmica.

El valor estabilizado en torno a 10^{-3} merece un comentario específico. Una pérdida de este orden es notablemente superior a la que alcanzaría un optimizador adaptativo como Adam con la misma arquitectura. La diferencia es enteramente atribuible a las propiedades del descenso por gradiente puro: al utilizar una tasa de aprendizaje fija, el algoritmo no puede acelerar el avance en direcciones de baja curvatura ni amortiguarlo en direcciones de alta curvatura, lo que se traduce en un comportamiento oscilatorio próximo al mínimo y en una convergencia más lenta. El presente ejemplo ilustra, por tanto, la motivación práctica para el desarrollo de algoritmos de optimización más sofisticados que han dominado la práctica del aprendizaje profundo en las últimas dos décadas.

Capítulo 4

Machine Learning para Ecuaciones Diferenciales

Los capítulos anteriores han establecido el marco teórico y operativo básico de las redes neuronales: su definición matemática como composiciones parametrizadas de funciones no lineales, las arquitecturas fundamentales, las funciones de activación, los mecanismos de regularización, y el flujo computacional completo de evaluación directa y retropropagación. En particular, el Teorema de Aproximación Universal garantiza que una red neuronal con una sola capa oculta puede aproximar cualquier función continua con precisión arbitraria, siempre que se disponga de suficientes neuronas. Esta capacidad de aproximación es la piedra angular que permite extender el uso de las redes neuronales más allá de las tareas clásicas de clasificación y regresión hacia un dominio fundamentalmente distinto: la resolución de ecuaciones diferenciales.

El paradigma convencional del aprendizaje supervisado requiere un conjunto de pares entrada-salida (x_i, y_i) con los cuales entrenar el modelo. La red aprende a replicar un mapeo desconocido a partir de ejemplos, sin embargo, en la mayoría de los problemas de la física matemática, lo que se dispone no es un conjunto de datos etiquetados, sino una **ley diferencial** (una ecuación diferencial ordinaria o parcial) que gobierna el fenómeno, junto con condiciones iniciales y de frontera. Discretizar esta ley sobre una malla y resolverla mediante diferencias finitas, elementos finitos o métodos espectrales ha sido el enfoque dominante durante décadas, con resultados notables pero también con limitaciones bien conocidas: la necesidad de generar mallas finas (cuyo costo crece exponencialmente con la dimensión), la rigidez frente a geometrías complejas, y la dificultad de incorporar datos experimentales dispersos dentro del proceso de solución.

La pregunta que da origen a este capítulo es conceptualmente directa: si una red neuronal puede aproximar cualquier función continua, ¿es posible entrenarla para que la función aprendida satisfaga una ecuación diferencial dada? La respuesta afirmativa a esta pregunta ha generado una familia de métodos que se distinguen del aprendizaje supervisado estándar en un aspecto fundamental: la función de pérdida no se construye únicamente a partir de datos observados, sino que incorpora explícitamente el **residuo** de la ecuación diferencial. De este modo, la propia ley física actúa como un regularizador que guía el proceso de optimización, reduciendo drásticamente la cantidad de datos necesarios y garantizando que la solución aproximada sea físicamente consistente.

Es importante reconocer que la idea de utilizar redes neuronales para resolver ecuaciones diferenciales no es reciente. Los trabajos pioneros de Lee y Kang (1990) y de Lagaris, Likas y Fotiadis (1998) exploraron esta posibilidad con redes de una sola capa oculta, demostrando que el enfoque era viable en principio. Sin embargo, las limitaciones computacionales de la época (la ausencia de diferenciación automática eficiente, la falta de hardware especializado para el entrenamiento y el

uso de funciones de activación propensas a la saturación) impidieron que estos métodos alcanzaran la madurez necesaria para competir con los métodos numéricos establecidos. El resurgimiento de este enfoque en la última década ha sido posible gracias a la convergencia de tres factores: la disponibilidad de marcos de aprendizaje profundo con diferenciación automática nativa (TensorFlow, PyTorch), la aceleración masiva por hardware gráfico, y los avances teóricos en la comprensión de las redes profundas.

En las secciones siguientes se desarrollan tres métodos que materializan esta idea desde perspectivas complementarias. Las *Physics-Informed Neural Networks* (PINNs) constituyen el marco fundacional: modifican la función de pérdida para penalizar el incumplimiento de la ecuación diferencial en un conjunto de puntos de colocación. Las PINNs auto-adaptativas (SA-PINNs) extienden este marco al introducir pesos de adaptación entrenables por punto, resolviendo problemas de convergencia en ecuaciones rígidas. Finalmente, las *Neural ODEs* adoptan una perspectiva dual: en lugar de usar la red para resolver una ecuación diferencial conocida, parametrizan la propia dinámica del sistema como una red neuronal, convirtiendo la propagación hacia adelante en la solución de un problema de valor inicial.

4.1. Physics-Informed Neural Networks (PINNs)

4.1.1. Formulación general del problema y la idea central

La propuesta de las PINNs parte de una observación directa: si una red neuronal $u_\theta(t, x)$ puede aproximar cualquier función continua, entonces en particular puede aproximar la solución de una ecuación diferencial. El problema es que, a diferencia de una tarea de regresión donde se dispone de pares (x_i, y_i), aquí no se conoce la solución que se quiere aproximar, precisamente eso es lo que se busca. Lo que sí se conoce es la **ecuación** que la solución debe satisfacer, junto con las condiciones iniciales y de frontera. La idea central de las PINNs consiste en utilizar esta información para construir una función de pérdida que, al ser minimizada, fuerce a la red a converger hacia la solución correcta.

Para formalizar esta idea, considérese una ecuación diferencial parcial parametrizada y no lineal de la forma general

$$u_t + \mathcal{N}[u;\, \lambda] = 0, \qquad x \in \Omega, \quad t \in [0,\, T], \tag{4.1}$$

donde $u(t, x)$ es la solución desconocida (o *latente*), $\mathcal{N}[\,\cdot\,;\lambda]$ es un operador diferencial no lineal parametrizado por λ, y $\Omega \subseteq \mathbb{R}^D$ es el dominio espacial. El término "latente" subraya que en muchas aplicaciones la solución completa no es directamente observable, solo se dispone de mediciones parciales, ruidosas o indirectas del estado del sistema. La ecuación (4.1) se complementa con condiciones iniciales

$$u(0, x) = u_0(x), \qquad x \in \Omega, \tag{4.2}$$

y condiciones de frontera

$$\mathcal{B}[u](t, x) = g(t, x), \qquad x \in \partial\Omega, \quad t \in [0,\, T], \tag{4.3}$$

donde $\mathcal{B}$ es un operador de frontera y g una función conocida. Juntas, definen un problema de valor inicial y de frontera bien planteado.

Ahora bien, si una función u satisface la ecuación (4.1), entonces al sustituirla en el lado izquierdo se obtiene exactamente cero: $u_t + \mathcal{N}[u] = 0$. Si en cambio se sustituye una aproximación u_θ que no es solución exacta, el resultado será una cantidad distinta de cero, el **residuo**, cuya magnitud mide qué tan lejos está u_θ de satisfacer la ecuación. La estrategia de las PINNs es, en esencia, entrenar la red para que este residuo sea lo más pequeño posible en un conjunto de puntos del dominio, al mismo

tiempo que la red reproduzca las condiciones iniciales y de frontera. El entrenamiento convierte así la resolución de la ecuación diferencial en un problema de optimización sobre los parámetros θ de la red.

Un aspecto importante de la formulación (4.1) es su generalidad deliberada. El operador $\mathcal{N}$ puede representar ecuaciones de naturaleza muy diversa: la ecuación de Burgers unidimensional con viscosidad ν, donde $\mathcal{N}[u] = u\,u_x - \nu\,u_{xx}$; la ecuación del calor, con $\mathcal{N}[u] = -\kappa\,u_{xx}$; o la ecuación de Schrödinger no lineal con su término cúbico $|h|^2 h$ y parte imaginaria. En un enfoque numérico clásico, cada una de estas ecuaciones requeriría un esquema de discretización adaptado a sus particularidades, la PINN en cambio, trata a todas de manera unificada, se cambia únicamente la definición del residuo f_θ, es decir, qué derivadas de u_θ se combinan, mientras que: la arquitectura de la red, el algoritmo de entrenamiento y la estructura de la función de pérdida permanecen esencialmente iguales. Esta universalidad del enfoque es una de sus fortalezas más atractivas desde el punto de vista práctico.

Conviene señalar que la forma $u_t + \mathcal{N}[u] = 0$ tampoco excluye problemas estacionarios ni sistemas de ecuaciones. Una ecuación elíptica como la ecuación de Laplace $\nabla^2 u = 0$ se trata eliminando la derivada temporal, de modo que la red $u_\theta(x)$ toma solo coordenadas espaciales como entrada. Los sistemas acoplados, como las ecuaciones de Navier-Stokes con incógnitas (u, v, p), se manejan empleando redes con múltiples salidas y residuos vectoriales. En todos los casos, el principio es el mismo, la ecuación diferencial se codifica en la función de pérdida.

4.1.2. Construcción de la red informada por la física

Habiendo esbozado la idea general, es momento de precisar su mecánica. Se aproxima la solución $u(t, x)$ mediante una red neuronal profunda $u_\theta(t, x)$, donde θ denota colectivamente todos los pesos y sesgos de la red. Desde el punto de vista de la red, las coordenadas espacio-temporales (t, x) son las *entradas* y el valor de la solución u en esas coordenadas es la *salida*. Para una ecuación en una dimensión espacial, la capa de entrada tiene dos neuronas (una para t y otra para x), y la capa de salida tiene una sola neurona que produce el valor aproximado de u.

El paso conceptual decisivo es definir el **residuo de la física** como

$$f_\theta(t, x) := \frac{\partial u_\theta}{\partial t} + \mathcal{N}[u_\theta]. \tag{4.4}$$

Obsérvese que f_θ es una función que puede evaluarse en cualquier punto (t, x) del dominio. Si los parámetros θ fueran tales que u_θ satisficiera la ecuación diferencial exactamente, entonces f_θ sería idénticamente cero en todo el dominio. En la práctica, f_θ mide punto a punto cuánto se desvía la aproximación neuronal de la ley diferencial. Es, literalmente, el error residual de la ecuación.

La función f_θ hereda la estructura computacional de la red u_θ, pero posee funciones de activación efectivas distintas debido a la acción del operador diferencial $\mathcal{N}$. Para entender esto, considérese una red con una sola capa oculta y función de activación σ:

$$u_\theta(t, x) = \sum_{j=1}^{N} \alpha_j\, \sigma(w_j^{(t)}\, t + w_j^{(x)}\, x + b_j). \tag{4.5}$$

Al calcular la derivada respecto a t, se obtiene

$$\frac{\partial u_\theta}{\partial t} = \sum_{j=1}^{N} \alpha_j\, w_j^{(t)}\, \sigma'(w_j^{(t)}\, t + w_j^{(x)}\, x + b_j). \tag{4.6}$$

La derivada de la red respecto a una entrada es, por lo tanto, otra red neuronal con los mismos pesos pero con la función de activación reemplazada por su derivada σ'. Para derivadas de segundo orden, aparece σ'', y así sucesivamente. Esta observación tiene una consecuencia práctica importante, la función de activación utilizada en PINNs debe ser suficientemente suave (diferenciable tantas veces como lo requiera el operador $\mathcal{N}$), lo que explica por qué la tangente hiperbólica, infinitamente diferenciable, es la elección preferida en PINNs, en lugar de la ReLU, cuya segunda derivada es cero en casi todo punto.

Lo notable es que f_θ puede evaluarse de forma exacta, sin recurrir a aproximaciones por diferencias finitas, utilizando **diferenciación automática**. Este punto merece una explicación detallada, pues constituye el mecanismo técnico que hace viables a las PINNs. En el capítulo anterior, la diferenciación automática se presentó como la herramienta que calcula los gradientes $\partial\mathcal{L}/\partial\theta$ necesarios para actualizar los pesos durante el entrenamiento. En las PINNs, esta misma herramienta se emplea con un propósito adicional y conceptualmente distinto: calcular las derivadas de la *salida* de la red respecto a sus *entradas* (t, x). Puesto que u_θ es una composición de operaciones elementales (multiplicaciones matriciales, sumas de sesgos, evaluaciones de funciones de activación), cada una de las cuales tiene derivadas conocidas, la regla de la cadena proporciona las derivadas parciales $\partial u_\theta/\partial t$, $\partial u_\theta/\partial x$, $\partial^2 u_\theta/\partial x^2$, etc., con gran presición y a un costo computacional comparable al de una sola evaluación directa de la red.

Es instructivo contrastar este enfoque con la alternativa de aproximar las derivadas mediante diferencias finitas, como $\partial u/\partial x \approx [u(t, x+\epsilon) - u(t, x-\epsilon)]/(2\epsilon)$. Esta aproximación introduce un error de truncamiento de orden $O(\epsilon^2)$, requiere múltiples evaluaciones de la red para cada derivada, y presenta problemas de cancelación numérica cuando ϵ es muy pequeño. La diferenciación automática elimina todas estas dificultades, proporcionando derivadas exactas en una sola pasada a través del grafo computacional.

4.1.3. Función de pérdida compuesta

El entrenamiento de una PINN se formula como la minimización de una función de pérdida que agrega contribuciones de tres fuentes de información distintas. La primera fuente es la **condición inicial**: los valores conocidos de la solución en el instante $t = 0$. La segunda fuente son las **condiciones de frontera**: las restricciones que la solución debe satisfacer en los bordes del dominio espacial. La tercera fuente —y la más distintiva— es la propia **ecuación diferencial**, evaluada en el interior del dominio.

Formalmente, la pérdida total se descompone como

$$\mathcal{L}(\theta) = \mathcal{L}_0(\theta) + \mathcal{L}_b(\theta) + \mathcal{L}_f(\theta), \tag{4.7}$$

donde cada componente cuantifica una desviación distinta. El término de condición inicial mide qué tan bien la red reproduce los valores prescritos en $t = 0$:

$$\mathcal{L}_0(\theta) = \frac{1}{N_0}\sum_{i=1}^{N_0} \left| u_\theta(0, x_0^{(i)}) - u_0(x_0^{(i)}) \right|^2, \tag{4.8}$$

donde $\{x_0^{(i)}\}_{i=1}^{N_0}$ son puntos de muestreo sobre el dominio en $t = 0$. El término de condición de frontera impone las restricciones en $\partial\Omega$:

$$\mathcal{L}_b(\theta) = \frac{1}{N_b}\sum_{i=1}^{N_b} \left| \mathcal{B}[u_\theta](t_b^{(i)}, x_b^{(i)}) - g(t_b^{(i)}, x_b^{(i)}) \right|^2, \tag{4.9}$$

donde $\{(t_b^{(i)}, x_b^{(i)})\}_{i=1}^{N_b}$ son puntos sobre la frontera. El término del residuo físico —el que distingue a las PINNs de una red convencional— penaliza la desviación de la ecuación diferencial:

$$\mathcal{L}_f(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| f_\theta(t_f^{(i)}, x_f^{(i)}) \right|^2, \tag{4.10}$$

donde $\{(t_f^{(i)}, x_f^{(i)})\}_{i=1}^{N_f}$ es un conjunto de **puntos de colocación** distribuidos en el interior del dominio espacio-temporal.

La estructura de la pérdida total puede visualizarse de forma compacta como

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_0} \sum_{i=1}^{N_0} \left| u_\theta(0, x_0^{(i)}) - u_0^{(i)} \right|^2}_{\text{condición inicial}} + \underbrace{\frac{1}{N_b} \sum_{i=1}^{N_b} \left| \mathcal{B}[u_\theta]_b^{(i)} - g_b^{(i)} \right|^2}_{\text{condición de frontera}} + \underbrace{\frac{1}{N_f} \sum_{i=1}^{N_f} \left| f_\theta(t_f^{(i)}, x_f^{(i)}) \right|^2}_{\text{residuo de la ecuación diferencial}} . \tag{4.11}$$

Es fundamental observar que, a diferencia del aprendizaje supervisado clásico, el término $\mathcal{L}_f$ **no requiere datos etiquetados**: únicamente necesita las coordenadas $(t_f^{(i)}, x_f^{(i)})$ de los puntos de colocación, pues el valor objetivo del residuo es siempre cero. En un problema de regresión estándar, la red aprende de pares (x_i, y_i) donde y_i es el valor correcto. En una PINN, los "datos" del término de la física son los propios puntos del dominio, y la "etiqueta" es siempre la misma: cero. La ecuación diferencial proporciona, por así decirlo, un conjunto infinito de restricciones —una en cada punto del dominio continuo— de las cuales se muestrean N_f durante el entrenamiento. Esto permite que las PINNs operen eficazmente en regímenes de escasez de datos, utilizando la ecuación diferencial como una fuente adicional y muy potente de información.

Cada componente de la pérdida ejerce un tipo de presión distinto sobre los parámetros de la red durante el entrenamiento. Los términos $\mathcal{L}_0$ y $\mathcal{L}_b$ obligan a la red a "pasar por" los valores correctos en los bordes del dominio espacio-temporal, de manera análoga a la interpolación. El término $\mathcal{L}_f$, por su parte, restringe el *comportamiento interior* de la función: no basta con que u_θ coincida con los datos en la frontera, sino que además su variación en el dominio debe ser consistente con la ley diferencial. Intuitivamente, $\mathcal{L}_0$ y $\mathcal{L}_b$ fijan los "extremos" de la solución, mientras que $\mathcal{L}_f$ controla su "forma" entre los extremos.

Este marco admite una extensión directa cuando se dispone de mediciones experimentales del sistema en puntos dispersos del interior del dominio. En tal caso, se agrega un cuarto término de ajuste a los datos observados:

$$\mathcal{L}_s(\theta) = \frac{1}{N_s} \sum_{i=1}^{N_s} \left| u_\theta(t_s^{(i)}, x_s^{(i)}) - u_{\text{obs}}^{(i)} \right|^2, \tag{4.12}$$

donde $\{(t_s^{(i)}, x_s^{(i)}, u_{\text{obs}}^{(i)})\}_{i=1}^{N_s}$ son los datos de sensores. Esta capacidad de asimilar datos experimentales de forma natural —sin necesidad de procedimientos separados de asimilación o filtrado— constituye una ventaja práctica significativa sobre los métodos numéricos clásicos.

4.1.4. Puntos de colocación y estrategias de muestreo

La calidad de la aproximación de una PINN depende de la distribución y la cantidad de los puntos de colocación $\{(t_f^{(i)}, x_f^{(i)})\}_{i=1}^{N_f}$. Estos puntos cumplen un papel análogo al de los nodos de una malla en los métodos de diferencias finitas: son las ubicaciones donde se exige que la ecuación diferencial se satisfaga. Sin embargo, a diferencia de una malla estructurada, los puntos de colocación

pueden distribuirse libremente por el dominio sin requerir conectividad ni regularidad topológica, lo que simplifica enormemente el tratamiento de geometrías complejas.

En la formulación original de Raissi et al., los puntos de colocación se generan mediante un *Latin Hypercube Sampling*, LHS. Esta estrategia divide cada dimensión del dominio en N_f intervalos iguales y selecciona exactamente un punto en cada intervalo, de forma que la proyección del conjunto sobre cualquier eje individual constituye una muestra estratificada. El resultado es una distribución cuasi-aleatoria con mejor cobertura del dominio que el muestreo uniforme puro: se evitan las aglomeraciones y los vacíos que el azar puede producir, asegurando que todas las regiones del dominio estén representadas. Para un dominio bidimensional (t, x), esto significa que el plano espacio-temporal se "cubre" de manera más uniforme que con puntos puramente aleatorios, aunque sin la rigidez de una malla regular.

La cantidad N_f de puntos de colocación suele ser considerablemente mayor que N_0 y N_b. Valores típicos oscilan entre $N_f = 10{,}000$ y $N_f = 20{,}000$ para problemas en una o dos dimensiones espaciales, mientras que N_0 y N_b suelen ser del orden de 50 a 200 puntos. Esta asimetría refleja la naturaleza del problema: las condiciones iniciales y de frontera definen restricciones sobre conjuntos de menor dimensión (una línea o un punto en el tiempo), mientras que el residuo de la ecuación debe controlarse sobre todo el dominio espacio-temporal, que tiene una dimensión más alta.

Un aspecto que merece consideración es la relación entre los puntos de colocación y la *maldición de la dimensionalidad*. A medida que la dimensión espacial D del problema crece, el número de puntos necesarios para cubrir uniformemente el dominio $\Omega \times [0, T] \subseteq \mathbb{R}^{D+1}$ crece exponencialmente. Para un problema en una dimensión espacial, $N_f = 20{,}000$ puntos es abundante; para un problema en diez dimensiones, esa misma cantidad sería insuficiente para representar adecuadamente el dominio. Este es un desafío abierto para las PINNs en problemas de alta dimensionalidad, y ha motivado el desarrollo de estrategias de muestreo adaptativo que concentran los puntos de colocación en las regiones donde el residuo es mayor, en lugar de distribuirlos uniformemente. Precisamente esta motivación subyace a las PINNs auto-adaptativas que se presentan en la sección siguiente.

4.1.5. Diferenciación automática: uso dual en PINNs

El cálculo del residuo f_θ requiere evaluar las derivadas parciales de u_θ respecto a las variables de entrada (t, x). La diferenciación automática realiza este cálculo propagando las reglas de derivación a través del grafo computacional de la red, sin aproximaciones numéricas. Para ilustrar concretamente la mecánica, considérese la ecuación de Burgers, cuyo residuo es

$$f_\theta = \frac{\partial u_\theta}{\partial t} + u_\theta \frac{\partial u_\theta}{\partial x} - \nu \frac{\partial^2 u_\theta}{\partial x^2}. \tag{4.13}$$

Este residuo involucra tres operaciones de derivación: una derivada temporal de primer orden, una derivada espacial de primer orden, y una derivada espacial de segundo orden. Cada una de ellas se obtiene aplicando la regla de la cadena a través de todas las capas de la red. La derivada de segundo orden $\partial^2 u_\theta / \partial x^2$ se calcula en dos etapas: primero se obtiene $v_\theta := \partial u_\theta / \partial x$ como una nueva función en el grafo computacional, y luego se diferencia v_θ nuevamente respecto a x. En los marcos modernos (TensorFlow, PyTorch), esta operación se implementa con funciones como `tf.GradientTape()` o `torch.autograd.grad()`, y el usuario no necesita calcular manualmente ninguna derivada.

Un aspecto que merece énfasis es que, en las PINNs, la diferenciación automática se emplea con un propósito *dual*. En el entrenamiento convencional de redes neuronales, la diferenciación automática calcula los gradientes de la función de pérdida respecto a los *parámetros* θ (pesos y sesgos) para actualizar el modelo. En las PINNs, además de este uso estándar, se utiliza para calcular las derivadas de la *salida* u_θ respecto a las *entradas* (t, x). Este segundo uso es el que

permite construir el residuo f_θ y, por lo tanto, *informar* a la red con la estructura de la ecuación diferencial. Ambas operaciones de diferenciación coexisten en el mismo grafo computacional: durante cada iteración de entrenamiento, se calculan primero las derivadas respecto a las entradas (para evaluar f_θ), luego se construye la pérdida total $\mathcal{L}$, y finalmente se calculan las derivadas de $\mathcal{L}$ respecto a los parámetros θ para actualizar los pesos. Este proceso encadenado es posible gracias a que la diferenciación automática puede componerse arbitrariamente: derivar una función que es a su vez resultado de una derivación previa.

4.1.6. Regularización implícita y eficiencia en datos

El término $\mathcal{L}_f$ de la ecuación diferencial actúa como un mecanismo de regularización estructurado que restringe el espacio de soluciones admisibles de una manera cualitativamente distinta a las técnicas de regularización estándar presentadas en el capítulo de fundamentos. Mientras que las regularizaciones ℓ^1 y ℓ^2 penalizan la magnitud de los pesos sin considerar la semántica del problema, y el *dropout* introduce ruido estocástico en las activaciones, el término $\mathcal{L}_f$ impone una restricción *con significado físico*: descarta toda función que, aunque ajuste perfectamente las condiciones de frontera, traspase la ley diferencial que gobierna el fenómeno.

Este efecto regularizador es particularmente valioso en regímenes de escasez de datos. En un escenario convencional de aprendizaje supervisado, aproximar una función en dos dimensiones con solo 50 puntos etiquetados conduciría inevitablemente al sobreajuste. La diferencia es que en las PINNs, los 20,000 puntos de colocación proporcionan información adicional sin requerir valores etiquetados: cada punto de colocación aporta una restricción de la forma "el residuo de la ecuación diferencial debe ser cero aquí". En conjunto, estas restricciones equivalen a una densidad de información muy superior a la que sugieren los escasos puntos etiquetados.

En cuanto a las opciones de arquitectura y optimización, la práctica habitual consiste en redes *feedforward* relativamente sencillas: entre 4 y 9 capas ocultas con 20 a 100 neuronas por capa, utilizando la tangente hiperbólica como función de activación. La elección de tanh sobre ReLU obedece a razones de diferenciabilidad: como se discutió anteriormente, el operador diferencial $\mathcal{N}$ requiere calcular derivadas de u_θ de orden igual o superior al de la ecuación, y las funciones de activación deben poseer derivadas continuas de ese orden. La optimización se realiza frecuentemente con el algoritmo L-BFGS, un método cuasi-Newton de lote completo que explota información de segunda orden del paisaje de pérdida. Este método resulta particularmente apropiado para PINNs porque los conjuntos de entrenamiento son moderados, lo que permite evaluar la pérdida sobre la totalidad de los datos en cada iteración sin costo prohibitivo. Para problemas con mayor volumen de datos o dominios de alta dimensionalidad, pueden emplearse optimizadores estocásticos como Adam, frecuentemente en un esquema híbrido: una fase inicial de Adam para escapar de la región de inicialización, seguida de una fase de L-BFGS para el refinamiento final.

4.1.7. El problema directo y el problema inverso

La formulación de PINNs admite naturalmente dos clases de problemas que, en los métodos numéricos clásicos, requieren tratamientos fundamentalmente distintos.

En el **problema directo** (o *data-driven solution*), los parámetros λ del operador diferencial son conocidos a priori y el objetivo es determinar la solución $u(t, x)$. Este es el problema clásico de resolver una ecuación diferencial: dada la ley física y las condiciones iniciales/de frontera, encontrar el estado del sistema en todo instante y posición. Todos los ejemplos discutidos hasta ahora pertenecen a esta categoría.

En el **problema inverso** (o *data-driven discovery*), los parámetros λ son desconocidos y se

busca identificarlos a partir de observaciones ruidosas del sistema. Este tipo de problema surge con frecuencia en la práctica experimental: se observa el comportamiento de un sistema físico —por ejemplo, la evolución de la velocidad de un fluido— y se desea determinar qué valores de los parámetros (viscosidad, conductividad, coeficientes de reacción) son consistentes con las observaciones. En los métodos numéricos clásicos, este problema requiere una formulación completamente separada, típicamente basada en técnicas de optimización inversa o filtrado bayesiano.

En el marco de PINNs, la transición del problema directo al inverso es sorprendentemente sencilla: los parámetros λ del operador diferencial se incorporan como **parámetros entrenables adicionales** de la red. La función de pérdida combina el ajuste a los datos observados con la imposición de la ecuación diferencial:

$$\mathcal{L}(\theta, \lambda) = \frac{1}{N_s} \sum_{i=1}^{N_s} \left| u_\theta(t^{(i)}, x^{(i)}) - u_{\text{obs}}^{(i)} \right|^2 + \frac{1}{N_f} \sum_{i=1}^{N_f} \left| f_{\theta,\lambda}(t_f^{(i)}, x_f^{(i)}) \right|^2. \tag{4.14}$$

La minimización conjunta respecto a θ y λ —realizada por el mismo algoritmo de optimización utilizado en el problema directo, sin modificaciones— permite recuperar simultáneamente la solución y los parámetros del modelo. Podemos ilustrar esta capacidad con la ecuación de Navier-Stokes, donde a partir de observaciones ruidosas y dispersas del campo de velocidades, la PINN identifica los parámetros λ_1 y λ_2 de la ecuación con errores relativos inferiores al 1 %, al mismo tiempo que reconstruye el campo de presión (una cantidad que no fue observada directamente).

4.2. PINNs Auto-Adaptativas (SA-PINNs)

4.2.1. El problema del desequilibrio en la optimización

A pesar de los resultados prometedores de las PINNs en numerosos problemas, la experiencia práctica ha revelado limitaciones significativas cuando se aplican a ecuaciones diferenciales **rígidas**: problemas cuya solución presenta transiciones espaciales abruptas, evolución temporal rápida, u ondas de choque. En estos casos, la PINN estándar puede producir aproximaciones imprecisas o incluso fallar en la convergencia. Un ejemplo es la ecuación de Allen-Cahn, que describe procesos de separación de fases en aleaciones: su solución desarrolla interfaces estrechas que se mueven rápidamente, y la PINN estándar tiene dificultades para capturar estas transiciones con precisión.

La causa raíz de este comportamiento reside en la naturaleza **multi-objetivo** de la función de pérdida (4.7). Minimizar simultáneamente los términos $\mathcal{L}_0$, $\mathcal{L}_b$ y $\mathcal{L}_f$ constituye un problema de optimización con objetivos potencialmente en conflicto. El descenso por gradiente, al ser un procedimiento voraz, puede reducir rápidamente algunos componentes de la pérdida a expensas de otros, creando un **desequilibrio** en las tasas de descenso. Por ejemplo, si el gradiente de $\mathcal{L}_f$ es mucho mayor en magnitud que el de $\mathcal{L}_0$, el optimizador priorizará la reducción del residuo de la ecuación diferencial e "ignorará" parcialmente la condición inicial. El resultado es una solución que satisface aproximadamente la ecuación diferencial pero que no parte del estado inicial correcto, lo que invalida la aproximación.

Este desequilibrio no es un artefacto de una mala elección de hiperparámetros, sino una consecuencia estructural del problema. La magnitud de los gradientes de cada componente depende de la escala de la solución, del orden del operador diferencial, y de la cantidad relativa de puntos en cada término. En problemas rígidos, estas escalas pueden diferir en varios órdenes de magnitud, lo que exacerba el desequilibrio.

Un primer intento de corregir este problema consiste en introducir pesos escalares que ponderen

cada componente de la pérdida:

$$\mathcal{L}(\theta) = \lambda_r \mathcal{L}_f(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_0 \mathcal{L}_0(\theta). \tag{4.15}$$

Por ejemplo, asignar $\lambda_0 \gg 1$ fuerza al optimizador a priorizar la condición inicial. Sin embargo, este enfoque tiene una limitación fundamental: la selección óptima de los pesos λ_r, λ_b, λ_0 depende fuertemente del problema específico y puede variar en varios órdenes de magnitud entre distintas ecuaciones. Más aún, incluso para una ecuación dada, los valores óptimos pueden cambiar a lo largo del entrenamiento, ya que las escalas relativas de los gradientes evolucionan conforme la red se entrena. Tratar estos pesos como hiperparámetros fijos convierte el problema de resolver la ecuación diferencial en un problema de sintonización de hiperparámetros, que puede ser igualmente difícil.

Se han propuesto diversas estrategias para mitigar este desequilibrio: el ajuste de tasa de aprendizaje por componente, donde los pesos se actualizan en cada época usando estadísticas de los gradientes retropropagados; el remuestreo adaptativo de los puntos de colocación, concentrándolos en las regiones de mayor residuo; y la ponderación basada en el kernel tangente neuronal (NTK), donde los pesos se ajustan según los autovalores de la matriz NTK. Todas estas estrategias comparten una limitación: aplican un peso *global* a todo un componente de la pérdida (por ejemplo, un único peso para todos los puntos de colocación), sin distinguir entre puntos fáciles y difíciles dentro de un mismo componente.

4.2.2. Pesos auto-adaptativos por punto de entrenamiento

La contribución central de las SA-PINNs de McClenny y Braga-Neto es reemplazar los pesos escalares globales por **pesos entrenables individuales para cada punto de entrenamiento**. En lugar de asignar un peso único λ_r a todo el término del residuo, cada punto de colocación $(t_f^{(i)}, x_f^{(i)})$ recibe su propio peso $\lambda_r^{(i)}$, y lo mismo ocurre con los puntos de frontera y condición inicial. Estos pesos no son hiperparámetros fijados por el usuario, sino variables entrenables que la red aprende a ajustar de forma autónoma durante el entrenamiento.

La analogía con los mecanismos de atención en visión por computadora es directa: así como una red de atención aprende a "enfocar" su procesamiento en las regiones más relevantes de una imagen —aumentando los pesos de los píxeles informativos y reduciendo los de las regiones de fondo—, los pesos auto-adaptativos de las SA-PINNs concentran el esfuerzo de entrenamiento en las regiones del dominio donde la solución es más difícil de aproximar.

Formalmente, la función de pérdida de las SA-PINNs incorpora una **función de máscara** $m(\lambda)$ que modula la contribución de cada punto:

$$\mathcal{L}_f(\theta, \boldsymbol{\lambda}_r) = \frac{1}{2} \sum_{i=1}^{N_r} m\Big(\lambda_r^{(i)}\Big) \left| f_\theta(x_r^{(i)}, t_r^{(i)}) \right|^2, \tag{4.16}$$

$$\mathcal{L}_b(\theta, \boldsymbol{\lambda}_b) = \frac{1}{2} \sum_{i=1}^{N_b} m\Big(\lambda_b^{(i)}\Big) \left| \mathcal{B}[u_\theta](x_b^{(i)}, t_b^{(i)}) - g(x_b^{(i)}, t_b^{(i)}) \right|^2, \tag{4.17}$$

$$\mathcal{L}_0(\theta, \boldsymbol{\lambda}_0) = \frac{1}{2} \sum_{i=1}^{N_0} m\Big(\lambda_0^{(i)}\Big) \left| u_\theta(x_0^{(i)}, 0) - h(x_0^{(i)}) \right|^2, \tag{4.18}$$

donde $\boldsymbol{\lambda}_r = (\lambda_r^{(1)}, \ldots, \lambda_r^{(N_r)})$, $\boldsymbol{\lambda}_b = (\lambda_b^{(1)}, \ldots, \lambda_b^{(N_b)})$ y $\boldsymbol{\lambda}_0 = (\lambda_0^{(1)}, \ldots, \lambda_0^{(N_0)})$ son los vectores de pesos auto-adaptativos. La pérdida total es $\mathcal{L} = \mathcal{L}_f + \mathcal{L}_b + \mathcal{L}_0$.

La función de máscara $m : [0, \infty) \to [0, \infty)$ debe satisfacer tres propiedades: ser no negativa, diferenciable (en $(0, \infty)$) y estrictamente creciente. La condición de monotonía es crucial: garantiza que valores mayores de λ produzcan pesos mayores en la pérdida, lo que es necesario para que el mecanismo de ascenso por gradiente (que se describe a continuación) funcione correctamente. Las familias de funciones de máscara más utilizadas son las **polinomiales** $m(\lambda) = c\lambda^q$ con $c, q > 0$, y las **sigmoidales**, que producen un comportamiento de "encendido/apagado" más abrupto. Una máscara cuadrática ($q = 2$) produce una adaptación gradual y suave; una máscara de orden $q = 4$ amplifica más agresivamente las diferencias entre pesos; una máscara sigmoidal puede producir una selección casi binaria de los puntos más difíciles. La elección de la máscara influye en la nitidez de la adaptación y en la estabilidad del entrenamiento: máscaras demasiado agresivas pueden sufrir problemas de gradiente que se desvanece en las regiones de saturación, de manera análoga a lo que ocurre con las funciones de activación sigmoidales en redes profundas.

4.2.3. Optimización minimax: descenso-ascenso simultáneo

La característica fundamental de las SA-PINNs —y lo que las distingue de todos los esquemas de ponderación previos— es que la función de pérdida se **minimiza** respecto a los pesos de la red θ, pero se **maximiza** respecto a los pesos de auto-adaptación $\boldsymbol{\lambda}$. El procedimiento de entrenamiento busca, por lo tanto, un **punto de silla** en el espacio conjunto $(\theta, \boldsymbol{\lambda})$: un punto donde θ minimiza la pérdida y $\boldsymbol{\lambda}$ la maximiza. Las reglas de actualización correspondientes son:

$$\theta^{k+1} = \theta^k - \eta^k \nabla_\theta \mathcal{L}(\theta^k, \boldsymbol{\lambda}^k), \tag{4.19}$$

$$\boldsymbol{\lambda}_r^{k+1} = \boldsymbol{\lambda}_r^k + \rho_r^k \nabla_{\boldsymbol{\lambda}_r} \mathcal{L}(\theta^k, \boldsymbol{\lambda}^k), \tag{4.20}$$

donde $\eta^k > 0$ es la tasa de aprendizaje para los pesos de la red y $\rho_r^k > 0$ es una tasa de aprendizaje separada para los pesos de adaptación. Las ecuaciones análogas se aplican a $\boldsymbol{\lambda}_b$ y $\boldsymbol{\lambda}_0$. Nótese el signo positivo en (4.20): mientras que los pesos de la red *descienden* por el gradiente (buscando minimizar la pérdida), los pesos de adaptación *ascienden* por el gradiente (buscando maximizarla).

La lógica de esta estrategia se aprecia al calcular explícitamente el gradiente respecto a un peso de adaptación individual. Para el i-ésimo punto de colocación:

$$\frac{\partial \mathcal{L}}{\partial \lambda_r^{(i)}} = \frac{1}{2}\, m'\left(\lambda_r^{(i)}\right) \left| f_\theta(x_r^{(i)}, t_r^{(i)}) \right|^2. \tag{4.21}$$

Puesto que $m' > 0$ (la máscara es estrictamente creciente) y el residuo al cuadrado es no negativo, se tiene $\partial \mathcal{L} / \partial \lambda_r^{(i)} \geq 0$ siempre. Dado que el paso de ascenso *suma* este gradiente al peso actual, los pesos de adaptación **crecen monótonamente** a lo largo del entrenamiento, y *crecen más rápido en los puntos donde el residuo es mayor*. Este mecanismo produce exactamente el comportamiento deseado: las regiones donde la red tiene mayor dificultad para satisfacer la ecuación diferencial reciben progresivamente más peso en la función de pérdida, forzando al optimizador a prestarles mayor atención.

Considérese, por ejemplo, la solución de la ecuación de Burgers, que desarrolla un frente de choque en $x = 0$ conforme avanza el tiempo. En las etapas iniciales del entrenamiento, el residuo será grande en las cercanías de este frente, donde la solución cambia abruptamente, y poco en las regiones donde la solución es suave. El mecanismo de SA-PINNs incrementará los pesos de adaptación en los puntos cercanos al frente de choque mucho más rápido que en las regiones suaves, concentrando automáticamente el esfuerzo de entrenamiento donde más se necesita. Este efecto es análogo al de una malla adaptativa en los métodos de diferencias finitas, pero con la ventaja de que la adaptación se realiza de forma continua y automática dentro del propio proceso de optimización.

Además de este gradiente positivo, la magnitud de las actualizaciones puede controlarse especificando un calendario para las tasas de aprendizaje ρ_r^k, ρ_b^k y ρ_0^k, lo que añade flexibilidad al esquema. También es posible fijar algunos pesos a valores constantes: por ejemplo, si se sabe *a priori* que las condiciones de frontera son fáciles de satisfacer, los pesos de frontera pueden mantenerse fijos y entrenar solo los pesos del residuo y la condición inicial.

En la implementación, el paso de ascenso sobre $\boldsymbol{\lambda}$ se realiza invirtiendo el signo del gradiente correspondiente dentro del mismo optimizador (típicamente Adam). Los pesos de adaptación se actualizan solo durante las iteraciones de Adam y se mantienen fijos durante una fase final opcional de refinamiento con L-BFGS. Esta separación evita inestabilidades numéricas que podrían surgir del acoplamiento entre la optimización cuasi-newtoniana y el mecanismo de ascenso.

4.2.4. Justificación teórica mediante el kernel tangente neuronal

Desde una perspectiva teórica, el efecto de los pesos auto-adaptativos puede analizarse a través del **kernel tangente neuronal** (NTK, por sus siglas en inglés). Este concepto, desarrollado originalmente por Jacot, Gabriel y Hongler (2018), establece que en el régimen de redes infinitamente anchas (es decir, cuando el número de neuronas por capa tiende a infinito), la dinámica del entrenamiento por descenso por gradiente está gobernada por un kernel determinístico. Específicamente, la evolución de las predicciones de la red durante el entrenamiento sigue una ecuación de la forma

$$\frac{d\mathbf{u}}{d\tau} = -\mathbf{K}(\tau)\,(\mathbf{u} - \mathbf{y}), \tag{4.22}$$

donde τ es el tiempo de entrenamiento (no el tiempo físico de la ecuación diferencial), $\mathbf{u}$ es el vector de predicciones de la red en los puntos de entrenamiento, $\mathbf{y}$ son los valores objetivo, y $\mathbf{K}$ es la matriz NTK, cuyos elementos K_{ij} miden la correlación entre los gradientes de la red evaluados en los puntos i y j.

En el contexto de PINNs, la matriz NTK tiene una estructura de bloques que refleja los distintos componentes de la pérdida: un bloque para los puntos de colocación, otro para los de frontera, y otro para los de condición inicial. Los autovalores de esta matriz determinan la velocidad de convergencia de cada componente. Cuando los autovalores asociados a un bloque son mucho menores que los de otro, ese componente converge proporcionalmente más lento, produciendo el desequilibrio descrito anteriormente.

Los análisis realizados por McClenny y Braga-Neto muestran que los pesos auto-adaptativos producen una **ecualización suave** de la distribución de autovalores del NTK. No solo reducen la disparidad entre los autovalores promedio de cada bloque (igualando las escalas entre componentes), sino que también suavizan la forma de la distribución dentro de cada bloque, evitando que unos pocos modos dominen la convergencia mientras otros quedan rezagados. Aunque este análisis es estrictamente válido solo en el límite de redes infinitamente anchas y con la suposición de que el NTK permanece aproximadamente constante durante el entrenamiento, proporciona una justificación heurística sólida del éxito empírico de las SA-PINNs en redes finitas, y sugiere que el mecanismo de auto-adaptación aborda una causa fundamental —y no meramente superficial— del desequilibrio en el entrenamiento de PINNs.

4.3. Ecuaciones Diferenciales Ordinarias Neuronales (Neural ODEs)

4.3.1. De las redes residuales al límite continuo

Las *Neural ODEs*, introducidas por Chen, Rubanova, Bettencourt y Duvenaud en 2018, adoptan una perspectiva conceptualmente distinta a la de las PINNs. Mientras que las PINNs utilizan una

red neuronal para aproximar la *solución* de una ecuación diferencial conocida, las Neural ODEs parametrizan la *dinámica* del sistema —es decir, el lado derecho de la ecuación diferencial— como una red neuronal. En lugar de resolver una ecuación dada, definen una nueva ecuación cuya estructura emerge del proceso de aprendizaje.

La motivación surge de una observación elegante sobre las **redes residuales** (ResNets). Una capa residual realiza la transformación

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t), \tag{4.23}$$

donde $\mathbf{h}_t$ es el estado oculto en la capa t y f es una transformación parametrizada. Si se identifica t con el tiempo y se toma $\Delta t = 1$, esta ecuación tiene exactamente la forma de una **discretización de Euler** de la ecuación diferencial $d\mathbf{h}/dt = f(\mathbf{h}(t), t, \theta)$, con paso temporal unitario. En una ResNet de L capas, la entrada $\mathbf{h}_0$ se transforma sucesivamente mediante L pasos de Euler hasta producir la salida $\mathbf{h}_L$.

Esta analogía entre capas de una red y pasos temporales de un integrador numérico fue observada independientemente por varios autores. La pregunta natural que plantean Chen et al. es: ¿qué ocurre si se toma el límite continuo, es decir, si se añaden cada vez más capas con pasos cada vez más pequeños? En dicho límite, la secuencia discreta de transformaciones se reemplaza por una ecuación diferencial ordinaria:

$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t),\, t,\, \theta)\,, \tag{4.24}$$

donde f es una red neuronal que recibe como entrada el estado actual $\mathbf{h}(t)$ y el tiempo t, y produce como salida la *velocidad de cambio* del estado. Los parámetros θ son compartidos para todos los instantes de tiempo —a diferencia de una ResNet, donde cada capa puede tener parámetros independientes—. Dada una entrada $\mathbf{h}(t_0)$ (la "capa de entrada" de la red), la "capa de salida" $\mathbf{h}(t_1)$ se define como la solución del problema de valor inicial (4.24) en el tiempo t_1:

$$\mathbf{h}(t_1) = \mathbf{h}(t_0) + \int_{t_0}^{t_1} f(\mathbf{h}(t),\, t,\, \theta)\, dt. \tag{4.25}$$

La evaluación de esta integral se delega a un *solver* numérico de ecuaciones diferenciales —por ejemplo, un método de Runge-Kutta adaptativo como Dormand-Prince o un método implícito como Adams—. El *solver* decide internamente cuántas evaluaciones de f son necesarias para alcanzar una precisión prescrita, lo que dota al modelo de una "profundidad adaptativa" que no existe en las redes discretas convencionales.

Es útil pensar en la Neural ODE como una generalización de la ResNet en la que la profundidad es un parámetro continuo en lugar de un entero fijo. En una ResNet de 100 capas, la información pasa por exactamente 100 transformaciones, independientemente de la complejidad de la entrada. En una Neural ODE, el *solver* puede evaluar f decenas de veces para una entrada compleja y solo un puñado de veces para una entrada simple, adaptando la capacidad computacional a la dificultad del caso.

4.3.2. El método del adjunto para la retropropagación

El desafío técnico principal de las Neural ODEs es la retropropagación del error a través del *solver*. En una red discreta, la retropropagación almacena las activaciones intermedias de cada capa durante el paso hacia adelante, y las recorre en orden inverso para calcular los gradientes. Diferenciar directamente a través de las operaciones internas del integrador numérico es posible en principio, pero ineficiente: requiere almacenar todos los estados intermedios $\mathbf{h}(t)$ evaluados durante

la integración, lo que consume una cantidad de memoria proporcional al número de evaluaciones de f. Para un *solver* adaptativo que realiza cientos de evaluaciones, este costo puede ser prohibitivo.

Se propuso una solución elegante basada en el **método de sensibilidad del adjunto**, un resultado clásico de la teoría de control óptimo originalmente formulado en 1962. La idea central es que los gradientes de la pérdida respecto a los parámetros pueden calcularse resolviendo una segunda ecuación diferencial *hacia atrás en el tiempo*, sin necesidad de almacenar los estados intermedios del paso hacia adelante.

Considérese una función de pérdida escalar $L(\mathbf{h}(t_1))$ que depende del estado final de la ODE. Se define el **adjunto** como el gradiente de la pérdida respecto al estado en cada instante de tiempo:

$$\mathbf{a}(t) = \frac{\partial L}{\partial \mathbf{h}(t)}. \tag{4.26}$$

Esta cantidad puede interpretarse como la "sensibilidad" de la pérdida ante perturbaciones del estado en el instante t: si $\mathbf{a}(t)$ tiene una componente grande, significa que un pequeño cambio en $\mathbf{h}(t)$ produciría un cambio significativo en la pérdida final. La dinámica del adjunto está gobernada por otra ODE, obtenida por diferenciación de la regla de la cadena:

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{h}(t),\, t,\, \theta)}{\partial \mathbf{h}}. \tag{4.27}$$

La ecuación (4.27) puede entenderse como el análogo continuo de la retropropagación discreta: así como la retropropagación propaga los gradientes capa por capa desde la salida hacia la entrada, la ecuación del adjunto propaga los gradientes de forma continua desde t_1 hacia t_0. El jacobiano $\partial f/\partial \mathbf{h}$ que aparece en esta ecuación es el análogo continuo de los jacobianos de las capas individuales que intervienen en la regla de la cadena discreta.

Integrando la ecuación del adjunto desde t_1 hasta t_0 con la condición final $\mathbf{a}(t_1) = \partial L/\partial \mathbf{h}(t_1)$, se obtiene $\partial L/\partial \mathbf{h}(t_0)$. El gradiente respecto a los parámetros θ se calcula mediante una tercera integral que acumula las contribuciones de todos los instantes:

$$\frac{dL}{d\theta} = -\int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{h}(t),\, t,\, \theta)}{\partial \theta}\, dt. \tag{4.28}$$

El integrando de (4.28) mide, en cada instante t, cuánto contribuye la sensibilidad del estado ($\mathbf{a}(t)$) multiplicada por la influencia directa de los parámetros sobre la dinámica ($\partial f/\partial \theta$) al gradiente total de la pérdida. Las tres integrales —para $\mathbf{h}(t)$, $\mathbf{a}(t)$ y $dL/d\theta$— pueden resolverse **simultáneamente** en una sola llamada al *solver*, concatenando el estado, el adjunto y los gradientes parciales en un único vector aumentado. El Algoritmo 4.3.2 resume este procedimiento.

Los productos vector-jacobiano $\mathbf{a}(t)^\top(\partial f/\partial \mathbf{h})$ y $\mathbf{a}(t)^\top(\partial f/\partial \theta)$ que aparecen en las ecuaciones (4.27) y (4.28) se calculan eficientemente mediante diferenciación automática en modo reverso, con un costo temporal comparable al de una sola evaluación de f. Un resultado empírico notable es que el número de evaluaciones de f requeridas por el *solver* durante el paso hacia atrás es aproximadamente la mitad del requerido durante el paso hacia adelante, lo que sugiere que el método del adjunto no solo es más eficiente en memoria, sino también más rápido que la retropropagación directa a través del integrador.

Cabe notar que la reconstrucción del estado $\mathbf{h}(t)$ durante la integración hacia atrás —necesaria para evaluar los jacobianos— introduce un error numérico que puede acumularse. En la práctica, este error se mantiene controlado si el *solver* utiliza tolerancias suficientemente estrictas, y se han desarrollado variantes del método del adjunto con correcciones de checkpointing para problemas particularmente sensibles.

Algoritmo: Derivada en modo reverso de un problema de valor inicial

Entrada: parámetros θ; tiempos t_0, t_1; estado final $\mathbf{h}(t_1)$; gradiente $\partial L/\partial \mathbf{h}(t_1)$

$\mathbf{s}_0 = \left[\mathbf{h}(t_1),\ \frac{\partial L}{\partial \mathbf{h}(t_1)},\ \mathbf{0}_{|\theta|}\right]$ ▷ Estado aumentado inicial

definir aug_dynamics($[\mathbf{h}(t),\ \mathbf{a}(t),\ \cdot]$, t, θ): ▷ Dinámica aumentada

devolver $\left[f(\mathbf{h}(t), t, \theta),\ -\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{h}},\ -\mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}\right]$

$\left[\mathbf{h}(t_0),\ \frac{\partial L}{\partial \mathbf{h}(t_0)},\ \frac{\partial L}{\partial \theta}\right] = \text{ODESolve}(\mathbf{s}_0,\ \text{aug_dynamics},\ t_1,\ t_0,\ \theta)$

Salida: $\frac{\partial L}{\partial \mathbf{h}(t_0)},\ \frac{\partial L}{\partial \theta}$

4.3.3. Ventajas computacionales

Las Neural ODEs ofrecen tres ventajas computacionales fundamentales frente a las redes profundas discretas, cada una de las cuales aborda una limitación práctica distinta del paradigma de redes por capas.

Eficiencia en memoria. Dado que el método del adjunto no requiere almacenar los estados intermedios del paso hacia adelante —estos se recalculan integrando hacia atrás desde $\mathbf{h}(t_1)$—, el costo de memoria es $O(1)$ respecto a la "profundidad" de la red (medida como el número de evaluaciones del *solver*), en contraste con el costo $O(L)$ de una red residual con L capas. Esta propiedad es especialmente significativa para el entrenamiento de modelos muy profundos, donde la memoria del acelerador gráfico constituye un cuello de botella frecuente. Entrenar una ResNet con 1,000 capas requiere almacenar las activaciones de todas ellas; una Neural ODE equivalente almacena únicamente el estado final.

Cómputo adaptativo. Los *solvers* numéricos modernos (como los de la familia Runge-Kutta-Fehlberg o los métodos de Adams) ajustan automáticamente el tamaño de paso durante la integración para mantener el error local dentro de una tolerancia prescrita. Esto significa que la "profundidad efectiva" de la red se adapta dinámicamente a la complejidad de cada entrada individual. Las entradas que requieren transformaciones sencillas se procesan con pocos pasos (baja profundidad efectiva), mientras que las más complejas demandan una evaluación más exhaustiva (alta profundidad efectiva). Este comportamiento no tiene análogo en las redes discretas convencionales, donde todas las entradas atraviesan exactamente el mismo número de capas independientemente de su complejidad. Además, la tolerancia del *solver* proporciona un control explícito del compromiso entre precisión y velocidad: se puede entrenar con alta precisión y después reducir la tolerancia en la fase de inferencia para aplicaciones en tiempo real.

Eficiencia en parámetros. En una red residual discreta, cada capa tiene su propio conjunto de parámetros θ_t, de modo que el número total de parámetros crece linealmente con la profundidad. En una Neural ODE, la función $f(\mathbf{h}(t), t, \theta)$ utiliza el mismo conjunto de parámetros θ para todos los instantes de tiempo, lo que implica un "amarre" automático de los parámetros entre "capas" adyacentes. Este compartimiento de parámetros reduce significativamente el número total de parámetros del modelo. Diversos experimentos sobre el conjunto MNIST muestran que una Neural ODE alcanza un error de clasificación de 0.42 % —comparable al 0.41 % de una ResNet— utilizando 0.22 millones de parámetros frente a los 0.60 millones de la ResNet: una reducción de más del 60 %.

4.3.4. Modelado de series temporales continuas

Una aplicación particularmente natural de las Neural ODEs es el modelado de **series temporales con muestreo irregular**, un escenario frecuente en registros médicos electrónicos (donde las mediciones se realizan en visitas clínicas irregulares), en datos de tráfico de red (con ráfagas y silencios de duración variable) y en neurofisiología (con potenciales de acción que ocurren a intervalos no uniformes). Las redes recurrentes clásicas (RNN, LSTM, GRU) requieren discretizar las observaciones en intervalos uniformes, lo que obliga a elegir una resolución temporal fija: demasiado fina desperdicia cómputo en los intervalos vacíos; demasiado gruesa pierde información en los momentos de alta actividad. Las Neural ODEs resuelven este problema al definir una trayectoria latente continua que puede evaluarse en *cualquier* instante de tiempo, sin discretización previa.

El modelo generativo se construye de la siguiente manera. Se asume que cada serie temporal está generada por una trayectoria latente $\mathbf{z}(t)$ que evoluciona de acuerdo con la Neural ODE $d\mathbf{z}/dt = f_\theta(\mathbf{z}(t))$, donde f_θ es una red neuronal compartida entre todas las series. Dado un estado inicial latente $\mathbf{z}_{t_0}$ muestreado de una distribución a priori $p(\mathbf{z}_{t_0})$, los estados latentes en los tiempos de observación $t_1, \ldots, t_N$ se obtienen resolviendo la ODE:

$$\mathbf{z}_{t_1}, \ldots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0},\, f_\theta,\, t_0, \ldots, t_N)\,, \tag{4.29}$$

y cada observación $\mathbf{x}_{t_i}$ se genera de acuerdo con una distribución condicionada al estado latente: $\mathbf{x}_{t_i} \sim p(\mathbf{x} \mid \mathbf{z}_{t_i}, \theta_x)$. El modelo completo puede entrenarse como un autocodificador variacional (*VAE*), utilizando una red recurrente que procesa las observaciones en orden inverso como red de reconocimiento.

Una propiedad notable de esta formulación es que, dado cualquier estado latente $\mathbf{z}(t)$, la trayectoria latente completa queda unívocamente determinada: se puede extrapolar hacia el futuro o interpolar entre observaciones con la misma facilidad, simplemente resolviendo la ODE en el intervalo deseado. Esto contrasta con los modelos recurrentes, donde la extrapolación más allá de la secuencia de entrenamiento requiere heurísticas adicionales.

4.4. Perspectivas y métodos complementarios

Los tres métodos presentados en las secciones anteriores —PINNs, SA-PINNs y Neural ODEs— representan las líneas de investigación más consolidadas en la intersección entre aprendizaje profundo y ecuaciones diferenciales, pero no agotan el panorama, el cual se ha expandido rápidamente y ha producido una variedad de enfoques complementarios que abordan limitaciones específicas o explotan estructuras matemáticas particulares. Otros tres métodos relevantes son:

Deep Ritz Method. Propuesto por E y Yu (2018) [8];

DeepONet. Introducido por Lu et al. (2021) [9];

Operador Neuronal de Fourier (FNO). Propuesto por Li et al. (2021) [10]

Con la exposición teórica de los métodos de aprendizaje profundo para ecuaciones diferenciales establecida en este capítulo, se dispone del marco conceptual necesario para abordar la implementación práctica y la evaluación experimental de estos métodos. En el capítulo siguiente se instrumentarán redes neuronales informadas por la física para resolver ecuaciones diferenciales ordinarias y parciales de complejidad ascendente, analizando la influencia de los hiperparámetros, las funciones de activación y las funciones de pérdida sobre la calidad de las soluciones obtenidas.

Capítulo 5

Solución de Ecuaciones mediante PINNs

Los cuatro capítulos previos han establecido el marco conceptual y operativo básico necesario para abordar la resolución de ecuaciones diferenciales mediante redes neuronales. El presente capítulo materializa esa preparación implementando redes neuronales informadas por la física para resolver una sucesión de problemas de complejidad ascendente, comparando sus resultados con soluciones analíticas o de referencia numérica de alta precisión.

La organización del capítulo obedece a una progresión deliberada en la naturaleza matemática de los problemas. Se inicia con una **ecuación diferencial ordinaria no lineal con rigidez moderada** —el oscilador de Van der Pol con $\mu = 1$—, escenario en el que la formulación PINN básica converge de manera confiable y permite validar el método antes de abordar problemas espaciotemporales. Acto seguido se aborda la **ecuación de onda** unidimensional, un problema hiperbólico cuya solución analítica permite una validación cuantitativa precisa. A continuación se trata la **ecuación del calor** unidimensional, ejemplo paradigmático de problema parabólico con efecto suavizante natural sobre la solución. La secuencia se cierra con la **ecuación de Poisson** unidimensional, problema estacionario que prescinde de la variable temporal y permite aislar los aspectos puramente espaciales del entrenamiento. Finalmente, en la quinta sección realiza un análisis cruzado de hiperparámetros y funciones de pérdida sobre los cuatro problemas anteriores, identificando patrones de comportamiento que trascienden la ecuación específica.

Cada una de las cuatro primeras secciones sigue una estructura común que facilita la comparación. Primero se enuncia el problema en su forma matemática, junto con las condiciones iniciales y de frontera que lo cierran; Segundo, se construye la formulación PINN específica: arquitectura de la red, definición del residuo, descomposición de la función de pérdida; Tercero, se presentan los aspectos relevantes de la implementación en PyTorch; Cuarto, se muestran los resultados —la solución aprendida, su comparación con la referencia y la evolución de la pérdida durante el entrenamiento—. Quinto, se realiza un estudio de sensibilidad variando los hiperparámetros clave del modelo.

Los hiperparámetros sometidos a análisis sistemático son tres: la **topología de la red** (número de capas ocultas y número de neuronas por capa), la **función de activación** (tangente hiperbólica, sigmoide, Swish), y la **función de pérdida** (variantes en la ponderación de los términos componentes). El criterio de comparación uniforme es el **error relativo en norma** L^2 entre la solución de la red y la referencia, definido como

$$\mathcal{E}_{L^2} = \frac{\|u_\theta - u_{\text{ref}}\|_2}{\|u_{\text{ref}}\|_2}, \tag{5.1}$$

calculado sobre una malla densa de evaluación independiente del conjunto de puntos de colocación

empleados en el entrenamiento.

El componente computacional del capítulo se desarrolla en *Python* sobre el marco *PyTorch*, aprovechando su mecanismo de diferenciación automática para la evaluación de los operadores diferenciales que componen el residuo y, sobre hardware Apple Silicon, su *backend* MPS para acelerar el entrenamiento mediante la unidad de procesamiento gráfico. Las soluciones de referencia para los problemas con solución analítica conocida se obtienen evaluando las expresiones cerradas correspondientes; para el oscilador de Van der Pol, donde no existe solución cerrada, empleamos el integrador `Radau` de la rutina `scipy.integrate.solve_ivp`, un método implícito de Runge-Kutta de orden cinco apto para ecuaciones rígidas. La totalidad del código —incluidos los *scripts* de entrenamiento, los archivos de configuración de hiperparámetros y las rutinas de visualización— se encuentra disponible en el repositorio público referenciado en el apéndice, lo que permite la reproducción íntegra de los experimentos.

5.1. Ecuaciones Diferenciales Ordinarias No Lineales y Rígidas

5.1.1. Planteamiento del problema

El oscilador de Van der Pol, propuesto por Balthasar van der Pol en 1926 para modelar circuitos electrónicos con tubos al vacío, ha trascendido su contexto original y se ha consolidado como uno de los ejemplos canónicos de la dinámica no lineal. Su versión autónoma se escribe como la ecuación diferencial de segundo orden

$$\ddot{y}(t) - \mu\left(1 - y(t)^2\right)\dot{y}(t) + y(t) = 0, \qquad t \in [0,\, T], \tag{5.2}$$

donde el parámetro $\mu > 0$ controla la intensidad de la no linealidad. Para $\mu = 0$ la ecuación se reduce al oscilador armónico simple, con soluciones puramente sinusoidales. A medida que μ crece, la dinámica se aleja progresivamente del comportamiento armónico: el término $-\mu(1 - y^2)\dot{y}$ actúa como un amortiguador no lineal que se vuelve negativo para $|y| < 1$ y positivo para $|y| > 1$. El balance entre estas dos regiones produce un *ciclo límite* estable hacia el cual converge cualquier trayectoria con condiciones iniciales no triviales.

Para abordar la ecuación numéricamente conviene reescribirla como un sistema de primer orden. Definiendo $y_1(t) = y(t)$ y $y_2(t) = \dot{y}(t)$, la ecuación (5.2) equivale a

$$\begin{cases} \dot{y}_1 = y_2, \\ \dot{y}_2 = \mu\,(1 - y_1^2)\,y_2 \;-\; y_1, \end{cases} \qquad t \in [0,\, T], \tag{5.3}$$

con condiciones iniciales

$$y_1(0) = 2, \qquad y_2(0) = 0. \tag{5.4}$$

La elección $y_1(0) = 2$ sitúa la condición inicial en la región $|y| > 1$, lo que garantiza una rápida convergencia hacia el ciclo límite. En los experimentos se adopta $\mu = 1$ como valor base, y se trabaja sobre el intervalo $t \in [0,\, 10]$, suficientemente amplio para observar más de un ciclo límite completo. Esta combinación de parámetros mantiene tanto la **no linealidad**, manifestada en el término cuadrático $(1 - y_1^2)$, y la **rigidez moderada**, evidenciada en la separación entre la variedad lenta y los tramos rápidos del ciclo límite. Los valores $\mu = 2, 3, 5$ se reservan para el estudio de sensibilidad.

Carácter rígido del problema. La rigidez para μ moderado o grande tiene un origen claramente identificable. Las trayectorias del sistema exhiben *oscilaciones de relajación*: largas fases en que las variables evolucionan suavemente, separadas por transiciones rápidas en las que $\dot{y}_2$ alcanza

magnitudes muy superiores a las observadas durante las fases lentas. La razón de rigidez —el cociente entre la escala temporal lenta y la rápida— crece con μ, llegando a varios órdenes de magnitud para μ grande. Para $\mu = 1$ las dos escalas temporales coexisten ya de forma claramente distinguible, aunque la transición rápida es todavía suave (en contraste con la transición casi discontinua de $\mu \geq 5$). Los integradores explícitos de paso fijo se ven obligados a adoptar un Δt acotado por la escala rápida en regiones de transición, mientras que ese mismo paso resulta desproporcionadamente fino para las fases lentas. La solución estándar consiste en recurrir a métodos implícitos cuya región de estabilidad cubre el semiplano izquierdo del plano complejo y permite trabajar con pasos dictados por la precisión deseada y no por requerimientos de estabilidad.

Para el caso de las PINNs, la rigidez no se traduce directamente en una restricción sobre el paso de integración —no existe un paso temporal explícito en el entrenamiento—, pero sí afecta la **estructura de la pérdida**. Las transiciones del ciclo de relajación producen residuos de mayor magnitud en las regiones de cambio rápido, mientras que las fases lentas generan residuos pequeños. El optimizador, al promediar las contribuciones de los puntos de colocación, tiende a concentrarse en las fases lentas a costa de las transiciones, donde la aproximación se degrada cuando μ crece. Este es precisamente uno de los escenarios para los que se diseñaron las SA-PINNs.

Solución de referencia. Al no admitir el sistema (5.3) una solución analítica cerrada, se requiere una referencia numérica de alta precisión contra la cual evaluar el desempeño de la red. Esta se obtiene mediante el integrador `Radau` de `scipy.integrate.solve_ivp`, un método implícito de Runge-Kutta de orden cinco con interpolación densa, ejecutado con tolerancias relativa y absoluta de 10^{-10}. La salida del integrador se interpola sobre la malla de evaluación común para permitir comparaciones *punto a punto* con la salida de la PINN.

5.1.2. Formulación PINN para el oscilador de Van der Pol

La aplicación del marco PINN al sistema (5.3) se construye siguiendo los pasos generales desarrollados en el capítulo anterior, adaptándolos a las particularidades del problema. La red neuronal aproximadora se denota

$$\mathbf{u}_\theta(t) \;=\; \big(y_{1,\theta}(t),\; y_{2,\theta}(t)\big)^\top, \tag{5.5}$$

donde θ representa colectivamente todos los pesos y sesgos. La capa de entrada consta de una sola neurona, que recibe el escalar t. La capa de salida consta de dos neuronas con activación lineal, correspondientes a las dos componentes del sistema. Las capas ocultas, así como su número y ancho, constituyen hiperparámetros que serán objeto de estudio sistemático.

La elección de una arquitectura con red única y salida vectorial es preferible a la alternativa de entrenar dos redes independientes para $y_{1,\theta}$ y $y_{2,\theta}$. Las dos componentes del sistema están acopladas a través del residuo, y compartir las capas ocultas permite que la red aprenda representaciones intermedias comunes que reflejan dicho acoplamiento.

Residuo del sistema. Las dos ecuaciones de (5.3) se reescriben en forma residual:

$$f_{1,\theta}(t) \;:=\; \frac{\mathrm{d}y_{1,\theta}}{\mathrm{d}t}(t) \;-\; y_{2,\theta}(t), \tag{5.6}$$

$$f_{2,\theta}(t) \;:=\; \frac{\mathrm{d}y_{2,\theta}}{\mathrm{d}t}(t) \;-\; \mu\left(1 - y_{1,\theta}(t)^2\right) y_{2,\theta}(t) \;+\; y_{1,\theta}(t). \tag{5.7}$$

Las derivadas $\mathrm{d}y_{1,\theta}/\mathrm{d}t$ y $\mathrm{d}y_{2,\theta}/\mathrm{d}t$ se calculan mediante diferenciación automática sobre el grafo computacional de la red. En PyTorch, esto se realiza con la función `torch.autograd.grad`, configurada con `create_graph=True` para permitir el cómputo posterior de los gradientes de la pérdida

respecto a los parámetros θ. El residuo vectorial $\mathbf{f}_\theta(t) = (f_{1,\theta}, f_{2,\theta})^\top$ se evalúa en un conjunto de **puntos de colocación** $\{t_f^{(i)}\}_{i=1}^{N_f} \subset [0,T]$, generados mediante muestreo cuasi-aleatorio uniforme sobre el intervalo temporal.

Función de pérdida. Dado que el problema es un problema de valor inicial puro —sin condiciones de frontera espacial—, la pérdida total se descompone únicamente en dos contribuciones: la de la condición inicial y la del residuo del sistema. Formalmente,

$$\mathcal{L}(\theta) \;=\; \lambda_0\,\mathcal{L}_0(\theta) \;+\; \lambda_f\,\mathcal{L}_f(\theta), \tag{5.8}$$

donde los pesos λ_0 y λ_f son hiperparámetros escalares. Las dos componentes son

$$\mathcal{L}_0(\theta) \;=\; \big|y_{1,\theta}(0) - 2\big|^2 \;+\; \big|y_{2,\theta}(0) - 0\big|^2, \tag{5.9}$$

$$\mathcal{L}_f(\theta) \;=\; \frac{1}{N_f}\sum_{i=1}^{N_f}\Big(\big|f_{1,\theta}(t_f^{(i)})\big|^2 \;+\; \big|f_{2,\theta}(t_f^{(i)})\big|^2\Big). \tag{5.10}$$

La componente $\mathcal{L}_0$ fuerza a la red a respetar las condiciones iniciales, mientras que $\mathcal{L}_f$ exige que el residuo del sistema sea pequeño en todo el dominio temporal.

La elección de los pesos λ_0 y λ_f no es trivial. Aunque la formulación matemática es simétrica en ambos términos, la magnitud típica de $\mathcal{L}_0$ y la de $\mathcal{L}_f$ difieren por construcción. Un peso $\lambda_0 = \lambda_f = 1$ resulta en un sub-pesado efectivo de la condición inicial, lo que en la práctica permite a la red derivar de la trayectoria correcta para satisfacer mejor el residuo en el dominio interior. Para contrarrestar este efecto se adopta $\lambda_0 = 100$, $\lambda_f = 1$ como valor base, ponderación que recupera la magnitud relativa apropiada y constituye el estándar empírico en la literatura de PINNs sin imposición dura de la condición inicial.

Selección de la función de activación. Como se discutió en el capítulo anterior, la función de activación debe ser diferenciable tantas veces como lo exija el orden del operador diferencial presente en el residuo. Para el sistema (5.3) el orden máximo es uno, por lo que basta con que la activación posea derivada continua. Esta condición es satisfecha por la mayoría de las funciones de activación estándar; sin embargo, la práctica habitual en PINNs es emplear la **tangente hiperbólica**, por tres razones complementarias: su suavidad infinita, la simetría de su rango $(-1, 1)$ alrededor de cero —que armoniza con la normalización estándar de entradas y favorece la estabilidad numérica— y su gradiente acotado, que evita los fenómenos extremos de explosión observados con activaciones no acotadas.

Distribución de los puntos de colocación. En un dominio unidimensional —el intervalo temporal $[0,T]$—, el muestreo de Latin Hypercube degenera al muestreo aleatorio estratificado simple. Se opta por un esquema cuasi-uniforme sobre $[0,T]$ con $N_f = 2{,}000$ puntos, valor suficiente para resolver la dinámica del ciclo límite para $\mu = 1$. La condición inicial se impone en el único punto $t = 0$.

Algoritmo de optimización. La pérdida se minimiza en dos fases consecutivas. La primera, de 8000 iteraciones, emplea el optimizador estocástico **Adam** con tasa de aprendizaje $\eta = 10^{-3}$; su capacidad de escapar de regiones planas de la pérdida resulta apropiada para una fase inicial de exploración. La segunda, de un máximo de 2000 iteraciones, emplea el método cuasi-Newton de lote completo **L-BFGS**, que aprovecha información de segunda orden para un refinamiento más preciso. Esta estrategia híbrida Adam $\rightarrow$ L-BFGS se ha mostrado robusta sobre una amplia variedad de problemas.

Con la formulación completa establecida, las subsecciones siguientes desarrollan la implementación concreta en PyTorch, los resultados obtenidos sobre el problema base y los estudios de sensibilidad que constituyen el núcleo experimental de este apartado.

5.1.3. Implementación en PyTorch

La formulación matemática establece la pérdida $\mathcal{L}(\theta) = \lambda_0 \mathcal{L}_0(\theta) + \lambda_f \mathcal{L}_f(\theta)$ como objetivo de optimización. La implementación práctica de esta formulación sobre el oscilador de Van der Pol con $\mu = 1$ requiere una sola modificación arquitectónica respecto a la descrita en la sección anterior: un **embedding de Fourier modesto** en la entrada de la red. La modificación es necesaria por la propiedad conocida como *sesgo espectral* de las redes neuronales con activaciones suaves —su inclinación natural a representar funciones de baja frecuencia—, fenómeno particularmente perjudicial para problemas oscilatorios como el que nos ocupa.

Embedding de Fourier. En lugar de presentar el escalar t directamente al perceptrón multicapa, se mapea previamente al vector

$$\boldsymbol{\varphi}(t) = \left(\tfrac{t}{T},\ \sin(\omega_1 t),\ \cos(\omega_1 t),\ \ldots,\ \sin(\omega_K t),\ \cos(\omega_K t)\right)^{\top}, \tag{5.11}$$

donde $\omega_k = 2\pi k/T$ son frecuencias armónicas cubrentes del dominio temporal y $K = 5$ es el número de frecuencias. El vector $\boldsymbol{\varphi}(t)$, de dimensión $2K + 1 = 11$, se introduce como entrada al perceptrón multicapa en lugar del escalar t/T. La inyección explícita de bases trigonométricas mitiga el sesgo espectral al proveer a la red un conjunto de modos oscilatorios listos para combinarse, reduciendo significativamente la barrera para representar soluciones con dinámica oscilatoria.

La elección $K = 5$ es deliberadamente moderada. Valores excesivamente bajos ($K \leq 2$) no proporcionan suficiente flexibilidad y la red recae sobre el sesgo espectral; valores excesivamente altos ($K \geq 20$) introducen un nuevo problema —el sesgo hacia soluciones con periodos comparables con T—, pues las frecuencias $\omega_k = 2\pi k/T$ favorecen geométricamente periodos de la forma T/k que no necesariamente coinciden con el periodo verdadero del ciclo límite. El valor $K = 5$ corresponde al óptimo empírico para el problema en estudio: frecuencias en el rango $\omega_1, \ldots, \omega_5 = 2\pi/T,\ \ldots,\ 10\pi/T$, suficientes para sustentar dinámica oscilatoria sin sobrerrestringir el espacio funcional.

A continuación se presenta la implementación completa en PyTorch, dividida en cuatro bloques que reproducen la estructura conceptual del modelo: la definición del modelo neuronal, la función de pérdida, el bucle de entrenamiento y el procedimiento de evaluación.

Definición del modelo neuronal

El siguiente fragmento implementa la red completa. La clase `FourierEmbedding` encapsula la transformación (5.11), y `VdPModel` combina dicho embedding con el perceptrón multicapa de cinco capas ocultas y veinte neuronas por capa.

```
import torch
import torch.nn as nn
import numpy as np

DTYPE = torch.float32


class Swish(nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x)

```

```

ACTIVATIONS = {'tanh': nn.Tanh, 'sigmoid': nn.Sigmoid, 'swish': Swish}


class FourierEmbedding(nn.Module):
    """phi(t) = [t/T, sin(omega_k t), cos(omega_k t)]_{k=1..K}"""
    def __init__(self, n_freqs=5, T=10.0):
        super().__init__()
        freqs = torch.arange(1, n_freqs + 1, dtype=DTYPE)
        self.register_buffer('omega', 2.0 * np.pi * freqs / T)
        self.T = T
        self.n_freqs = n_freqs

    @property
    def output_dim(self):
        return 1 + 2 * self.n_freqs

    def forward(self, t):
        x = t * self.omega.view(1, -1)
        return torch.cat([t / self.T, torch.sin(x), torch.cos(x)], dim=1)


class VdPModel(nn.Module):
    """phi(t) -> 5 capas x 20 neuronas (tanh) -> (y1, y2)."""
    def __init__(self, n_layers=5, n_neurons=20, activation='tanh',
                 n_freqs=5, T=10.0):
        super().__init__()
        Act = ACTIVATIONS[activation]
        self.embed = FourierEmbedding(n_freqs=n_freqs, T=T)
        in_dim = self.embed.output_dim
        layers = [nn.Linear(in_dim, n_neurons), Act()]
        for _ in range(n_layers - 1):
            layers += [nn.Linear(n_neurons, n_neurons), Act()]
        layers += [nn.Linear(n_neurons, 2)]
        self.net = nn.Sequential(*layers)
        for m in self.net.modules():
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                nn.init.zeros_(m.bias)
        self.T = T

    def forward(self, t):
        return self.net(self.embed(t))
```

Listing 5.1: Modelo neuronal con embedding de Fourier.

Función de pérdida

La función `pinn_loss` implementa la suma ponderada $\mathcal{L} = \lambda_0 \mathcal{L}_0 + \lambda_f \mathcal{L}_f$. Los residuos del sistema se evalúan mediante diferenciación automática con `torch.autograd.grad`; la instrucción `create_graph=True` permite calcular posteriormente los gradientes de la pérdida respecto a los parámetros θ a través del grafo computacional así construido.

```
def pinn_loss(model, t_coll, mu, y0, lambda_ic=100.0, lambda_res=1.0):
    """L = lambda_ic * L_IC + lambda_res * L_residual."""
    # --- Residuo ---
    t = t_coll.clone().requires_grad_(True)
    u = model(t)
```

```
    y1, y2 = u[:, 0:1], u[:, 1:2]
    dy1_dt = torch.autograd.grad(
        y1, t, torch.ones_like(y1), create_graph=True
    )[0]
    dy2_dt = torch.autograd.grad(
        y2, t, torch.ones_like(y2), create_graph=True
    )[0]
    f1 = dy1_dt - y2
    f2 = dy2_dt - mu * (1.0 - y1**2) * y2 + y1
    L_res = torch.mean(f1**2 + f2**2)

    # --- Condicion inicial ---
    t0 = torch.zeros(1, 1, dtype=DTYPE).to(t_coll.device)
    u0 = model(t0)
    L_ic = (u0[0, 0] - y0[0])**2 + (u0[0, 1] - y0[1])**2

    return lambda_ic * L_ic + lambda_res * L_res, L_ic, L_res
```

Listing 5.2: Función de pérdida: residuo del sistema más condición inicial.

Bucle de entrenamiento Adam → L-BFGS

La estrategia discutida se implementa en dos fases secuenciales sobre el mismo conjunto de parámetros. La primera utiliza Adam con tasa de aprendizaje $\eta = 10^{-3}$ durante 8000 iteraciones; la segunda emplea L-BFGS con búsqueda de línea de Wolfe durante hasta 2000 iteraciones para el refinamiento cuasi-Newton. Los 2000 puntos de colocación se fijan al inicio del entrenamiento mediante muestreo aleatorio uniforme sobre el intervalo $[0, T]$.

```
def train(model, mu, T=10.0, n_adam=8000, n_lbfgs=2000, n_coll=2000,
          lr_adam=1e-3, lambda_ic=100.0, y0=(2.0, 0.0), seed=42):
    torch.manual_seed(seed); np.random.seed(seed)
    t_coll = torch.rand(n_coll, 1, dtype=DTYPE) * T

    history = {'iter': [], 'loss': []}
    g_iter = [0]
    def record(L, every=200):
        if g_iter[0] % every == 0:
            history['iter'].append(g_iter[0])
            history['loss'].append(L.item())
        g_iter[0] += 1

    # Fase 1: Adam
    opt = torch.optim.Adam(model.parameters(), lr=lr_adam)
    for i in range(n_adam):
        opt.zero_grad()
        L, _, _ = pinn_loss(model, t_coll, mu, y0, lambda_ic)
        L.backward()
        opt.step()
        record(L)

    # Fase 2: L-BFGS
    opt = torch.optim.LBFGS(
        model.parameters(), lr=1.0, max_iter=n_lbfgs,
        tolerance_grad=1e-11, tolerance_change=1e-13,
        history_size=100, line_search_fn='strong_wolfe',
    )
    def closure():
        opt.zero_grad()
```

```
        L, _, _ = pinn_loss(model, t_coll, mu, y0, lambda_ic)
        L.backward()
        record(L)
        return L
    opt.step(closure)
    L, _, _ = pinn_loss(model, t_coll, mu, y0, lambda_ic)
    return history, L.item()
```

Listing 5.3: Procedimiento de entrenamiento en dos fases.

Script principal de evaluación

El *script* principal articula los componentes anteriores: genera la solución de referencia con `scipy.integrate.solve_ivp` en su variante `Radau`, instancia el modelo, lo entrena y evalúa el error relativo L^2 definido. Los resultados se guardan para su posterior visualización y análisis.

```
import time
from scipy.integrate import solve_ivp

MU, T, Y0 = 1.0, 10.0, (2.0, 0.0)

def vdp_rhs(t, y, mu):
    return [y[1], mu * (1.0 - y[0]**2) * y[1] - y[0]]

def reference_solution(mu, T):
    t_eval = np.linspace(0.0, T, 2001)
    sol = solve_ivp(vdp_rhs, [0.0, T], list(Y0), args=(mu,),
                    method='Radau', t_eval=t_eval,
                    rtol=1e-10, atol=1e-10, max_step=0.05)
    return t_eval, sol.y[0], sol.y[1]

def evaluate(model, t_array):
    model.eval()
    with torch.no_grad():
        t = torch.tensor(t_array.reshape(-1, 1), dtype=DTYPE)
        u = model(t).cpu().numpy()
    return u[:, 0], u[:, 1]

def relative_l2(pred, ref):
    return float(np.linalg.norm(pred - ref) / np.linalg.norm(ref))


t_ref, y1_ref, y2_ref = reference_solution(MU, T)
model = VdPModel(n_layers=5, n_neurons=20, activation='tanh',
                 n_freqs=5, T=T)
t0 = time.time()
history, L_final = train(model, mu=MU, T=T,
                         n_adam=8000, n_lbfgs=2000, n_coll=2000,
                         lambda_ic=100.0, y0=Y0)
elapsed = time.time() - t0

y1_pinn, y2_pinn = evaluate(model, t_ref)
print(f"Tiempo: {elapsed:.1f} s    Perdida final: {L_final:.4e}")
print(f"E_L2(y1) = {relative_l2(y1_pinn, y1_ref):.4e}")
print(f"E_L2(y2) = {relative_l2(y2_pinn, y2_ref):.4e}")
```

Listing 5.4: Script principal de entrenamiento y evaluación.

El código se ha probado en PyTorch 2.12.0 sobre el *backend* MPS de Apple Silicon. El experimento base completo (8000 iteraciones Adam más hasta 2000 iteraciones L-BFGS) tomó aproximadamente tres minutos en un chip M4. Sobre CPU el tiempo se elevó a aproximadamente cinco minutos, principalmente por el costo de las evaluaciones de diferenciación automática en cada iteración del optimizador estocástico. El repositorio del proyecto contiene la versión modular completa así como los *scripts* auxiliares para los estudios de sensibilidad.

5.1.4. Resultados y comparación con la solución de referencia

Tras ejecutar el procedimiento de entrenamiento, la red converge a una aproximación de la solución del sistema con error relativo L^2 de 2.43×10^{-4} para la componente y_1 y 3.07×10^{-4} para y_2, evaluados sobre una malla uniforme de 2001 puntos en $[0, T]$ independiente del conjunto de colocación empleado en el entrenamiento. El tiempo total de entrenamiento fue de 193.4 segundos en un chip Apple M4 con *backend* MPS, y la pérdida residual final alcanzó el valor 4.32×10^{-6}.

La calidad de la aproximación se examina desde cuatro perspectivas complementarias, presentadas en las figuras siguientes.

Trayectorias temporales. La figura 5.1 superpone las dos componentes de la solución de la red sobre la referencia de Radau. La concordancia es visualmente indistinguible en la escala de la figura, lo que se confirmará cuantitativamente más adelante mediante el análisis del error puntual.

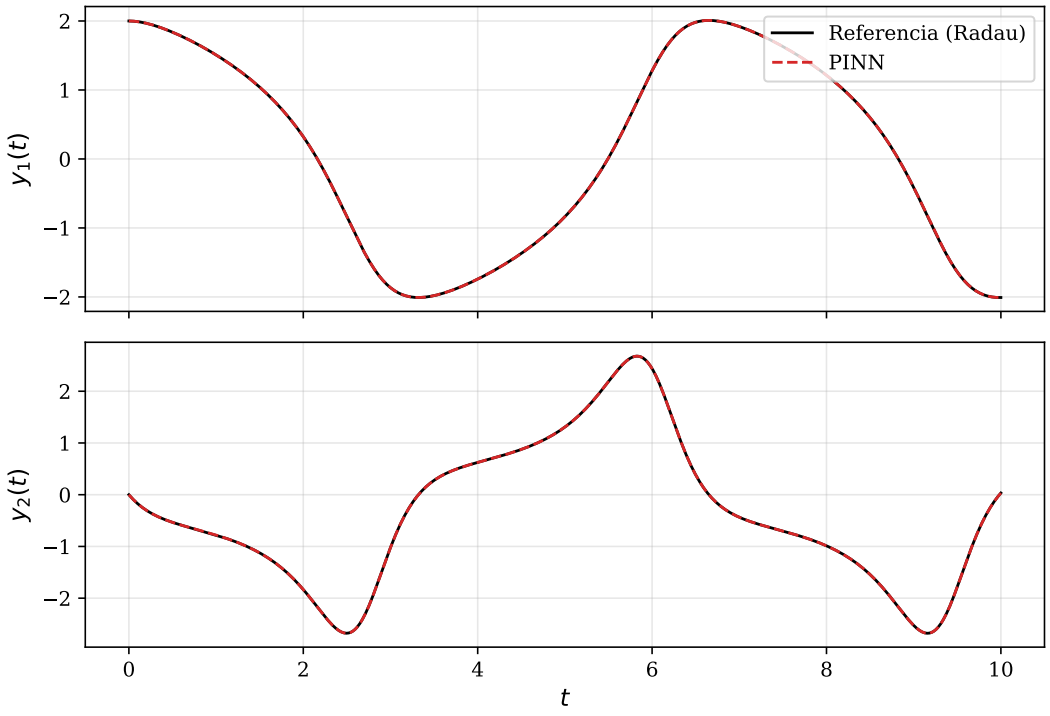


Figura 5.1: Comparación entre la solución de la PINN (línea discontinua roja) y la solución de referencia obtenida con el integrador Radau (línea continua negra) para $\mu = 1$. Panel superior: componente $y_1(t)$; panel inferior: componente $y_2(t)$.

Retrato de fase. La figura 5.2 muestra la proyección (y_1, y_2) de la trayectoria, donde se aprecia explícitamente el ciclo límite característico del oscilador de Van der Pol. La trayectoria parte de la

condición inicial $(2, 0)$ y converge rápidamente hacia el ciclo límite. La PINN reproduce fielmente tanto la fase inicial como la forma del ciclo límite.

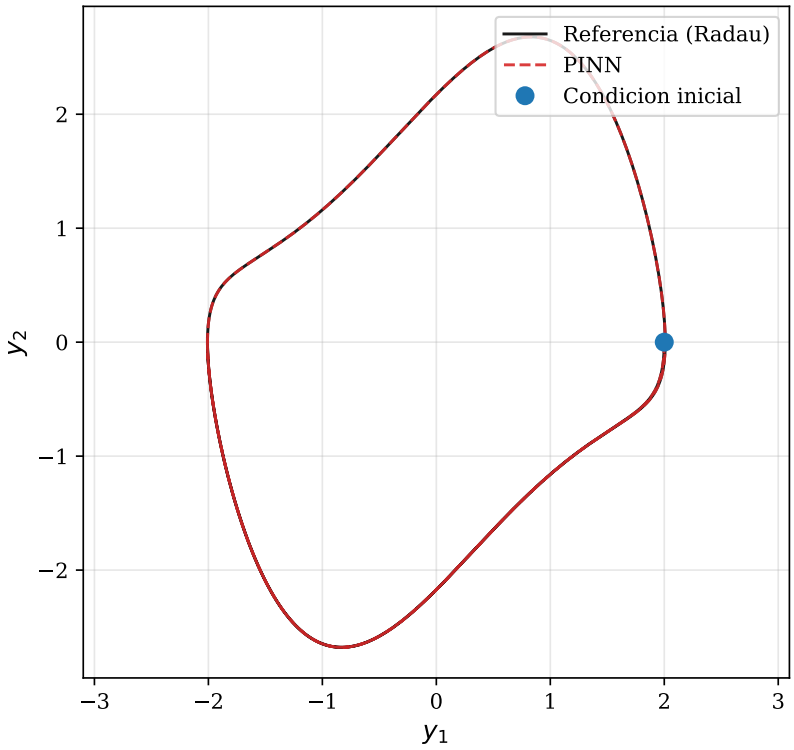


Figura 5.2: Retrato de fase de la solución para $\mu = 1$. La condición inicial $(2, 0)$ está marcada en azul. La trayectoria converge rápidamente al ciclo límite estable característico del oscilador.

Error puntual en el tiempo. La figura 5.3 reporta el error absoluto puntual $|y_{i,\theta}(t) - y_{i,\text{ref}}(t)|$ en escala semilogarítmica. El error oscila entre 10^{-5} y 10^{-3} en todo el dominio, con picos suaves coincidentes con las regiones de mayor curvatura del ciclo de relajación. Las regiones de mayor variación local de la solución constituyen las zonas más exigentes de aproximar, aunque para $\mu = 1$ la diferencia entre regiones lenta y rápida es todavía moderada y la PINN consigue mantener el error acotado.

Evolución de la pérdida durante el entrenamiento. La figura 5.4 presenta el valor de $\mathcal{L}(\theta)$ a lo largo de las iteraciones de Adam y L-BFGS. Se observan dos regímenes claramente diferenciados: una fase inicial de descenso rápido durante las primeras iteraciones de Adam, seguida de una región de progreso lento donde el optimizador estocástico encuentra dificultades para refinar la solución; y una fase final de descenso pronunciado al activarse L-BFGS, donde la información de segundo orden permite reducir la pérdida en aproximadamente cinco órdenes de magnitud adicionales. Esta dinámica es característica de la estrategia híbrida y justifica empíricamente la elección de la prescripción Adam $\rightarrow$ L-BFGS.

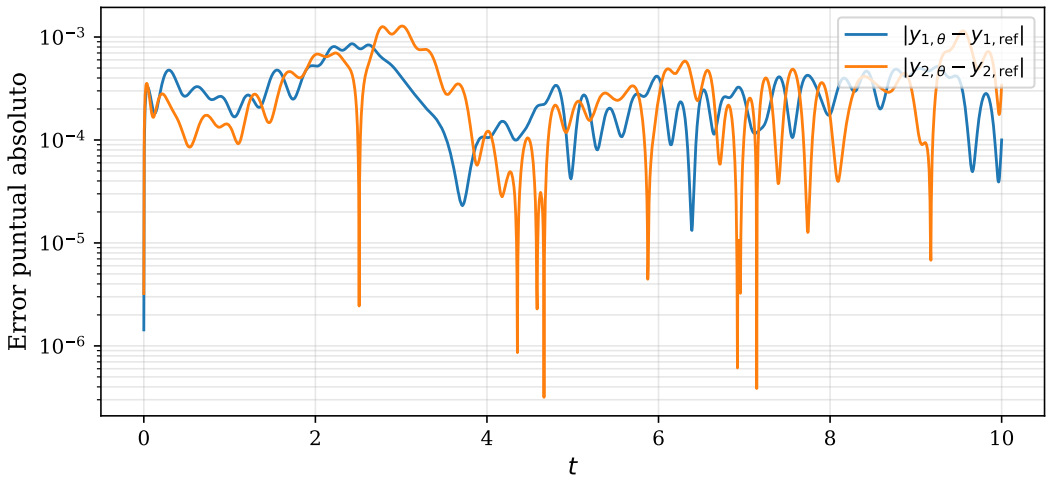


Figura 5.3: Error absoluto puntual entre la PINN y la solución de referencia para las componentes y_1 y y_2, en escala semilogarítmica. El error se mantiene entre 10^{-5} y 10^{-3} en todo el dominio.

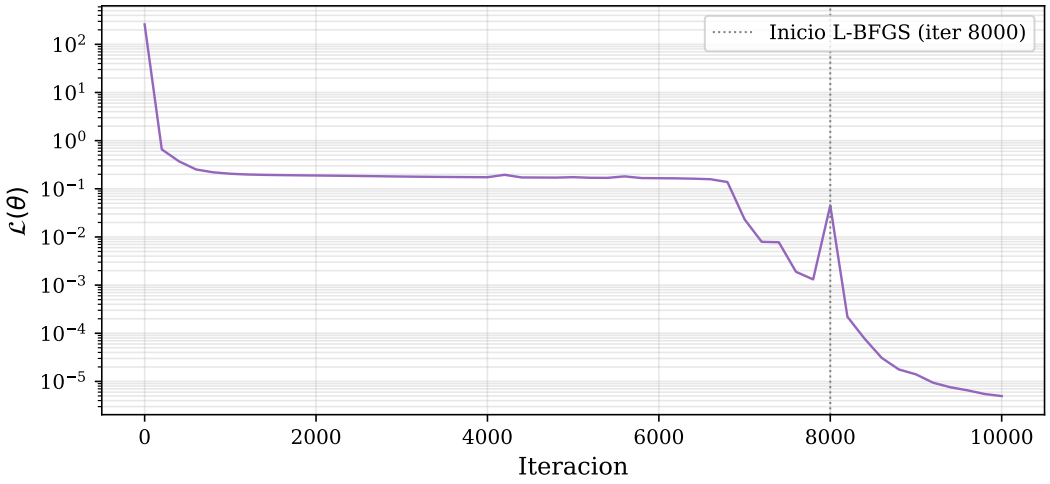


Figura 5.4: Evolución de la función de pérdida $\mathcal{L}(\theta)$ durante el entrenamiento en escala semilogarítmica. La línea vertical punteada marca el inicio de la fase L-BFGS en la iteración 8000.

Síntesis cuantitativa. La tabla 5.1 consolida los indicadores numéricos de calidad del experimento base.

5.1.5. Estudios de sensibilidad

El experimento base resuelve el problema de manera satisfactoria, pero la configuración elegida —cinco capas ocultas de veinte neuronas, activación tanh, $K = 5$ frecuencias de Fourier, pesos $\lambda_0 = 100$ y $\lambda_f = 1$— es solo una entre muchas posibles. Los estudios de sensibilidad que se presentan a continuación cuantifican cómo varía la calidad de la solución al modificar cuatro ejes de la configuración: la topología de la red, la función de activación, la ponderación de la condición inicial, y el parámetro de no linealidad μ del problema mismo. Cada experimento de sensibilidad se entrena con el mismo presupuesto computacional que el caso base (8000 iteraciones de Adam y 2000

Tabla 5.1: Indicadores de calidad del experimento base para $\mu = 1$, $T = 10$.

Indicador	Valor
Pérdida residual final $\mathcal{L}(\theta^\star)$	4.32×10^{-6}
Error relativo L^2 en y_1 ($\mathcal{E}_{L^2}^{y_1}$)	2.43×10^{-4}
Error relativo L^2 en y_2 ($\mathcal{E}_{L^2}^{y_2}$)	3.07×10^{-4}
Tiempo de la fase Adam	154.7 s
Tiempo de la fase L-BFGS	38.7 s
Tiempo total	193.4 s
Hardware	Apple M4 (MPS)

de L-BFGS) y con la misma semilla aleatoria, lo que aísla el efecto del hiperparámetro estudiado.

El criterio de comparación es uniforme: el error relativo $\mathcal{E}_{L^2}$, reportado por separado para las dos componentes del sistema.

Topología de la red

La tabla 5.2 contrasta cuatro configuraciones de profundidad y ancho, manteniendo fijos el resto de hiperparámetros (activación tanh, $K = 5$ frecuencias, $\lambda_0 = 100$).

Tabla 5.2: Sensibilidad respecto a la topología de la red. Activación tanh, $K = 5$, $\lambda_0 = 100$.

Configuración	$\mathcal{E}_{L^2}^{y_1}$	$\mathcal{E}_{L^2}^{y_2}$	Tiempo (s)
3 capas × 20 neuronas	7.21×10^{-4}	9.97×10^{-4}	139.7
5 capas × 20 neuronas (base)	2.43×10^{-4}	3.07×10^{-4}	144.2
5 capas × 50 neuronas	2.62×10^{-4}	3.56×10^{-4}	173.2
7 capas × 80 neuronas	5.40×10^{-4}	7.65×10^{-4}	142.8

Las cuatro configuraciones convergen exitosamente. La de tres capas con veinte neuronas, la de menor capacidad expresiva, alcanza un error tres veces superior al de la configuración base sin degradación catastrófica: el problema, aunque oscilatorio, es suficientemente bien condicionado para que una arquitectura modesta lo resuelva con precisión del orden de 10^{-3}. La configuración base con cinco capas de veinte neuronas presenta el menor error, mejorando en un factor tres a la configuración de tres capas y resultando marginalmente superior a la de cinco capas con cincuenta neuronas. Aumentar adicionalmente la capacidad a siete capas con ochenta neuronas, en cambio, **degrada el error**: aunque la pérdida residual final disminuye —reflejando un ajuste más fino del residuo sobre los puntos de colocación—, el error L^2 sobre la malla de evaluación independiente aumenta. Este es un caso paradigmático de **sobreajuste sobre los puntos de colocación**: la red de mayor capacidad encuentra modos de oscilación entre puntos de colocación que reducen la pérdida puntual sin mejorar la solución real. La conclusión práctica es que, para este problema, cinco capas de veinte neuronas constituye un compromiso óptimo entre capacidad expresiva y resistencia al sobreajuste.

Función de activación

La tabla 5.3 compara tres funciones de activación sobre la arquitectura base de cinco capas por veinte neuronas, reportando además la pérdida residual final como indicador secundario.

Los resultados son contundentes. La función **sigmoide** falla catastróficamente: el error L^2 es de orden uno, con una pérdida residual final tres órdenes de magnitud superior a la de las otras

Tabla 5.3: Sensibilidad respecto a la función de activación. Arquitectura 5×20, $K = 5$, $\lambda_0 = 100$.

Activación	$\mathcal{E}_{L^2}^{y_1}$	$\mathcal{E}_{L^2}^{y_2}$	Pérdida final
Tangente hiperbólica tanh (base)	2.43×10^{-4}	3.07×10^{-4}	4.32×10^{-6}
Sigmoide σ	4.77	0.98	2.00×10^{-2}
Swish	7.02×10^{-5}	9.24×10^{-5}	1.36×10^{-6}

dos activaciones. La causa es estructural: el rango $(0, 1)$ de la sigmoide es asimétrico respecto al origen, lo que dificulta a las capas internas representar funciones con valores negativos típicos del oscilador. La asimetría obliga a las capas finales a realizar un trabajo de centrado y escalado que las activaciones simétricas resuelven naturalmente, y la saturación a ambos extremos del rango ralentiza la convergencia.

La comparación entre tanh y **Swish** ($\sigma(x) \cdot x$), por su parte, arroja un hallazgo **empíricamente inesperado**: Swish supera a tanh por un factor superior a tres, alcanzando un error L^2 de 7.02×10^{-5} frente a 2.43×10^{-4} de la tangente hiperbólica. La pérdida residual final también es menor con Swish, lo que sugiere que la mejoría no es un artefacto de mejor adaptación a los puntos de colocación sino una representación más fiel de la solución completa. La interpretación es plausible: Swish, al ser no monótona y no acotada, posee una capacidad expresiva intrínsecamente mayor que las activaciones acotadas, y su gradiente no se satura para entradas grandes, lo que facilita el aprendizaje de las regiones de mayor curvatura del ciclo límite. Pese a estos resultados, la elección de tanh como activación base se mantiene en el resto de los experimentos del capítulo por dos razones complementarias: (i) es el estándar en la literatura de PINNs, lo que facilita la comparación con trabajos previos; (ii) su gradiente acotado proporciona una mayor estabilidad numérica en problemas con derivadas de orden superior —como los que se abordan en las secciones siguientes—, donde la falta de acotamiento de Swish puede generar inestabilidades durante el entrenamiento.

Ponderación de la condición inicial

La tabla 5.4 contrasta tres elecciones del peso λ_0 en la pérdida, manteniendo $\lambda_f = 1$ y el resto de hiperparámetros en sus valores base.

Tabla 5.4: Sensibilidad respecto a la ponderación de la condición inicial. Arquitectura 5×20, tanh, $K = 5$, $\lambda_f = 1$.

λ_0	$\mathcal{E}_{L^2}^{y_1}$	$\mathcal{E}_{L^2}^{y_2}$	Pérdida final
1	1.06×10^{-4}	1.49×10^{-4}	6.38×10^{-7}
100 (base)	2.43×10^{-4}	3.07×10^{-4}	4.32×10^{-6}
1000	4.77×10^{-3}	6.78×10^{-3}	6.44×10^{-5}

Este estudio arroja un segundo hallazgo; la configuración con $\lambda_0 = 1$ —pesos iguales para la condición inicial y el residuo— alcanza un error inferior al caso base con $\lambda_0 = 100$, con una mejora aproximada de un factor dos en ambas componentes. El oscilador de Van der Pol tiene un campo vectorial muy informativo cerca de la condición inicial. La trayectoria que parte de $(2, 0)$ tiene un primer tramo cuyo gradiente está determinado casi por completo por $\mathbf{f}(\mathbf{y}(0))$, lo que significa que satisfacer el residuo en los puntos de colocación tempranos conduce automáticamente a una buena aproximación de la condición inicial. Sobre-pesar la componente $\mathcal{L}_0$ resulta entonces redundante y desvía recursos del optimizador hacia un objetivo ya cubierto por el residuo, a costa del refinamiento de la solución en el resto del dominio. Para $\lambda_0 = 1000$ el efecto es más severo: la pérdida queda dominada por la componente $\mathcal{L}_0$ y el optimizador prácticamente ignora el residuo, generando una

solución que satisface la condición inicial con precisión innecesaria mientras pierde fidelidad en el interior del dominio.

Este hallazgo no debe generalizarse: en problemas con condiciones iniciales menos informativas, o en problemas con condiciones de frontera espacial donde $\mathcal{L}_0$ y $\mathcal{L}_f$ son ambas necesarias para fijar la solución, la asimetría en los pesos vuelve a ser pertinente. La elección de $\lambda_0 = 100$ como valor base se mantiene por su carácter de estándar empírico robusto en la literatura general de PINNs; la ganancia adicional disponible con $\lambda_0 = 1$ se reporta aquí como un descubrimiento del estudio de sensibilidad.

Régimen rígido: variación de μ

Como prueba de robustez frente a la dificultad del problema, la tabla 5.5 reporta los resultados de la configuración base aplicada al sistema con $\mu \in \{1, 2, 3, 5\}$, donde el carácter rígido se intensifica progresivamente: la duración relativa de las transiciones disminuye y los gradientes locales del campo vectorial alcanzan magnitudes crecientes.

Tabla 5.5: Desempeño en función del parámetro de no linealidad μ. Arquitectura 5×20, tanh, $K = 5$, $\lambda_0 = 100$.

μ	$\mathcal{E}_{L^2}^{y_1}$	$\mathcal{E}_{L^2}^{y_2}$	Pérdida final
1 (base)	2.43×10^{-4}	3.07×10^{-4}	4.32×10^{-6}
2	0.79	1.04	3.94×10^{-6}
3	0.82	0.99	5.51×10^{-6}
5	0.81	1.00	2.78×10^{-6}

Los resultados muestran un patrón muy específico y conceptualmente importante. Al pasar de $\mu = 1$ a $\mu = 2$, el error L^2 se degrada bruscamente del orden 10^{-4} al orden uno —es decir, la solución de la red deja de ser representativa de la solución verdadera—, y este nivel de error se mantiene aproximadamente constante para $\mu = 3$ y $\mu = 5$. Lo más revelador es que la pérdida residual final permanece pequeña, del mismo orden que en el caso base. En otras palabras, **la red ha minimizado el residuo del sistema sin haber encontrado la solución verdadera**.

Este fenómeno se explica por la coexistencia de múltiples mínimos locales en el paisaje de la función de pérdida. Para $\mu \geq 2$ existen soluciones falsas que satisfacen el residuo en los puntos de colocación con alta precisión pero no corresponden a la trayectoria del sistema desde la condición inicial $(2, 0)$. El optimizador, partiendo de una inicialización aleatoria, puede converger a uno de estos mínimos ilegítimos sin que la pérdida indique problema alguno. La condición inicial impuesta de forma suave con $\lambda_0 = 100$ resulta insuficiente para guiar al optimizador hacia el mínimo correcto, pues el peso relativo sobre $\mathcal{L}_0$ se diluye en cuanto la red satisface el residuo en el resto del dominio.

Este modo de falla constituye una validación empírica de la motivación de las técnicas adaptativas. Las SA-PINNs —con pesos auto-adaptativos por punto de colocación— y los métodos de descomposición de dominio XPINN están específicamente diseñados para superar este modo de falla, asignando mayor peso a regiones del dominio donde la solución es más difícil de capturar y rompiendo el problema en subdominios más tratables. La implementación de SA-PINN aplicada a $\mu = 2, 3, 5$ excede el alcance del presente capítulo y constituye una línea de trabajo futuro.

Conclusiones del estudio de sensibilidad

Los experimentos anteriores permiten extraer cuatro conclusiones aplicables a la resolución del oscilador de Van der Pol mediante PINNs:

1. La topología tiene una zona óptima alrededor de las cinco capas ocultas con veinte neuronas. Por debajo de esa zona, la capacidad expresiva limita la calidad de la aproximación; por encima, aparece sobreajuste sobre los puntos de colocación —reducción de la pérdida acompañada de aumento del error L^2—, fenómeno especialmente claro en la configuración de siete capas con ochenta neuronas.

2. La elección de la función de activación tiene un impacto significativo. La sigmoide falla catastróficamente por su rango asimétrico, mientras que la activación Swish supera a la tangente hiperbólica por un factor superior a tres. Se conserva tanh como activación base por su estabilidad numérica en problemas con derivadas de orden superior, pero los resultados sugieren que Swish merece consideración en estudios futuros sobre problemas de orden bajo.

3. La ponderación λ_0 de la condición inicial admite valores menores a los típicamente recomendados sin perjuicio para la solución, e incluso con ganancia: $\lambda_0 = 1$ logra mejor error que $\lambda_0 = 100$ en este problema específico. La razón es la naturaleza muy informativa del campo vectorial cerca de la condición inicial, que hace redundante el sobre-peso. El valor estándar $\lambda_0 = 100$ se mantiene como base por su robustez en problemas con condiciones iniciales menos informativas.

4. La aplicabilidad de la formulación PINN básica está estrictamente acotada por el parámetro de rigidez del problema. Para $\mu = 1$ la convergencia es robusta y de alta precisión; para $\mu \geq 2$ el optimizador converge a mínimos locales ilegítimos con pérdida residual pequeña pero error global del orden uno. Esta degradación motiva empíricamente las extensiones adaptativas.

Estas conclusiones se contrastarán con las obtenidas en los problemas de las secciones siguientes para identificar patrones de comportamiento que trascienden la ecuación específica.

5.2. Ecuación de Onda en 1D

Tras validar la metodología PINN sobre una ecuación diferencial ordinaria no lineal y rígida, continuamos con su primera aplicación a una ecuación diferencial parcial. La ecuación de onda unidimensional constituye el ejemplo canónico de ecuación hiperbólica de segundo orden. Su estudio permite verificar el comportamiento de las PINN sobre un problema con dependencia simultánea en variables espacial y temporal, con condiciones de frontera no triviales, y con un operador diferencial que involucra derivadas de segundo orden.

5.2.1. Planteamiento del problema

Se considera el problema de valor inicial y de frontera para la ecuación de onda unidimensional homogénea con coeficiente constante,

$$\frac{\partial^2 u}{\partial t^2}(x,t) \;=\; c^2 \frac{\partial^2 u}{\partial x^2}(x,t), \qquad (x,t) \in [0,L] \times [0,T], \tag{5.12}$$

donde $u : [0,L] \times [0,T] \to \mathbb{R}$ es la incógnita y $c > 0$ es la velocidad de propagación de las ondas. La ecuación (5.12) modela una multitud de fenómenos físicos, todos ellos caracterizados por la conservación de la energía y la inexistencia de mecanismos disipativos.

El problema se cierra con condiciones iniciales y de frontera. Las condiciones iniciales prescriben el estado del sistema en $t = 0$:

$$u(x,0) = f(x), \qquad \frac{\partial u}{\partial t}(x,0) = g(x), \qquad x \in [0,L], \tag{5.13}$$

y las condiciones de frontera, de tipo Dirichlet homogéneas, fijan los valores de la solución en los extremos espaciales del dominio durante todo el intervalo temporal:

$$u(0,t) = 0, \qquad u(L,t) = 0, \qquad t \in [0,T]. \tag{5.14}$$

Físicamente, las condiciones de frontera corresponden al caso de una cuerda con ambos extremos fijos. Las funciones f y g representan, respectivamente, el desplazamiento inicial y la velocidad inicial del medio.

Para los experimentos numéricos de este capítulo se adoptan los parámetros

$$L = 1, \qquad T = 1, \qquad c = 2, \tag{5.15}$$

con condiciones iniciales

$$f(x) = \sin(\pi x), \qquad g(x) = 0, \tag{5.16}$$

elección que sitúa la prueba en el régimen donde el método PINN básico converge de manera confiable y la solución analítica es particularmente simple.

Solución analítica de referencia. El problema admite una solución cerrada que se obtiene de manera directa mediante separación de variables. Se postula $u(x,t) = \phi(x)\,\psi(t)$, lo que conduce a la pareja de ecuaciones diferenciales ordinarias $\phi''(x) + \lambda\phi(x) = 0$ y $\psi''(t) + c^2\lambda\psi(t) = 0$, con λ constante de separación. Las condiciones de frontera fuerzan $\lambda = (n\pi/L)^2$ con n entero positivo, y los modos espaciales son $\phi_n(x) = \sin(n\pi x/L)$. La superposición general

$$u(x,t) \;=\; \sum_{n=1}^{\infty} \Big[A_n \cos\Big(\frac{n\pi c}{L} t\Big) + B_n \sin\Big(\frac{n\pi c}{L} t\Big) \Big] \sin\Big(\frac{n\pi}{L} x\Big) \tag{5.17}$$

incorpora las condiciones iniciales a través de los coeficientes de Fourier. Para $f(x) = \sin(\pi x)$ sobre $[0,1]$, el desarrollo en serie de Fourier reduce la suma a un único término, $A_1 = 1$ y todos los demás $A_n = 0$. La condición $g(x) = 0$ implica $B_n = 0$ para todo n. Por tanto, la solución exacta del problema es

$$u(x,t) \;=\; \sin(\pi x)\,\cos(2\pi t), \tag{5.18}$$

una **onda estacionaria** cuyo perfil espacial es el primer modo de la cuerda y cuya amplitud oscila armónicamente en el tiempo con periodo $T_{\text{osc}} = 2L/c = 1$. Sobre el intervalo $[0,T] = [0,1]$ se observa por tanto exactamente un periodo temporal completo de la oscilación, lo que le da al experimento una cobertura no trivial del comportamiento dinámico de la solución sin requerir tiempos de simulación prolongados.

Características del problema relevantes para PINNs. La elección de la onda estacionaria como caso base implica tres propiedades que conviene anticipar para interpretar los resultados:

- *Acotamiento.* La amplitud $|u(x,t)|$ está acotada por 1 en todo el dominio, lo que evita los problemas numéricos asociados a soluciones que crecen exponencialmente o que presentan singularidades. La normalización implícita facilita la convergencia del optimizador.
- *Suavidad.* La solución (5.18) es analítica en (x,t), por lo que admite expansiones de Taylor en cualquier punto. Esta regularidad coincide con la regularidad de las funciones representables por redes neuronales con activaciones de clase C^∞, situando al problema en un escenario favorable para la aproximación.
- *Compatibilidad estructural con bases de Fourier.* Los modos del problema son funciones sinusoidales en x con frecuencias $n\pi/L$. Un embedding de Fourier compatible con estas frecuencias en la entrada de la red permite que las capas internas aprovechen una base de funciones próxima a la solución verdadera.

5.2.2. Formulación PINN para la ecuación de onda

La aplicación del marco PINN al problema sigue los pasos generales del capítulo anterior, con dos diferencias estructurales respecto al caso del oscilador de Van der Pol: el dominio es bidimensional —(x, t) en vez de únicamente t— y el operador diferencial incluye derivadas de segundo orden tanto en x como en t. Estas dos diferencias dictan, respectivamente, la dimensión de la capa de entrada de la red y el orden de la diferenciación automática anidada que se requerirá durante el cálculo del residuo.

La red neuronal aproximadora se denota

$$u_\theta : [0, L] \times [0, T] \to \mathbb{R}, \qquad (x, t) \mapsto u_\theta(x, t), \tag{5.19}$$

donde θ representa colectivamente todos los pesos y sesgos de la red. La capa de entrada consta de dos neuronas que reciben el par (x, t); la capa de salida tiene una única neurona con activación lineal que produce el valor de u en ese punto. Las capas ocultas y su estructura interna son objeto de análisis sistemático.

Residuo de la ecuación. El residuo asociado a la ecuación de onda se define como

$$r_\theta(x, t) \;:=\; \frac{\partial^2 u_\theta}{\partial t^2}(x, t) \;-\; c^2 \frac{\partial^2 u_\theta}{\partial x^2}(x, t). \tag{5.20}$$

La evaluación numérica de este mismo requiere el cómputo de cuatro derivadas parciales hasta segundo orden mediante diferenciación automática. En PyTorch, esto se realiza encadenando dos llamadas a `torch.autograd.grad`: la primera obtiene $\partial u_\theta/\partial t$ (respectivamente $\partial u_\theta/\partial x$) a partir del grafo computacional asociado al *forward pass* del modelo; la segunda obtiene $\partial^2 u_\theta/\partial t^2$ a partir del grafo asociado a la primera derivada. Ambas invocaciones se realizan con `create_graph=True`, lo que permite calcular posteriormente los gradientes de la pérdida total respecto a los parámetros θ a través de toda la cadena.

Estructura cuádruple de la pérdida. A diferencia del problema de valor inicial puro del oscilador de Van der Pol, donde la pérdida se descomponía en sólo dos términos (condición inicial y residuo), el problema de valor inicial y de frontera de la ecuación de onda presenta cuatro restricciones independientes: la condición sobre la posición inicial $u(x, 0) = f(x)$, la condición sobre la velocidad inicial $u_t(x, 0) = g(x)$, las condiciones de frontera de Dirichlet $u(0, t) = u(L, t) = 0$, y el residuo de la PDE en el interior del dominio. La pérdida total se descompone, en consecuencia, en cuatro contribuciones:

$$\mathcal{L}(\theta) \;=\; \lambda_r \, \mathcal{L}_r(\theta) \;+\; \lambda_{IC,1} \, \mathcal{L}_{IC,1}(\theta) \;+\; \lambda_{IC,2} \, \mathcal{L}_{IC,2}(\theta) \;+\; \lambda_{BC} \, \mathcal{L}_{BC}(\theta). \tag{5.21}$$

Cada componente es el promedio cuadrático del residuo correspondiente sobre un conjunto de puntos

de muestreo:

$$\mathcal{L}_r(\theta) = \frac{1}{N_r}\sum_{i=1}^{N_r}\left|r_\theta(x_r^{(i)}, t_r^{(i)})\right|^2, \tag{5.22}$$

$$\mathcal{L}_{IC,1}(\theta) = \frac{1}{N_0}\sum_{i=1}^{N_0}\left|u_\theta(x_0^{(i)}, 0) - f(x_0^{(i)})\right|^2, \tag{5.23}$$

$$\mathcal{L}_{IC,2}(\theta) = \frac{1}{N_0}\sum_{i=1}^{N_0}\left|\frac{\partial u_\theta}{\partial t}(x_0^{(i)}, 0) - g(x_0^{(i)})\right|^2, \tag{5.24}$$

$$\mathcal{L}_{BC}(\theta) = \frac{1}{2N_b}\sum_{j=1}^{N_b}\Big(\left|u_\theta(0, t_b^{(j)})\right|^2 + \left|u_\theta(L, t_b^{(j)})\right|^2\Big). \tag{5.25}$$

Los puntos $\{(x_r^{(i)}, t_r^{(i)})\}_{i=1}^{N_r}$ se distribuyen en el dominio interior $[0, L] \times [0, T]$, los puntos $\{x_0^{(i)}\}_{i=1}^{N_0}$ se distribuyen sobre la condición inicial en $t = 0$, y los puntos $\{t_b^{(j)}\}_{j=1}^{N_b}$ se distribuyen sobre el intervalo temporal y sirven para muestrear simétricamente ambos extremos espaciales.

Ponderación de los términos. La justificación de los pesos λ_r, $\lambda_{IC,1}$, $\lambda_{IC,2}$ y λ_{BC} es análoga a la discutida para el oscilador de Van der Pol, generalizada al contexto multidimensional. La componente $\mathcal{L}_r$ se evalúa sobre un volumen del dominio $[0, L] \times [0, T]$, mientras que $\mathcal{L}_{IC,1}$ y $\mathcal{L}_{IC,2}$ se evalúan sobre la línea $t = 0$ y $\mathcal{L}_{BC}$ se evalúa sobre las dos líneas $x \in \{0, L\}$. Las tres restricciones de datos tienen, en consecuencia, una dimensión geométrica menor que el residuo y un peso natural inferior en la pérdida promediada. Para contrarrestar este desequilibrio se adopta

$$\lambda_r = 1, \qquad \lambda_{IC,1} = \lambda_{IC,2} = \lambda_{BC} = 100, \tag{5.26}$$

combinación que constituye el estándar empírico para problemas hiperbólicos en PINNs sin imposición arquitectónica de las restricciones.

Selección de la función de activación. La presencia de derivadas de segundo orden en el residuo impone una condición de regularidad más estricta sobre la función de activación que la del problema de la sección anterior. Se requiere que la activación sea al menos de clase C^2 —es decir, dos veces diferenciable con derivadas continuas—, de modo que la diferenciación automática anidada produzca resultados bien definidos y estables. La **tangente hiperbólica** tanh satisface esta condición.

Distribución de los puntos de muestreo. La generación de los conjuntos $\{(x_r, t_r)\}$, $\{x_0\}$ y $\{t_b\}$ se realiza mediante muestreo aleatorio uniforme sobre los dominios respectivos. Se emplean

$$N_r = 2{,}000, \qquad N_0 = 200, \qquad N_b = 200, \tag{5.27}$$

puntos en cada caso. La proporción $N_0/N_r = N_b/N_r = 0.1$ refleja, por un lado, la menor dimensión geométrica de las regiones donde se imponen las restricciones de datos, y por otro lado el hecho empírico de que —reforzadas por los pesos $\lambda_{IC,1}, \lambda_{IC,2}, \lambda_{BC} = 100$— las restricciones de datos se satisfacen con holgura aun con un número modesto de puntos.

Algoritmo de optimización. La pérdida se minimiza con la misma estrategia híbrida Adam → L-BFGS adoptada para el problema anterior. La primera fase, de 5000 iteraciones, emplea Adam con tasa de aprendizaje $\eta = 10^{-3}$ como búsqueda exploratoria. La segunda fase, de un máximo de 2500 iteraciones, emplea L-BFGS con búsqueda de línea de Wolfe para el refinamiento cuasi-Newton aprovechando información de segundo orden.

5.2.3. Implementación en PyTorch

La implementación combina el modelo neuronal con embedding de Fourier bidimensional, la función de pérdida de cuatro términos, el muestreo de los tres conjuntos de puntos y el bucle de entrenamiento. Cada uno de estos componentes se presenta en un fragmento de código a continuación.

Embedding de Fourier bidimensional. La estrategia descrita en la subsección 5.1.3 para mitigar el sesgo espectral se generaliza al dominio bidimensional. El escalar (x, t) se mapea al vector

$$\boldsymbol{\varphi}(x,t) = \left(\tfrac{x}{L},\ \tfrac{t}{T},\ \{\sin(\omega_k^x x), \cos(\omega_k^x x)\}_{k=1}^{K_x},\ \{\sin(\omega_\ell^t t), \cos(\omega_\ell^t t)\}_{\ell=1}^{K_t} \right)^\top, \tag{5.28}$$

con frecuencias $\omega_k^x = k\pi/L$ y $\omega_\ell^t = 2\pi\ell/T$. Las primeras coinciden exactamente con las frecuencias de los modos propios de Dirichlet del problema, lo que provee a la red una base sinusoidal estructuralmente compatible con la solución. Se adopta $K_x = K_t = 4$ —valor modesto que cubre los primeros cuatro modos en cada dimensión y resulta suficiente para representar la solución con holgura sin introducir sesgos hacia periodos proporcionales específicos—.

```python
import torch
import torch.nn as nn
import numpy as np

DTYPE = torch.float32


class Swish(nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x)


ACTIVATIONS = {'tanh': nn.Tanh, 'sigmoid': nn.Sigmoid, 'swish': Swish}


class FourierEmbedding2D(nn.Module):
    """phi(x, t) = [x/L, t/T, sin/cos(omega_x_k x), sin/cos(omega_t_l t)]."""
    def __init__(self, n_freqs_x=4, n_freqs_t=4, L=1.0, T=1.0):
        super().__init__()
        freqs_x = torch.arange(1, n_freqs_x + 1, dtype=DTYPE)
        freqs_t = torch.arange(1, n_freqs_t + 1, dtype=DTYPE)
        self.register_buffer('omega_x', np.pi * freqs_x / L)
        self.register_buffer('omega_t', 2.0 * np.pi * freqs_t / T)
        self.L, self.T = L, T
        self.n_freqs_x, self.n_freqs_t = n_freqs_x, n_freqs_t

    @property
    def output_dim(self):
        return 2 + 2 * self.n_freqs_x + 2 * self.n_freqs_t

    def forward(self, x, t):
        xx = x * self.omega_x.view(1, -1)
        tt = t * self.omega_t.view(1, -1)
        return torch.cat([
            x / self.L, t / self.T,
            torch.sin(xx), torch.cos(xx),
            torch.sin(tt), torch.cos(tt),
        ], dim=1)

```

```

class WaveModel(nn.Module):
    """phi(x,t) -> 5 capas x 20 neuronas (tanh) -> u(x,t)."""
    def __init__(self, n_layers=5, n_neurons=20, activation='tanh',
                 n_freqs_x=4, n_freqs_t=4, L=1.0, T=1.0):
        super().__init__()
        Act = ACTIVATIONS[activation]
        self.embed = FourierEmbedding2D(n_freqs_x, n_freqs_t, L, T)
        in_dim = self.embed.output_dim
        layers = [nn.Linear(in_dim, n_neurons), Act()]
        for _ in range(n_layers - 1):
            layers += [nn.Linear(n_neurons, n_neurons), Act()]
        layers += [nn.Linear(n_neurons, 1)]
        self.net = nn.Sequential(*layers)
        for m in self.net.modules():
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                nn.init.zeros_(m.bias)

    def forward(self, x, t):
        return self.net(self.embed(x, t))
```

Listing 5.5: Modelo neuronal con embedding de Fourier bidimensional.

Función de pérdida. El siguiente fragmento implementa el cálculo de los cuatro términos (5.22)–(5.25) y devuelve tanto la pérdida ponderada total como las componentes individuales para fines de monitoreo. Las dobles llamadas a `torch.autograd.grad` —necesarias para obtener u_{tt} y u_{xx}— constituyen el núcleo computacionalmente más costoso del entrenamiento.

```
def f_ic(x):  # condicion inicial: u(x, 0) = sin(pi x)
    return torch.sin(np.pi * x)

def g_ic(x):  # velocidad inicial: u_t(x, 0) = 0
    return torch.zeros_like(x)


def pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, c,
              lambda_r=1.0, lambda_ic1=100.0,
              lambda_ic2=100.0, lambda_bc=100.0):
    # ----- Residuo PDE: u_tt - c^2 u_xx -----
    x = x_int.clone().requires_grad_(True)
    t = t_int.clone().requires_grad_(True)
    u = model(x, t)
    u_t  = torch.autograd.grad(u,   t, torch.ones_like(u),   create_graph=True)[0]
    u_tt = torch.autograd.grad(u_t, t, torch.ones_like(u_t), create_graph=True)[0]
    u_x  = torch.autograd.grad(u,   x, torch.ones_like(u),   create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)[0]
    r = u_tt - c**2 * u_xx
    L_r = torch.mean(r**2)

    # ----- CI 1: u(x, 0) = f(x) -----
    t_zero = torch.zeros_like(x_ic)
    u_ic = model(x_ic, t_zero)
    L_ic1 = torch.mean((u_ic - f_ic(x_ic))**2)

    # ----- CI 2: u_t(x, 0) = g(x) -----
    t_zero_grad = torch.zeros_like(x_ic).requires_grad_(True)
    u_ic_v = model(x_ic, t_zero_grad)
    u_t_ic = torch.autograd.grad(
```

```
        u_ic_v, t_zero_grad, torch.ones_like(u_ic_v), create_graph=True
    )[0]
    L_ic2 = torch.mean((u_t_ic - g_ic(x_ic))**2)

    # ----- CF: u(0, t) = u(L, t) = 0 -----
    u_bc = model(x_bc, t_bc)
    L_bc = torch.mean(u_bc**2)

    L_total = (lambda_r * L_r + lambda_ic1 * L_ic1
               + lambda_ic2 * L_ic2 + lambda_bc * L_bc)
    return L_total, L_r, L_ic1, L_ic2, L_bc
```

Listing 5.6: Función de pérdida para la ecuación de onda.

Muestreo y bucle de entrenamiento. La rutina `sample_points` genera los tres conjuntos requeridos: N_r puntos en el interior, N_0 puntos sobre la condición inicial y N_b puntos sobre cada borde espacial. La función `train` encapsula la estrategia Adam $\rightarrow$ L-BFGS.

```
def sample_points(n_interior=2000, n_ic=200, n_bc=200, L=1.0, T=1.0, seed=42):
    torch.manual_seed(seed); np.random.seed(seed)
    x_int = torch.rand(n_interior, 1, dtype=DTYPE) * L
    t_int = torch.rand(n_interior, 1, dtype=DTYPE) * T
    x_ic  = torch.rand(n_ic, 1, dtype=DTYPE) * L
    t_bc  = torch.rand(2 * n_bc, 1, dtype=DTYPE) * T
    x_bc  = torch.cat([
        torch.zeros(n_bc, 1, dtype=DTYPE),
        torch.full((n_bc, 1), float(L), dtype=DTYPE),
    ], dim=0)
    return x_int, t_int, x_ic, x_bc, t_bc


def train(model, c, L=1.0, T=1.0, n_adam=5000, n_lbfgs=2500,
          n_interior=2000, n_ic=200, n_bc=200,
          lr_adam=1e-3, lambda_data=100.0, seed=42):
    x_int, t_int, x_ic, x_bc, t_bc = sample_points(
        n_interior=n_interior, n_ic=n_ic, n_bc=n_bc, L=L, T=T, seed=seed,
    )

    # Fase 1: Adam
    opt = torch.optim.Adam(model.parameters(), lr=lr_adam)
    for i in range(n_adam):
        opt.zero_grad()
        L_total, *_ = pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, c,
                                lambda_ic1=lambda_data,
                                lambda_ic2=lambda_data,
                                lambda_bc=lambda_data)
        L_total.backward()
        opt.step()

    # Fase 2: L-BFGS
    opt = torch.optim.LBFGS(
        model.parameters(), lr=1.0, max_iter=n_lbfgs,
        tolerance_grad=1e-11, tolerance_change=1e-13,
        history_size=100, line_search_fn='strong_wolfe',
    )
    def closure():
        opt.zero_grad()
        L_total, *_ = pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, c,
                                lambda_ic1=lambda_data,
```

```
                                    lambda_ic2=lambda_data,
                                    lambda_bc=lambda_data)
        L_total.backward()
        return L_total
    opt.step(closure)
    L_total, *_ = pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, c,
                            lambda_ic1=lambda_data,
                            lambda_ic2=lambda_data,
                            lambda_bc=lambda_data)
    return L_total.item()
```

Listing 5.7: Muestreo de puntos y bucle de entrenamiento.

El código se ha probado en PyTorch 2.12.0 sobre el *backend* MPS de Apple Silicon. El experimento base tomó aproximadamente cinco minutos en un chip M4 (frente a los tres minutos del oscilador de Van der Pol), incremento explicado por el costo de la diferenciación automática anidada: cada evaluación del residuo realiza cuatro retropropagaciones a través del grafo computacional, frente a las dos que requería el sistema dinámico de la sección anterior.

5.2.4. Resultados y comparación con la solución de referencia

La ejecución del procedimiento descrito produce una red entrenada cuya evaluación sobre una malla densa de 201×201 puntos en $[0, L] \times [0, T]$ proporciona la aproximación numérica $u_\theta(x_i, t_j)$ que se compara con la solución exacta evaluada sobre la misma malla. Los indicadores cuantitativos de calidad del experimento base son los siguientes: el **error relativo** L^2 alcanza $\mathcal{E}_{L^2} = 2.66 \times 10^{-3}$, el **error puntual máximo** es $\mathcal{E}_{\text{máx}} = 4.33 \times 10^{-3}$ y la **pérdida residual final** es $\mathcal{L}(\theta^\star) = 4.54 \times 10^{-4}$. El tiempo total de entrenamiento fue de 299.8 segundos en un chip Apple M4 con *backend* MPS.

Una característica destacada del resultado, ya anticipada en la discusión sobre el desequilibrio dimensional de los términos de la pérdida, es la disparidad entre las magnitudes finales de cada componente. Al término del entrenamiento, las cuatro contribuciones son

$$\mathcal{L}_r = 4.40 \times 10^{-4}, \quad \mathcal{L}_{IC,1} = 1.68 \times 10^{-7}, \quad \mathcal{L}_{IC,2} = 1.95 \times 10^{-7}, \quad \mathcal{L}_{BC} = 7.72 \times 10^{-7},$$

es decir, el residuo de la PDE es **tres a cuatro órdenes de magnitud mayor** que cualquier componente de datos. Esta asimetría confirma el diagnóstico estructural: las restricciones de datos —condiciones puntuales o lineales— se satisfacen con holgura una vez que la red dispone de la flexibilidad funcional adecuada, mientras que el residuo es una condición de volumen exigida en todo el interior del dominio y constituye el cuello de botella numérico real del problema.

La calidad de la aproximación se examina visualmente a través de cuatro figuras complementarias.

Mapas de la solución y el error. La figura 5.5 muestra, en tres paneles alineados, la solución de la red, la referencia analítica y el mapa del error puntual. Los dos primeros paneles son visualmente indistinguibles entre sí: ambos exhiben el patrón característico de una onda estacionaria, con franjas de amplitud máxima centradas en $x = 0.5$ que invierten su signo periódicamente en t. El tercer panel revela la estructura espacial del error de aproximación, organizada en bandas suaves cuyos picos se concentran preferentemente en las zonas de mayor amplitud de la solución; los valores máximos no superan 4.33×10^{-3} y el patrón espacial muestra el carácter sistemático —no estocástico— del error de la red.

Cortes temporales. La figura 5.6 muestra el perfil espacial $u(x, t_k)$ para cuatro instantes característicos $t_k \in \{0,\ 0.25,\ 0.5,\ 0.75\}$. Estos valores coinciden, respectivamente, con la amplitud máxima

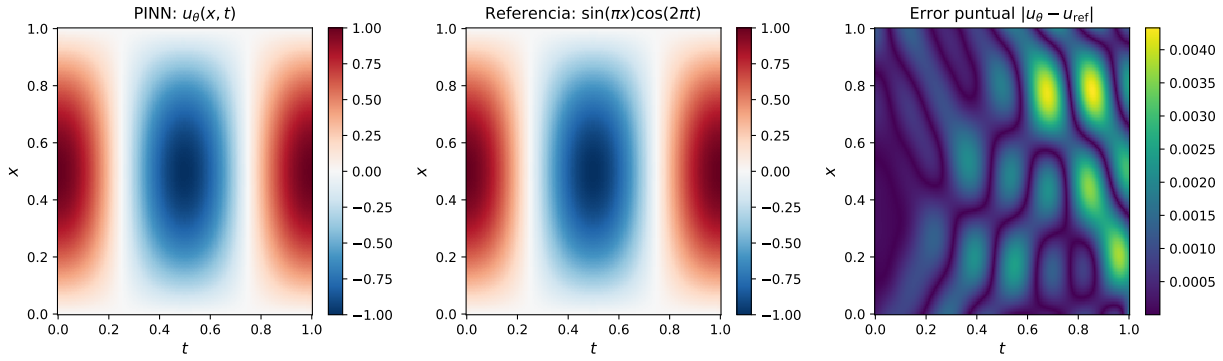


Figura 5.5: Comparación entre la solución de la PINN (izquierda) y la solución analítica $u(x,t) = \sin(\pi x)\cos(2\pi t)$ (centro). El panel derecho muestra el error puntual $|u_\theta - u_{\text{ref}}|$, cuyos picos no superan 4.33×10^{-3}.

inicial ($\cos(0) = 1$), el primer paso por cero ($\cos(\pi/2) = 0$), la amplitud mínima ($\cos(\pi) = -1$) y el segundo paso por cero ($\cos(3\pi/2) = 0$). En todos los cortes la curva de la PINN coincide visualmente con la referencia. Los cortes en $t = 0.25$ y $t = 0.75$ son particularmente informativos: aunque la amplitud teórica es exactamente nula, la PINN produce una recta horizontal indistinguible del eje, lo que evidencia la correcta cancelación entre los términos del residuo en esas regiones.

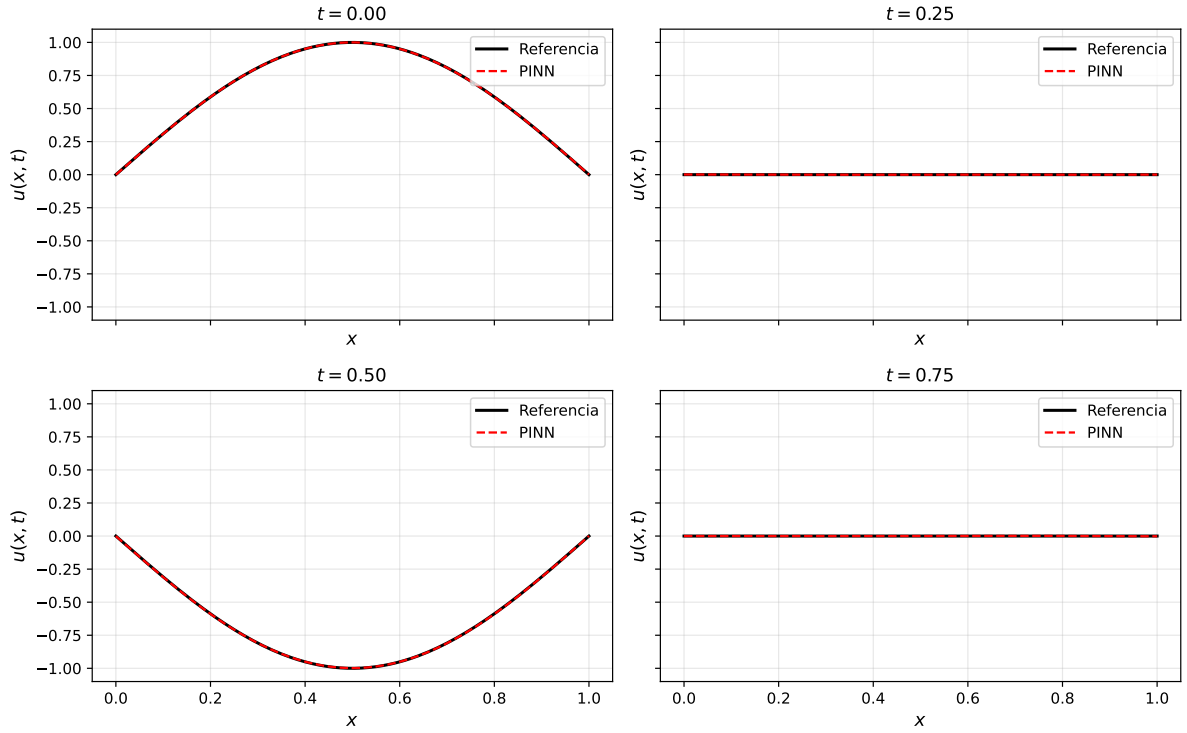


Figura 5.6: Cortes temporales $u(x, t_k)$ para $t_k \in \{0, 0.25, 0.5, 0.75\}$. La curva continua negra es la referencia analítica; la discontinua roja es la PINN. La coincidencia visual es absoluta.

Síntesis cuantitativa. La tabla 5.6 consolida los indicadores numéricos del experimento base.

Tabla 5.6: Indicadores de calidad del experimento base para la ecuación de onda con $c = 2$, $L = T = 1$ y condición inicial $u(x, 0) = \sin(\pi x)$, $u_t(x, 0) = 0$.

Indicador	Valor
Pérdida total final $\mathcal{L}(\theta^\star)$	4.54×10^{-4}
Componente $\mathcal{L}_r$ (residuo PDE)	4.40×10^{-4}
Componente $\mathcal{L}_{IC,1}$ (posición inicial)	1.68×10^{-7}
Componente $\mathcal{L}_{IC,2}$ (velocidad inicial)	1.95×10^{-7}
Componente $\mathcal{L}_{BC}$ (frontera Dirichlet)	7.72×10^{-7}
Error relativo L^2 $\mathcal{E}_{L^2}$	2.66×10^{-3}
Error puntual máximo $\mathcal{E}_{\text{máx}}$	4.33×10^{-3}
Tiempo de la fase Adam	202.6 s
Tiempo de la fase L-BFGS	97.2 s
Tiempo total	299.8 s
Hardware	Apple M4 (MPS)

El experimento valida el funcionamiento del marco PINN para una ecuación diferencial parcial hiperbólica de segundo orden, con error relativo del orden 10^{-3} obtenido en aproximadamente cinco minutos de cómputo sobre hardware estándar. La precisión alcanzada es comparable a la del oscilador de Van der Pol pero con un grado de magnitud menor; este margen refleja la mayor dificultad estructural del problema bidimensional con cuatro restricciones simultáneas, y servirá como punto de comparación natural para los problemas parabólicos y estacionarios de las secciones siguientes. Antes de avanzar a esos casos, la siguiente sección analiza la sensibilidad del resultado base respecto a los hiperparámetros principales: topología de la red, función de activación, ponderación de los términos de la pérdida y velocidad de propagación c.

5.2.5. Estudios de sensibilidad

Análogamente al estudio realizado para el oscilador de Van der Pol, se examina la sensibilidad de la formulación PINN para la ecuación de onda al variar cuatro ejes de su configuración: la topología de la red, la función de activación, la ponderación de los términos de datos en la pérdida y la velocidad de propagación c del problema. Cada experimento de sensibilidad se entrena con el mismo presupuesto computacional que el caso base (5000 iteraciones de Adam y 2500 de L-BFGS) y con la misma semilla aleatoria, lo que aísla el efecto del hiperparámetro estudiado. La comparación se realiza mediante los dos errores espacio-temporales globales definidos sobre la malla densa de 201×201 puntos: el error relativo L^2 y el error puntual máximo $\mathcal{E}_{\text{máx}} = \max_{x,t} |u_\theta(x, t) - u_{\text{ref}}(x, t)|$.

Topología de la red

La tabla 5.7 contrasta cuatro configuraciones de profundidad y ancho, manteniendo fijos el resto de hiperparámetros (activación tanh, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$).

Las cuatro configuraciones convergen exitosamente y, lo más llamativo, lo hacen al **mismo orden de magnitud**: el error L^2 relativo se sitúa entre 1.42×10^{-3} y 2.20×10^{-3} para todas las topologías probadas, con un margen entre la mejor (3 capas $\times$ 20 neuronas) y la peor (5 capas $\times$ 20 neuronas, casualmente la base) de apenas un factor 1.5. La interpretación es directa: el problema de la onda estacionaria con condición inicial $\sin(\pi x)$ y $c = 2$ es estructuralmente simple desde la perspectiva de capacidad expresiva —la solución verdadera tiene una sola frecuencia espacial y una

Tabla 5.7: Sensibilidad respecto a la topología de la red. Activación tanh, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$, $c = 2$.

Configuración	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Tiempo (s)
3 capas × 20 neuronas	1.42×10^{-3}	4.25×10^{-3}	181.2
5 capas × 20 neuronas (base)	2.20×10^{-3}	3.81×10^{-3}	308.6
5 capas × 50 neuronas	1.74×10^{-3}	5.84×10^{-3}	380.4
7 capas × 80 neuronas	1.70×10^{-3}	3.23×10^{-3}	421.4

sola frecuencia temporal— y una red modesta dispone ya de capacidad suficiente para representarla con fidelidad. Aumentar el número de parámetros más allá del mínimo necesario produce mejorías marginales pero no transforma cualitativamente el resultado.

Tres observaciones adicionales merecen mención. Primero, la configuración base de cinco capas con veinte neuronas es marginalmente la peor del grupo en $\mathcal{E}_{L^2}$; esto sugiere que para este problema específico, configuraciones algo distintas ofrecen ligeras ventajas. Segundo, la configuración de tres capas resulta más eficiente: alcanza un error comparable o mejor en aproximadamente la mitad del tiempo de las configuraciones más grandes. Tercero, la configuración de cinco capas con cincuenta neuronas presenta el $\mathcal{E}_{\text{máx}}$ más alto del grupo (5.84×10^{-3}, claramente por encima de los demás) aun teniendo un $\mathcal{E}_{L^2}$ competitivo, lo que indica que su error está más concentrado en regiones localizadas del dominio.

Función de activación

La tabla 5.8 compara tres funciones de activación sobre la arquitectura base de cinco capas por veinte neuronas, reportando además la pérdida final como indicador secundario.

Tabla 5.8: Sensibilidad respecto a la función de activación. Arquitectura 5×20, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$, $c = 2$.

Activación	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
Tangente hiperbólica tanh (base)	2.20×10^{-3}	3.81×10^{-3}	3.26×10^{-4}
Sigmoide σ	2.02×10^{-3}	3.54×10^{-3}	4.36×10^{-4}
Swish	1.12×10^{-3}	1.84×10^{-3}	2.00×10^{-4}

Los resultados muestran tres comportamientos diferenciados pero menos extremos que los observados para el oscilador de Van der Pol. La activación **Swish** vuelve a ser la mejor del grupo, con un error $\mathcal{E}_{L^2} = 1.12 \times 10^{-3}$ que mejora al de tanh por un factor cercano a dos. Swish, no monótona y no acotada, proporciona representaciones de mayor expresividad que las activaciones acotadas tradicionales para problemas con dinámica oscilatoria suave. La consistencia del resultado entre dos problemas estructuralmente distintos (una EDO no lineal rígida y una EDP hiperbólica lineal) refuerza la conclusión y sugiere que para PINNs sobre soluciones suaves, Swish merece consideración como activación por defecto.

El comportamiento de la sigmoide, en cambio, contrasta radicalmente con lo observado en el problema anterior. Mientras que para el oscilador de Van der Pol la sigmoide fallaba catastróficamente ($\mathcal{E}_{L^2}$ del orden uno), aquí su error es prácticamente equivalente al de tanh (2.02×10^{-3} frente a 2.20×10^{-3}, una diferencia inferior al 10 %). La explicación es la simetría estructural del problema: la solución $u(x,t) = \sin(\pi x)\cos(2\pi t)$ oscila simétricamente entre -1 y 1, sin ningún sesgo intrínseco hacia uno de los signos. Aunque el rango natural de la sigmoide es $(0,1)$, las capas finales del perceptrón pueden recentrar y reescalar sin dificultad estructural, y la asimetría intrínseca de la

activación no se traduce en un sesgo sobre la solución representada. Este contraste con Van der Pol subraya la importancia de elegir la activación en función de la estructura específica del problema y no de reglas universales.

Ponderación de los términos de datos

La tabla 5.9 contrasta tres elecciones del peso común $\lambda_{\text{data}} := \lambda_{IC,1} = \lambda_{IC,2} = \lambda_{BC}$ en la pérdida, manteniendo $\lambda_r = 1$ y el resto de hiperparámetros en sus valores base.

Tabla 5.9: Sensibilidad respecto a la ponderación de los términos de datos. Arquitectura 5×20, tanh, $K_x = K_t = 4$, $\lambda_r = 1$, $c = 2$.

λ_{data}	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
10	1.68×10^{-2}	2.55×10^{-2}	1.40×10^{-3}
100 (base)	2.20×10^{-3}	3.81×10^{-3}	3.26×10^{-4}
1000	4.81×10^{-4}	7.64×10^{-4}	6.00×10^{-4}

El resultado muestra una monotonía clara y contraintuitiva: a mayor ponderación de los términos de datos, menor error final, con $\lambda_{\text{data}} = 1000$ alcanzando $\mathcal{E}_{L^2} = 4.81 \times 10^{-4}$ —casi un orden de magnitud por debajo del caso base con $\lambda_{\text{data}} = 100$, y dos órdenes por debajo del caso degradado con $\lambda_{\text{data}} = 10$—. Este patrón contradice el hallazgo análogo del oscilador de Van der Pol, donde la disminución del peso de la condición inicial mejoraba el resultado, y exige una explicación estructural.

La razón está en la **dimensión geométrica de las restricciones de datos**. En el problema de valor inicial puro de la sección anterior, la condición inicial es un único punto del dominio temporal ($t = 0$, un valor escalar), por lo que su contribución a la pérdida promediada es muy pequeña y el peso λ_0 actúa principalmente como un ancla puntual que el residuo de la EDO satisface ya casi por sí mismo cerca de $t = 0$. En el problema de la ecuación de onda, en cambio, las restricciones de datos abarcan tres líneas del dominio bidimensional —la condición inicial sobre $t = 0$ y las dos condiciones de frontera sobre $x = 0$ y $x = L$—, cubriendo en conjunto una porción no trivial de la geometría. Restringir estas regiones con peso fuerte fija la solución en una fracción significativa del contorno del dominio, lo que propaga consistencia hacia el interior por continuidad. El residuo, por su parte, satisface la PDE relativo a las condiciones de contorno: si éstas están mal restringidas, el residuo puede satisfacerse en un conjunto de soluciones desplazadas que no corresponden al problema original.

Una observación sutil pero importante: con $\lambda_{\text{data}} = 1000$, la pérdida total final (6.00×10^{-4}) es **mayor** que con $\lambda_{\text{data}} = 100$ (3.26×10^{-4}), aun cuando el error real de la solución es menor. Esta aparente paradoja se resuelve recordando que la pérdida total $\mathcal{L} = \mathcal{L}_r + \lambda_{\text{data}}(\mathcal{L}_{IC,1} + \mathcal{L}_{IC,2} + \mathcal{L}_{BC})$ depende explícitamente del peso λ_{data}: a mayor peso, mayor magnitud asignada a las componentes de datos aun cuando éstas alcanzan valores menores en términos absolutos. Esta circunstancia ilustra un principio metodológico importante para las PINNs: **la pérdida total no es necesariamente proporcional a la calidad de la solución**, y comparaciones entre configuraciones con distinta ponderación deben basarse en métricas externas independientes del esquema de pesos —como precisamente los errores L^2 y L^∞ contra la referencia analítica que se emplea aquí—.

Régimen exigente: variación de la velocidad de propagación c

Como prueba de robustez frente a la dificultad estructural del problema, la tabla 5.10 reporta los resultados de la configuración base aplicada al sistema con $c \in \{1, 2, 4, 8\}$. Al aumentar c se incre-

menta proporcionalmente la frecuencia temporal de la solución analítica $u(x,t) = \sin(\pi x)\cos(c\pi t)$, lo que se traduce en oscilaciones más rápidas que la red debe capturar dentro del mismo intervalo $[0, T]$ con el mismo presupuesto computacional.

Tabla 5.10: Desempeño en función de la velocidad de propagación c. Arquitectura 5×20, tanh, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$.

c	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
1	9.86×10^{-4}	3.44×10^{-3}	4.97×10^{-4}
2 (base)	2.20×10^{-3}	3.81×10^{-3}	3.26×10^{-4}
4	5.08×10^{-2}	8.20×10^{-2}	3.67×10^{-2}
8	9.81×10^{-1}	1.03	1.97

El resultado exhibe una degradación monótona de la calidad con la velocidad de propagación, con tres regímenes claramente diferenciados. Para $c \in \{1, 2\}$ la PINN converge a errores del orden 10^{-3} y la pérdida final permanece pequeña, indicando convergencia robusta al mínimo correcto. Para $c = 4$ el error salta dos órdenes de magnitud, alcanzando $\mathcal{E}_{L^2} = 5.08 \times 10^{-2}$ con una pérdida final también elevada (3.67×10^{-2}), lo que indica que la red no logra minimizar el residuo de la PDE en el presupuesto computacional disponible. Para $c = 8$ la solución no converge: $\mathcal{E}_{L^2} \approx 1$ y la pérdida residual es del orden uno, evidencia de que el optimizador queda atrapado lejos del mínimo correcto.

La explicación es estructural y específica del esquema de embedding adoptado. El embedding de Fourier bidimensional definido con $K_t = 4$ contiene frecuencias temporales $\omega_\ell^t = 2\pi\ell/T$ para $\ell \in \{1, 2, 3, 4\}$, es decir, su mayor componente es $\omega_4^t = 8\pi$. La solución analítica para $c = 8$ tiene factor temporal $\cos(8\pi t)$, cuya frecuencia 8π coincide exactamente con el límite superior representable por el embedding. En esta situación, la red dispone de un único modo del embedding —justamente $\cos(\omega_4^t t) = \cos(8\pi t)$— para reproducir todo el comportamiento temporal de la solución, sin ningún margen para que las capas ocultas modulen o combinen modos. Para $c = 4$, la frecuencia requerida 4π se halla dentro del espectro representable ($\omega_2^t = 4\pi$) pero sin redundancia espectral, lo que explica la degradación intermedia.

Este modo de falla difiere del observado para el oscilador de Van der Pol en una característica importante. En la sección anterior, el fallo para $\mu \geq 2$ se atribuía a la existencia de falsos mínimos locales en el paisaje de la pérdida; aquí, en cambio, el fallo es atribuible a la **insuficiencia de la base funcional explícitamente inyectada por el embedding**. Las dos fuentes de falla motivan estrategias correctivas diferentes: para la del Van der Pol son apropiadas las técnicas adaptativas SA-PINN o XPINN; para la de la onda con c grande es apropiado **aumentar** K_t —el caso $c = 8$ requeriría idealmente $K_t \geq 8$ para disponer de redundancia espectral— o reemplazar el embedding fijo por un *Fourier feature mapping* aleatorio aprendido, técnica que distribuye las frecuencias del embedding estocásticamente sobre un rango amplio y elimina la dependencia crítica de elegir K_t a priori.

Conclusiones del estudio de sensibilidad

Los experimentos anteriores permiten extraer cuatro conclusiones para la resolución de la ecuación de onda 1D mediante PINNs:

1. La topología tiene impacto reducido en este problema. Cuatro configuraciones que varían el número de parámetros en factor de diez producen errores en el mismo orden de magnitud ($\mathcal{E}_{L^2} \in [1.4, 2.2] \times 10^{-3}$), evidencia de que la PINN se halla en un **régimen de saturación**

de capacidad: una red modesta basta. La configuración base puede modificarse libremente hacia versiones más eficientes (3 capas × 20 neuronas) sin sacrificio significativo de precisión.

2. La función de activación admite tres elecciones razonables, con Swish como mejor opción del grupo (factor 2 sobre tanh). La sigmoide, que falló catastróficamente en el oscilador de Van der Pol, se comporta aquí prácticamente igual que la tangente hiperbólica, debido a la simetría estructural de la solución alrededor de cero. El hallazgo refuerza la **idoneidad de Swish** ya identificada en la sección anterior y advierte contra las recomendaciones universales sobre activaciones en PINNs.

3. La ponderación de los términos de datos exhibe **comportamiento opuesto al observado en el problema de valor inicial puro**: a mayor λ_{data}, menor error, con la mejora más sustancial al pasar de $\lambda_{\text{data}} = 100$ a $\lambda_{\text{data}} = 1000$ ($\mathcal{E}_{L^2}$ se reduce de 2.20×10^{-3} a 4.81×10^{-4}). La explicación es la dimensión geométrica de las restricciones: en problemas con condiciones iniciales y de frontera extensas (líneas o superficies del dominio espacio-temporal), restringirlas con peso fuerte estabiliza el problema global y permite al residuo refinar la solución en el interior con consistencia.

4. La aplicabilidad del esquema con embedding de Fourier de orden fijo $K_x = K_t = 4$ está acotada por la velocidad de propagación: converge robustamente para $c \leq 2$, degrada para $c = 4$ y falla para $c = 8$. El modo de falla es **insuficiencia espectral del embedding** (a diferencia del modo de falla por mínimos locales del Van der Pol con $\mu \geq 2$), y motiva el uso de *Fourier feature mappings* aleatorios o estrategias de selección adaptativa de frecuencias como vías para extender la aplicabilidad del método al régimen exigente.

5.3. Ecuación del Calor en 1D

La sección anterior abordó la ecuación de onda como ejemplo de problema hiperbólico de segundo orden. Esta sección aborda su contraparte natural en la taxonomía de las ecuaciones diferenciales parciales lineales de segundo orden: la **ecuación del calor**, ejemplo paradigmático de problema parabólico de primer orden temporal. El contraste estructural entre ambas ecuaciones es profundo: en el orden del operador diferencial, en el número y estructura de las restricciones impuestas, en las propiedades cualitativas de la solución y, como consecuencia, en el comportamiento de la formulación PINN aplicada a su resolución.

5.3.1. Planteamiento del problema

Se considera el problema de valor inicial y de frontera para la ecuación del calor unidimensional homogénea con coeficiente constante,

$$\frac{\partial u}{\partial t}(x,t) \;=\; \alpha \frac{\partial^2 u}{\partial x^2}(x,t), \qquad (x,t) \in [0,L] \times [0,T], \tag{5.29}$$

donde $u : [0,L] \times [0,T] \to \mathbb{R}$ es la incógnita y $\alpha > 0$ es la **difusividad térmica** —el coeficiente que cuantifica la rapidez con que el calor se propaga a través del medio—. La ecuación (5.29) modela una amplia clase de fenómenos físicos caracterizados por procesos difusivos: la conducción del calor en una barra delgada, la difusión de un soluto a baja concentración en un fluido, la propagación de un campo de presión en un medio poroso o el suavizado de imágenes mediante el método de difusión isótropa. Todos estos fenómenos comparten una característica común que distingue radicalmente

la ecuación parabólica de la ecuación hiperbólica de la sección precedente: la **irreversibilidad temporal**.

El problema se cierra con condiciones iniciales y de frontera. A diferencia de la ecuación de onda, cuyo carácter de segundo orden en el tiempo requería *dos* condiciones iniciales —posición y velocidad—, la ecuación del calor es de **primer orden en el tiempo** y por tanto requiere una *única* condición inicial:

$$u(x,0) = f(x), \qquad x \in [0, L]. \tag{5.30}$$

Las condiciones de frontera, también de tipo Dirichlet homogéneas, mantienen la temperatura nula en los extremos del dominio espacial durante todo el intervalo temporal:

$$u(0,t) = 0, \qquad u(L,t) = 0, \qquad t \in [0, T]. \tag{5.31}$$

Físicamente, las condiciones (5.31) corresponden al caso de una barra unidimensional cuyos extremos se mantienen en contacto térmico con un baño a temperatura cero, lo que actúa como sumidero perfecto de calor.

Parámetros del experimento base. Se adoptan los valores

$$L = 1, \qquad T = 1, \qquad \alpha = 0.1, \tag{5.32}$$

con condición inicial

$$f(x) = \sin(\pi x). \tag{5.33}$$

La elección de $\alpha = 0.1$ es deliberada: la escala de tiempo característica del problema, $\tau = 1/(\alpha\pi^2) \approx 1.013$, coincide aproximadamente con la duración del intervalo temporal $T = 1$, por lo que durante el experimento se observa exactamente un tiempo característico de decaimiento. La amplitud máxima de la solución, partiendo de 1 en $t = 0$, alcanza aproximadamente $e^{-\alpha\pi^2 T} \approx 0.37$ en $t = T = 1$. Esta elección equilibra dos consideraciones opuestas: valores de α menores producirían una solución casi estacionaria de poco interés dinámico, mientras que valores mayores conducirían a un decaimiento tan rápido que la solución colapsaría a niveles numéricamente próximos a cero antes del final del intervalo.

Solución analítica de referencia. El problema admite solución cerrada que se obtiene por separación de variables. Postulando $u(x,t) = \phi(x)\,\psi(t)$ y sustituyendo en (5.29), se obtiene la pareja de ecuaciones diferenciales ordinarias $\phi''(x) + \lambda\phi(x) = 0$ y $\psi'(t) + \alpha\lambda\psi(t) = 0$, con λ constante de separación. Las condiciones de frontera fuerzan $\lambda = (n\pi/L)^2$ con n entero positivo, los modos espaciales son $\phi_n(x) = \sin(n\pi x/L)$, y los modos temporales son **exponenciales decrecientes** $\psi_n(t) = e^{-\alpha(n\pi/L)^2 t}$. La superposición general,

$$u(x,t) \;=\; \sum_{n=1}^{\infty} B_n \,\sin\!\Big(\frac{n\pi}{L}x\Big)\,\exp\!\Big(-\alpha\big(\frac{n\pi}{L}\big)^2 t\Big), \tag{5.34}$$

con coeficientes B_n determinados por la expansión en serie de Fourier de la condición inicial. Para $f(x) = \sin(\pi x)$ en $[0,1]$, la expansión se reduce a un único término con $B_1 = 1$, lo que produce la solución exacta del problema:

$$u(x,t) \;=\; \sin(\pi x)\, e^{-\alpha\pi^2 t}. \tag{5.35}$$

La solución es el producto de un perfil espacial fijo por un factor temporal de decaimiento exponencial con constante de tiempo $\tau = 1/(\alpha\pi^2)$.

Propiedades estructurales del problema parabólico. La ecuación del calor presenta tres propiedades cualitativas que la distinguen netamente de la ecuación de onda y que son relevantes para anticipar el comportamiento del método PINN:

1. *Efecto suavizante.* El operador parabólico actúa como un low-pass filter en cada paso temporal: las componentes de alta frecuencia espacial decaen exponencialmente más rápido que las de baja frecuencia. Para la solución, el factor $e^{-\alpha\pi^2 t}$ correspondería a $e^{-9\alpha\pi^2 t}$ si la condición inicial hubiera contenido el tercer modo $\sin(3\pi x)$, decaimiento nueve veces más rápido. Esta propiedad implica que la solución es **infinitamente regular** para $t > 0$, aun si f es sólo continua.

2. *Principio del máximo.* La solución no puede exceder los valores extremos de la condición inicial y de las condiciones de frontera: $\min_x f(x) \le u(x,t) \le \max_x f(x)$ para todo (x,t) en el dominio. Para el problema con $f(x) = \sin(\pi x) \ge 0$ y condiciones de frontera nulas, la solución permanece no negativa en todo el dominio: $u(x,t) \in [0,1]$. Esta propiedad puede emplearse como criterio de validación física de la salida de la PINN.

3. *Decaimiento de la energía.* La cantidad $E(t) = \int_0^L u^2(x,t)\,dx$ es monótonamente decreciente en el tiempo. Para nuestra solución, $E(t) = (L/2)\,e^{-2\alpha\pi^2 t}$.

Estas tres propiedades anticipan que la formulación PINN, aplicada a la ecuación del calor con la receta desarrollada para la ecuación de onda, debería arrojar resultados **al menos tan precisos** como los obtenidos para la onda y posiblemente significativamente más precisos: la solución es más suave, la geometría espacio-temporal es más sencilla y el espacio de soluciones a explorar es menor.

5.3.2. Formulación PINN para la ecuación del calor

La aplicación del marco PINN al problema sigue los pasos generales ya desarrollados en las dos secciones precedentes, con **dos simplificaciones estructurales** relevantes respecto al caso de la ecuación de onda. Primero, el operador diferencial es de primer orden en el tiempo, lo que reduce la profundidad de la diferenciación automática anidada de cuatro derivadas parciales a sólo tres. Segundo, el problema requiere una única condición inicial sobre la posición, lo que reduce el número de componentes de la pérdida de cuatro a tres.

La red neuronal aproximadora se denota

$$u_\theta : [0,L] \times [0,T] \to \mathbb{R}, \qquad (x,t) \mapsto u_\theta(x,t), \tag{5.36}$$

con la misma estructura empleada en la sección anterior: dos neuronas de entrada que reciben (x,t), una neurona de salida que produce el valor de u, y cinco capas ocultas con veinte neuronas por capa como caso base.

Residuo de la ecuación. El residuo asociado a la ecuación del calor se define como

$$r_\theta(x,t) \;:=\; \frac{\partial u_\theta}{\partial t}(x,t) \;-\; \alpha\,\frac{\partial^2 u_\theta}{\partial x^2}(x,t). \tag{5.37}$$

Su evaluación numérica requiere tres derivadas parciales obtenidas mediante diferenciación automática: una primera derivada temporal u_t y dos derivadas espaciales u_x y u_{xx}. La derivada temporal se obtiene en una sola llamada a `torch.autograd.grad` sobre la salida del modelo; las dos derivadas espaciales se obtienen mediante dos llamadas consecutivas, la primera para u_x y la segunda para u_{xx}. Todas las invocaciones se realizan con `create_graph=True` para permitir la retropropagación posterior de los gradientes de la pérdida respecto a los parámetros θ.

Estructura triple de la pérdida. La pérdida se descompone en tres contribuciones independientes —el residuo de la PDE, la condición inicial y las condiciones de frontera—. Formalmente,

$$\mathcal{L}(\theta) \;=\; \lambda_r\,\mathcal{L}_r(\theta) \;+\; \lambda_{IC}\,\mathcal{L}_{IC}(\theta) \;+\; \lambda_{BC}\,\mathcal{L}_{BC}(\theta), \tag{5.38}$$

donde

$$\mathcal{L}_r(\theta) = \frac{1}{N_r}\sum_{i=1}^{N_r}\left|r_\theta(x_r^{(i)}, t_r^{(i)})\right|^2, \tag{5.39}$$

$$\mathcal{L}_{IC}(\theta) = \frac{1}{N_0}\sum_{i=1}^{N_0}\left|u_\theta(x_0^{(i)}, 0) - f(x_0^{(i)})\right|^2, \tag{5.40}$$

$$\mathcal{L}_{BC}(\theta) = \frac{1}{2N_b}\sum_{j=1}^{N_b}\Big(\left|u_\theta(0, t_b^{(j)})\right|^2 + \left|u_\theta(L, t_b^{(j)})\right|^2\Big). \tag{5.41}$$

Los conjuntos de puntos $\{(x_r, t_r)\}$, $\{x_0\}$ y $\{t_b\}$ se distribuyen análogamente al caso de la onda: N_r puntos en el interior del dominio espacio-temporal, N_0 puntos sobre la línea $t = 0$ que sostiene la condición inicial, y N_b puntos sobre cada borde espacial $x \in \{0, L\}$.

Ponderación de los términos. Se adoptan los pesos

$$\lambda_r = 1, \qquad \lambda_{IC} = \lambda_{BC} = 100. \tag{5.42}$$

Fijamos las restricciones lineales con peso mayor para estabilizar la solución global y dejar al residuo refinar la dinámica interior.

Observación sobre el modo de falla potencial. La solución analítica de la ecuación del calor con condiciones de frontera homogéneas y sin fuentes **decae a cero**: $\lim_{t\to\infty} u(x,t) = 0$ uniformemente. Esto introduce un modo de falla específico de los problemas parabólicos amortiguados: la red neuronal podría converger a la solución idénticamente nula $u_\theta(x,t) \equiv 0$, que satisface trivialmente tanto el residuo $r_\theta = 0$ como las condiciones de frontera $u_\theta|_{x=0,L} = 0$, comprometiendo únicamente la condición inicial. La existencia de este mínimo local ilegítimo del residuo motiva el uso de un peso significativo $\lambda_{IC} = 100$ sobre la condición inicial, lo que penaliza la trayectoria hacia esta solución trivial con suficiente fuerza para evitar su atracción durante el entrenamiento.

Selección de la función de activación. El operador parabólico (5.29) involucra derivadas hasta segundo orden, requisito que se satisface con holgura por la tangente hiperbólica, así como por la sigmoide y por Swish. Se adopta tanh como activación base, en línea con la práctica establecida en las secciones anteriores.

Embedding de Fourier y muestreo. Se mantiene el embedding de Fourier bidimensional definido anteriormente con $K_x = K_t = 4$, pese a que la solución del problema parabólico no es periódica en t sino exponencialmente decreciente. La conservación del esquema admite una doble justificación: por un lado, las bases trigonométricas son **conjuntos densos en el espacio de funciones continuas** sobre un intervalo acotado, por lo que la red puede recombinar los modos del embedding para reproducir el decaimiento exponencial sin necesidad de bases especializadas; por otro lado, el uso del mismo embedding en problemas de naturaleza distinta permite contrastar empíricamente la robustez del método más allá de su aplicabilidad estricta. El muestreo emplea $N_r = 2{,}000$ puntos interiores, $N_0 = 200$ puntos para la condición inicial y $N_b = 200$ puntos para cada borde espacial.

Algoritmo de optimización. Se mantiene la estrategia híbrida Adam → L-BFGS con 5000 iteraciones de Adam ($\eta = 10^{-3}$) seguidas de hasta 2500 iteraciones de L-BFGS con búsqueda de línea de Wolfe.

5.3.3. Implementación en PyTorch

La implementación reutiliza la estructura modular desarrollada para la ecuación de onda con los ajustes propios de la formulación: residuo con una sola derivada temporal, pérdida con tres componentes y sin la segunda condición inicial. La definición del modelo neuronal con embedding de Fourier es esencialmente idéntica a la presentada en la sección 5.2.3 y se omite por brevedad; el cambio sustantivo se concentra en las funciones de pérdida y entrenamiento.

```python
import torch
import torch.nn as nn
import numpy as np


def f_ic(x):  # condicion inicial: u(x, 0) = sin(pi x)
    return torch.sin(np.pi * x)


def pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, alpha,
              lambda_r=1.0, lambda_ic=100.0, lambda_bc=100.0):
    # ----- Residuo PDE: u_t - alpha u_xx -----
    x = x_int.clone().requires_grad_(True)
    t = t_int.clone().requires_grad_(True)
    u = model(x, t)
    u_t  = torch.autograd.grad(u,   t, torch.ones_like(u),   create_graph=True)[0]
    u_x  = torch.autograd.grad(u,   x, torch.ones_like(u),   create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)[0]
    r = u_t - alpha * u_xx
    L_r = torch.mean(r**2)

    # ----- CI: u(x, 0) = f(x) -----
    t_zero = torch.zeros_like(x_ic)
    u_ic = model(x_ic, t_zero)
    L_ic = torch.mean((u_ic - f_ic(x_ic))**2)

    # ----- CF: u(0, t) = u(L, t) = 0 -----
    u_bc = model(x_bc, t_bc)
    L_bc = torch.mean(u_bc**2)

    L_total = lambda_r * L_r + lambda_ic * L_ic + lambda_bc * L_bc
    return L_total, L_r, L_ic, L_bc
```

Listing 5.8: Función de pérdida para la ecuación del calor.

```python
def train(model, alpha, L=1.0, T=1.0, n_adam=5000, n_lbfgs=2500,
          n_interior=2000, n_ic=200, n_bc=200,
          lr_adam=1e-3, lambda_data=100.0, seed=42):
    x_int, t_int, x_ic, x_bc, t_bc = sample_points(
        n_interior=n_interior, n_ic=n_ic, n_bc=n_bc,
        L=L, T=T, seed=seed,
    )

    # Fase 1: Adam
    opt = torch.optim.Adam(model.parameters(), lr=lr_adam)
    for i in range(n_adam):
        opt.zero_grad()
        L_total, *_ = pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, alpha,
                                lambda_ic=lambda_data, lambda_bc=lambda_data)
        L_total.backward()
        opt.step()
```

```
    # Fase 2: L-BFGS
    opt = torch.optim.LBFGS(
        model.parameters(), lr=1.0, max_iter=n_lbfgs,
        tolerance_grad=1e-11, tolerance_change=1e-13,
        history_size=100, line_search_fn='strong_wolfe',
    )
    def closure():
        opt.zero_grad()
        L_total, *_ = pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, alpha,
                                lambda_ic=lambda_data, lambda_bc=lambda_data)
        L_total.backward()
        return L_total
    opt.step(closure)
    L_total, *_ = pinn_loss(model, x_int, t_int, x_ic, x_bc, t_bc, alpha,
                            lambda_ic=lambda_data, lambda_bc=lambda_data)
    return L_total.item()
```

Listing 5.9: Procedimiento de entrenamiento adaptado a la pérdida con tres componentes.

El código se ha probado en PyTorch 2.12.0 sobre el *backend* MPS de Apple Silicon. El experimento base se completó en aproximadamente tres minutos en un chip M4, un tiempo notablemente inferior a los cinco minutos requeridos por la ecuación de onda del experimento equivalente. La diferencia se explica por la simplificación del residuo y por el menor número de componentes de la pérdida.

5.3.4. Resultados y comparación con la solución de referencia

La ejecución del procedimiento sobre la configuración base produce una aproximación numérica $u_\theta(x,t)$ que se compara con la solución exacta evaluada sobre una malla densa de 201×201 puntos en $[0,L]\times[0,T]$. Los indicadores cuantitativos son: **error relativo** L^2 de $\mathcal{E}_{L^2} = 3.99 \times 10^{-5}$, **error puntual máximo** de $\mathcal{E}_{\text{máx}} = 1.09 \times 10^{-4}$ y **pérdida residual final** de $\mathcal{L}(\theta^\star) = 9.95 \times 10^{-7}$. El tiempo total de entrenamiento fue de 170.1 segundos en un chip Apple M4 con *backend* MPS.

Estos números requieren una lectura cuidadosa en comparación con el caso de la ecuación de onda. Mientras que aquel problema alcanzó $\mathcal{E}_{L^2} = 2.66 \times 10^{-3}$ con la misma arquitectura y presupuesto computacional, el problema parabólico de esta sección alcanza un error **aproximadamente dos órdenes de magnitud menor**. Esta mejora dramática no se explica por diferencias en la formulación PINN —que es esencialmente la misma— sino por la naturaleza estructural del problema: la solución del calor es producto-separable, infinitamente regular y de baja oscilación temporal, propiedades que coinciden con el sesgo inductivo natural de las redes con activaciones suaves.

Las componentes individuales de la pérdida final exhiben además una asimetría aun más pronunciada que la observada en la sección anterior:

$$\mathcal{L}_r = 8.92 \times 10^{-7}, \quad \mathcal{L}_{IC} = 3.89 \times 10^{-10}, \quad \mathcal{L}_{BC} = 8.10 \times 10^{-10}.$$

Las componentes de datos son tres órdenes de magnitud menores que el residuo, frente a la diferencia de tres a cuatro órdenes observada en el caso de la onda.

Mapas de la solución y el error. La figura 5.7 muestra los tres paneles habituales: la solución de la red, la referencia analítica y el mapa del error puntual. Los dos primeros paneles son visualmente indistinguibles entre sí, ambos mostrando el patrón canónico de un único modo difusivo: un máximo concentrado en $x = 1/2$, $t = 0$ que decae exponencialmente hacia los bordes en x y hacia tiempos posteriores. El tercer panel revela un mapa de error puntual con picos no superiores a $1.09 \times$

10^{-4}, distribuidos en un patrón **difuso y sin estructura sistemática**. La ausencia de bandas estructuradas en el error es una manifestación del efecto suavizante del operador parabólico, que tiende a homogeneizar las imperfecciones locales del optimizador.

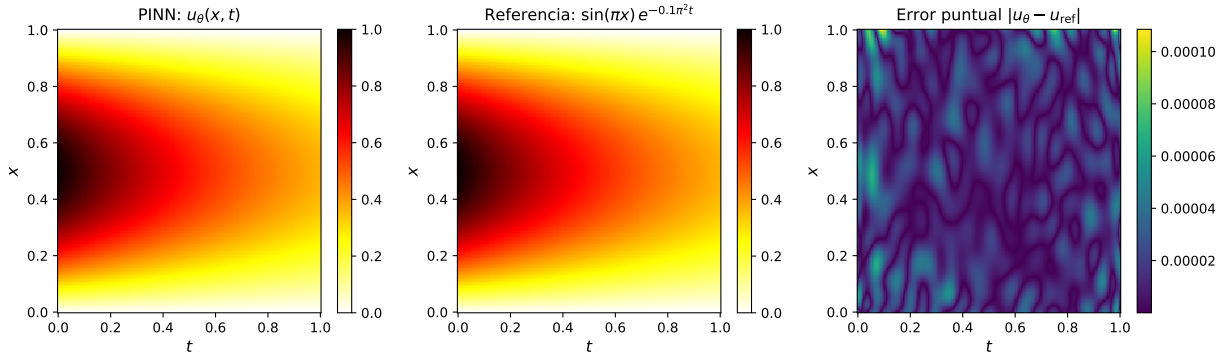


Figura 5.7: Comparación entre la solución de la PINN (izquierda) y la solución analítica $u(x,t) = \sin(\pi x)\, e^{-\alpha\pi^2 t}$ con $\alpha = 0.1$ (centro). El panel derecho muestra el error puntual $|u_\theta - u_{\text{ref}}|$, cuyos picos no superan 1.09×10^{-4}.

Cortes temporales. La figura 5.8 muestra el perfil $u(x, t_k)$ para cuatro instantes característicos $t_k \in \{0, 0.25, 0.5, 1.0\}$, revelando explícitamente la dinámica de decaimiento de la amplitud. En $t = 0$, el perfil es exactamente la condición inicial $\sin(\pi x)$ con amplitud máxima 1. En $t = 0.25$, la amplitud se ha reducido a aproximadamente 0.78; en $t = 0.5$, a aproximadamente 0.61; en $t = 1$, a aproximadamente 0.37. Estos valores coinciden con $e^{-\alpha\pi^2 t_k}$ y son reproducidos por la PINN con precisión visualmente perfecta. La conservación del perfil sinusoidal sin distorsión espacial a lo largo del tiempo es la firma característica de la separabilidad de la solución.

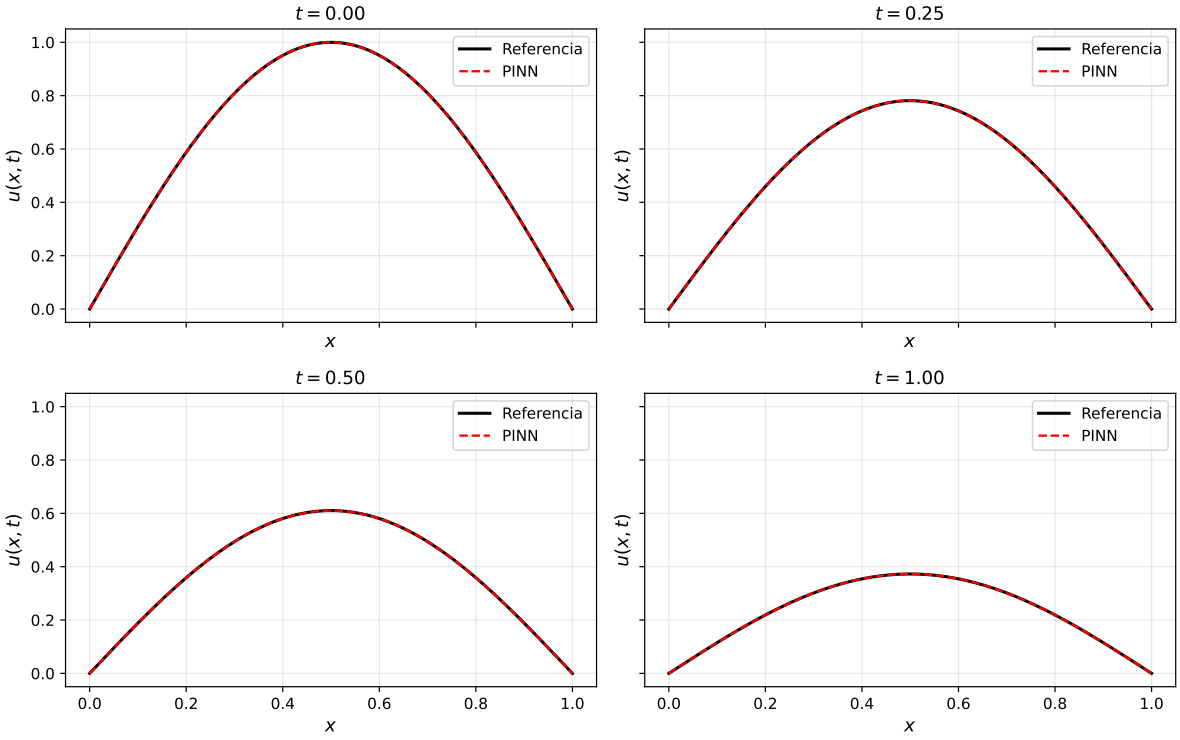


Figura 5.8: Cortes temporales $u(x, t_k)$ para $t_k \in \{0,\ 0.25,\ 0.5,\ 1.0\}$. La curva continua negra es la referencia analítica; la discontinua roja es la PINN. La amplitud decae exponencialmente conservando el perfil sinusoidal $\sin(\pi x)$.

Verificación del decaimiento exponencial. La figura 5.9 compara la amplitud máxima de la solución de la PINN, $\max_x |u_\theta(x, t)|$, con la predicción analítica $e^{-\alpha\pi^2 t}$, en dos escalas. En el panel de la izquierda (escala lineal), las dos curvas se superponen visualmente sin distinción. En el panel de la derecha (escala semilogarítmica), la curva analítica se transforma en una línea recta de pendiente $-\alpha\pi^2 \approx -0.987$, y la curva de la PINN coincide con ella en toda la extensión del intervalo. Este resultado es la confirmación visual más directa de que la PINN no se limita a aproximar valores puntuales de la solución, sino que captura correctamente la **ley de decaimiento exponencial**.

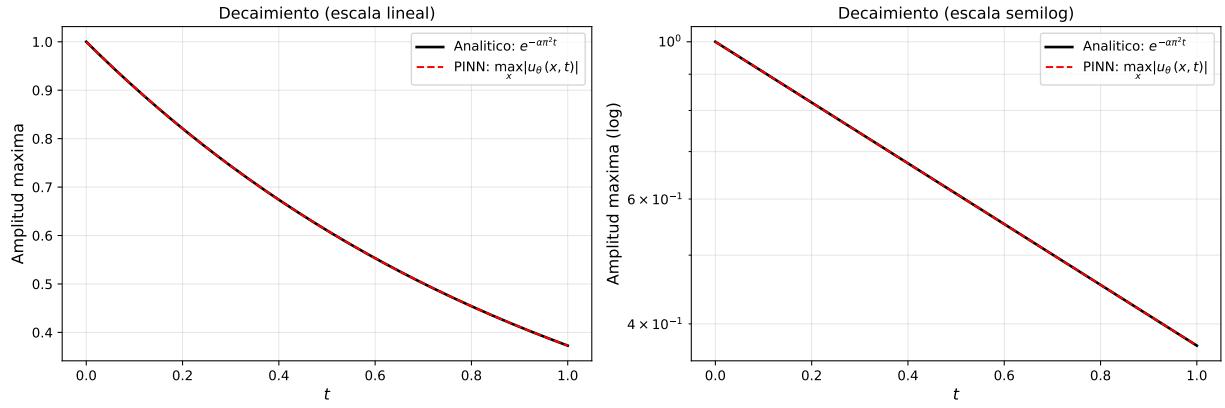


Figura 5.9: Decaimiento de la amplitud máxima $\max_x |u(x,t)|$ en función del tiempo. Izquierda: escala lineal. Derecha: escala semilogarítmica, donde el decaimiento exponencial se manifiesta como una línea recta de pendiente $-\alpha\pi^2$. La PINN reproduce la ley analítica con precisión visual completa.

Síntesis cuantitativa. La tabla 5.11 consolida los indicadores numéricos del experimento base.

Tabla 5.11: Indicadores de calidad del experimento base para la ecuación del calor con $\alpha = 0.1$, $L = T = 1$ y condición inicial $u(x,0) = \sin(\pi x)$.

Indicador	Valor
Pérdida total final $\mathcal{L}(\theta^\star)$	9.95×10^{-7}
Componente $\mathcal{L}_r$ (residuo PDE)	8.92×10^{-7}
Componente $\mathcal{L}_{IC}$ (condición inicial)	3.89×10^{-10}
Componente $\mathcal{L}_{BC}$ (frontera Dirichlet)	8.10×10^{-10}
Error relativo L^2 $\mathcal{E}_{L^2}$	3.99×10^{-5}
Error puntual máximo $\mathcal{E}_{\text{máx}}$	1.09×10^{-4}
Tiempo de la fase Adam	111.9 s
Tiempo de la fase L-BFGS	58.2 s
Tiempo total	170.1 s
Hardware	Apple M4 (MPS)

El experimento confirma que el operador parabólico es **el escenario más favorable de los problemas espaciotemporales abordados hasta el momento**. Con la misma arquitectura, el mismo embedding, los mismos pesos de pérdida y el mismo presupuesto computacional empleados para la ecuación de onda, la PINN alcanza una precisión dos órdenes de magnitud superior. Los resultados son relevantes para la metodología general: las PINNs son particularmente efectivas para problemas cuyas soluciones poseen alta regularidad y baja oscilación, propiedades que coinciden estructuralmente con las soluciones de ecuaciones parabólicas amortiguadas. La siguiente subsección examina cuán robusto es este resultado al variar los hiperparámetros principales, con énfasis particular en el comportamiento del método frente a difusividades α extremas que podrían inducir el modo de falla por solución trivial discutido anteriormente.

5.3.5. Estudios de sensibilidad

Análogamente a los estudios realizados en las dos secciones precedentes, se examina la sensibilidad de la formulación PINN para la ecuación del calor al variar cuatro ejes de su configuración: la

topología de la red, la función de activación, la ponderación de los términos de datos en la pérdida y la difusividad térmica α del problema. Cada experimento de sensibilidad se entrena con el mismo presupuesto computacional que el caso base (5000 iteraciones de Adam y 2500 de L-BFGS) y con la misma semilla aleatoria. La comparación uniforme se realiza mediante el error relativo L^2 y el error puntual máximo.

El análisis de los resultados se realiza en contraste explícito con los hallazgos para la ecuación de onda, lo que permite identificar tanto las propiedades del método PINN que se mantienen estables a través de problemas de distinta naturaleza como los aspectos cuyo comportamiento depende críticamente de la estructura específica de la ecuación.

Topología de la red

La tabla 5.12 contrasta cuatro configuraciones de profundidad y ancho, manteniendo fijos el resto de hiperparámetros.

Tabla 5.12: Sensibilidad respecto a la topología de la red. Activación tanh, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$, $\alpha = 0.1$.

Configuración	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
3 capas × 20 neuronas	5.58×10^{-5}	2.66×10^{-4}	5.00×10^{-7}
5 capas × 20 neuronas (base)	4.11×10^{-5}	1.29×10^{-4}	1.01×10^{-6}
5 capas × 50 neuronas	5.53×10^{-5}	4.18×10^{-4}	7.52×10^{-7}
7 capas × 80 neuronas	1.44×10^{-4}	5.33×10^{-4}	1.12×10^{-6}

Las cuatro configuraciones convergen al mismo orden de magnitud del error ($\mathcal{E}_{L^2} \sim 10^{-5}$ a 10^{-4}), pero el patrón revela lo siguiente: **la configuración base es la mejor** y la configuración más grande —siete capas con ochenta neuronas— es **aproximadamente** 3.5 **veces peor**. Este margen es más pronunciado que el observado para la ecuación de onda en la tabla 5.7 (donde la diferencia entre la mejor y la peor era apenas un factor 1.5), y constituye una manifestación particularmente clara del fenómeno de **sobreajuste sobre los puntos de colocación**.

La razón estructural es la siguiente: la solución de la ecuación de calor es producto-separable con *un único modo activo* —el primer modo de Fourier en x multiplicado por una exponencial en t—, lo que significa que la capacidad expresiva mínima necesaria para representarla es modesta. Una vez alcanzado el umbral mínimo de expresividad, la capacidad adicional no aporta a la aproximación de la solución verdadera, pero sí proporciona grados de libertad adicionales que la red emplea para ajustar el residuo sobre el conjunto finito de puntos de colocación, comprometiendo la generalización al continuo. La configuración de siete capas con ochenta neuronas exhibe el peor $\mathcal{E}_{\text{máx}}$ del grupo, evidencia clara del sobreajuste: el error está concentrado en regiones específicas del dominio que la red no resuelve bien pese a tener pérdida residual baja.

La conclusión metodológica es importante: para problemas con soluciones de baja complejidad espectral, **la sobreparametrización es contraproducente**. La configuración base de cinco capas con veinte neuronas representa la mejor opción para la familia de problemas considerada.

Función de activación

La tabla 5.13 compara las tres funciones de activación sobre la arquitectura base de cinco capas por veinte neuronas.

Las tres activaciones convergen al mismo orden de magnitud, con diferencias relativas modestas. Swish es marginalmente la mejor, seguida por tanh y luego por la sigmoide. La diferencia más

Tabla 5.13: Sensibilidad respecto a la función de activación. Arquitectura 5×20, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$, $\alpha = 0.1$.

Activación	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
Tangente hiperbólica tanh (base)	4.11×10^{-5}	1.29×10^{-4}	1.01×10^{-6}
Sigmoide σ	5.61×10^{-5}	3.49×10^{-4}	1.04×10^{-6}
Swish	3.41×10^{-5}	1.01×10^{-4}	6.60×10^{-7}

significativa entre las tres se observa en el error puntual máximo: Swish y tanh producen $\mathcal{E}_{\text{máx}}$ del orden 10^{-4}, mientras que la sigmoide produce 3.49×10^{-4} —aproximadamente 2.7 veces el valor de las otras dos—. Esta disparidad indica que la sigmoide, aun cuando logra precisión global comparable, presenta picos locales de error más pronunciados.

La superioridad de Swish constituye la tercera confirmación del mismo hallazgo a través de problemas de naturaleza muy distinta. La consistencia del resultado eleva la observación de un hallazgo casual a una recomendación metodológica robusta: **para PINNs sobre problemas con soluciones suaves, Swish es preferible a** tanh, aunque la diferencia se vuelve menos espectacular cuando el problema es estructuralmente sencillo.

El comportamiento de la sigmoide en este problema merece comentario aparte. La solución $u(x,t) = \sin(\pi x)\, e^{-\alpha\pi^2 t}$ es no negativa en todo el dominio, lo que podría haber favorecido a la sigmoide cuyo rango natural es $(0, 1)$. El resultado muestra que la sigmoide es competitiva, pero no superior. La hipótesis del beneficio estructural por compatibilidad de rangos queda **no confirmada** por el experimento, indicando que la geometría intrínseca de cada activación es más relevante que la compatibilidad superficial de su rango con el del problema.

Ponderación de los términos de datos

La tabla 5.14 contrasta tres elecciones del peso común $\lambda_{\text{data}} := \lambda_{IC} = \lambda_{BC}$, manteniendo $\lambda_r = 1$ y el resto de hiperparámetros en sus valores base.

Tabla 5.14: Sensibilidad respecto a la ponderación de los términos de datos. Arquitectura 5×20, tanh, $K_x = K_t = 4$, $\lambda_r = 1$, $\alpha = 0.1$.

λ_{data}	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
10	4.23×10^{-5}	4.20×10^{-4}	3.35×10^{-7}
100 (base)	4.11×10^{-5}	1.29×10^{-4}	1.01×10^{-6}
1000	2.04×10^{-4}	3.81×10^{-4}	8.76×10^{-6}

Este estudio revela lo siguiente: la ponderación $\lambda_{\text{data}} = 1000$ **degrada** el error en aproximadamente un factor 5 respecto al caso base, mientras que $\lambda_{\text{data}} = 10$ produce un error prácticamente equivalente al de la base. Este comportamiento es **exactamente opuesto** al observado para la ecuación de onda en la tabla 5.9, donde $\lambda_{\text{data}} = 1000$ era la mejor opción del grupo y reducía el error en un orden de magnitud sobre la base.

En la ecuación de onda, las condiciones de frontera $u(0,t) = u(L,t) = 0$ son **restrictivas en sentido no trivial**: la solución oscila simétricamente entre -1 y $+1$ en el interior del dominio, y son las condiciones de frontera las que fuerzan los nodos espaciales fijos en los bordes. Fijar estas condiciones con peso fuerte aporta información independiente sobre la geometría de la solución, y el resultado es una mejora del error global.

En la ecuación de calor, en cambio, las mismas condiciones de frontera $u(0,t) = u(L,t) = 0$ son **automáticamente compatibles con la dinámica espontánea** del problema: la solución decae

monótonamente hacia cero en toda la barra, y los valores nulos en los bordes son consistentes con esa tendencia natural. Las condiciones de frontera no aportan información estructuralmente distinta a la que el residuo de la PDE ya induce por sí mismo. Sobre-pesarlas con $\lambda_{\text{data}} = 1000$ es por tanto redundante: el optimizador dedica esfuerzo a reforzar una condición ya bien satisfecha a expensas del residuo en el interior del dominio, donde reside la verdadera información sobre la solución.

Este contraste entre los problemas hiperbólico y parabólico constituye una **lección metodológica valiosa**: la ponderación óptima de los términos de una pérdida PINN *no es una característica universal del método* sino que depende críticamente de la naturaleza específica del problema y, en particular, de si las restricciones impuestas aportan información estructural genuina o son meramente compatibles con la dinámica espontánea.

Régimen de difusividad: variación de α

La tabla 5.15 reporta los resultados de la configuración base aplicada al problema con $\alpha \in \{0.05, 0.1, 0.5, 1.0\}$. Aumentar α acelera el decaimiento exponencial: para $\alpha = 1.0$, la amplitud final analítica $e^{-\alpha\pi^2 T}$ se reduce a aproximadamente 5.2×10^{-5}, próxima al límite de resolución numérica.

Tabla 5.15: Desempeño en función de la difusividad térmica α. Arquitectura 5×20, tanh, $K_x = K_t = 4$, $\lambda_{\text{data}} = 100$.

α	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
0.05	7.53×10^{-5}	2.16×10^{-4}	1.21×10^{-6}
0.1 (base)	4.11×10^{-5}	1.29×10^{-4}	1.01×10^{-6}
0.5	1.51×10^{-4}	1.60×10^{-4}	2.44×10^{-6}
1.0	4.61×10^{-4}	4.96×10^{-4}	6.60×10^{-6}

El patrón es radicalmente distinto al observado en los problemas anteriores. Mientras que el oscilador de Van der Pol exhibía un *cliff* de falla catastrófica al pasar de $\mu = 1$ a $\mu = 2$, y la ecuación de onda mostraba un patrón similar al pasar de $c = 2$ a $c = 8$, la ecuación de calor presenta **degradación monótona y suave**: el error crece de 4×10^{-5} a 5×10^{-4} al pasar de $\alpha = 0.1$ a $\alpha = 1.0$, un factor que se mantiene dentro de un mismo orden de magnitud. **No hay falla catastrófica**.

Lo más relevante es la verificación implícita de una predicción cualitativa. En la subsección 5.3.2 se anticipó la posibilidad de un **modo de falla por colapso a la solución trivial**: para α grande, la solución verdadera es casi nula en gran parte del dominio, y la red podría converger a $u_\theta \equiv 0$ —que satisface el residuo y las condiciones de frontera, fallando sólo en la condición inicial—. El experimento con $\alpha = 1.0$ contradice esa preocupación: la PINN no colapsa al cero trivial, sino que reproduce la solución verdadera —incluyendo el decaimiento extremadamente rápido— con error L^∞ de apenas 5×10^{-4}. La ponderación $\lambda_{IC} = 100$ resulta ser suficiente para anclar la condición inicial $\sin(\pi x)$ con la fuerza necesaria para guiar al optimizador hacia el mínimo correcto, aun en el régimen de decaimiento más extremo del estudio.

Un detalle metodológico merece atención. El crecimiento del error relativo $\mathcal{E}_{L^2}$ al aumentar α se debe en parte a la **disminución de la norma de referencia**: para α mayor, $\|u_{\text{ref}}\|_2$ es menor (la solución completa tiene menor magnitud), y el mismo error absoluto se traduce en un error relativo más grande. Por ejemplo, para $\alpha = 1.0$ el error puntual máximo es 5.0×10^{-4} y la solución analítica tiene magnitud máxima 1 (en $t = 0$) que decae a $\sim 10^{-5}$ (en $t = 1$); el error relativo $\mathcal{E}_{L^2} = 4.6 \times 10^{-4}$ refleja en buena medida el efecto del denominador encogido. La métrica de error absoluto $\mathcal{E}_{\text{máx}}$ crece sólo en factor ~ 4 entre la base y $\alpha = 1.0$, mientras que $\mathcal{E}_{L^2}$ crece en factor 11.

Una observación adicional resulta de interés: la configuración con $\alpha = 0.05$ —el problema físicamente *más fácil* del grupo, con decaimiento más lento— exhibe error mayor que la base (7.5×10^{-5} frente a 4.1×10^{-5}). La explicación más plausible es que el embedding de Fourier en t con frecuencias $\omega_\ell^t = 2\pi\ell/T$ está estructuralmente mejor adaptado a la escala temporal de variación correspondiente a $\alpha = 0.1$ que a la del régimen de decaimiento muy lento, ilustrando que la elección de los parámetros del embedding produce un emparejamiento implícito con la escala temporal del problema.

Conclusiones del estudio de sensibilidad

Los experimentos anteriores permiten extraer cuatro conclusiones para la resolución de la ecuación de calor 1D mediante PINNs, varias de ellas en contraste explícito con los hallazgos análogos de la sección anterior:

1. La topología produce **sobreajuste pronunciado sobre los puntos de colocación** en configuraciones grandes, de manera más nítida que para la ecuación de onda. La explicación es la baja complejidad espectral de la solución del problema parabólico, que satura rápidamente la demanda de capacidad expresiva. **La configuración modesta de la base es óptima**; agregar parámetros es contraproducente.

2. La función de activación tiene impacto modesto, con las tres opciones convergiendo al mismo orden de magnitud. La preeminencia marginal de **Swish sobre** tanh **se confirma por tercera vez consecutiva**, lo que consolida la recomendación metodológica de considerar Swish como alternativa por defecto a tanh en problemas con soluciones suaves. La hipótesis estructural de que la sigmoide se beneficiaría del rango natural $(0, 1)$ no se confirma empíricamente: la sigmoide es competitiva pero no superior.

3. La ponderación de los términos de datos exhibe el **patrón opuesto** al observado para la ecuación de onda: $\lambda_{\text{data}} = 1000$ *degrada* el error en factor 5 frente a $\lambda_{\text{data}} = 100$, mientras que para la onda $\lambda_{\text{data}} = 1000$ era óptimo. La razón estructural es la compatibilidad espontánea entre las condiciones de frontera homogéneas y la tendencia natural del sistema parabólico a decaer a cero: fijar las condiciones de frontera con peso fuerte resulta redundante. Este contraste nos indica que la ponderación óptima depende críticamente de si las restricciones aportan información estructural genuina o son meramente compatibles con la dinámica espontánea.

4. La aplicabilidad de la formulación PINN básica frente al régimen de difusividad extrema es **notablemente más robusta** que frente a los regímenes exigentes de los problemas anteriores. La degradación con α creciente es monótona y suave, sin falla catastrófica, y el modo de falla por colapso a la solución trivial —potencialmente esperable para α grande— no se materializa: el peso $\lambda_{IC} = 100$ es suficiente para evitar la atracción del falso mínimo en $u_\theta \equiv 0$. Esta robustez es consistente con la mayor estabilidad estructural de los operadores parabólicos amortiguados, cuya naturaleza misma elimina muchos de los patrones patológicos de las ecuaciones hiperbólicas y rígidas.

5.4. Ecuación de Laplace (Poisson) en 1D

Las tres secciones anteriores han abordado problemas **espacio-temporales**. La presente sección rompe ese molde unificado al abordar un problema **estacionario puro**: la ecuación de Poisson en una dimensión espacial. La ausencia de variable temporal cambia cada componente del análisis — el número de variables independientes, la naturaleza de las condiciones que cierran el problema, la

estructura de la pérdida y el tiempo de cómputo— y completa la taxonomía clásica de las ecuaciones diferenciales parciales lineales de segundo ordena: hiperbólica, parabólica y, ahora, **elíptica**.

5.4.1. Planteamiento del problema

Se considera la ecuación de Poisson unidimensional con coeficiente unitario:

$$-\frac{d^2u}{dx^2}(x) \;=\; f(x), \qquad x \in [0, L], \tag{5.43}$$

donde $u : [0, L] \to \mathbb{R}$ es la incógnita y $f : [0, L] \to \mathbb{R}$ es el **término fuente** prescrito. El signo negativo del operador es una convención estándar que produce un operador definido positivo y conecta directamente con la formulación variacional energética del problema. La ecuación (5.43) modela una amplia clase de fenómenos físicos en **equilibrio**: la distribución estacionaria de temperatura en una barra con fuentes internas, el potencial eléctrico generado por una distribución de carga lineal, la deformación de una membrana sometida a una carga vertical, o el potencial gravitatorio de una distribución de masa unidimensional. Todos estos problemas comparten una característica que los distingue de los abordados en las secciones precedentes: la **ausencia de dinámica temporal**.

Como consecuencia inmediata de esta diferencia estructural, el problema **no admite condiciones iniciales** —no existe un instante $t = 0$ desde el cual evolucionar—. El cierre del problema se realiza únicamente mediante condiciones de frontera, que en el caso de tipo Dirichlet prescriben los valores de u en los dos extremos del intervalo:

$$u(0) = u_L, \qquad u(L) = u_R. \tag{5.44}$$

A diferencia de las condiciones de frontera de las dos secciones anteriores —donde la dependencia temporal hacía que cada condición ocupara una *línea* del dominio espacio-temporal—, las condiciones (5.44) son ahora **exactamente dos puntos**, lo que cambia cualitativamente su tratamiento numérico.

Parámetros del experimento base. Se adoptan los valores

$$L = 1, \qquad u_L = u_R = 0, \qquad f(x) = \pi^2 \sin(\pi x). \tag{5.45}$$

La elección del término fuente $f(x) = \pi^2 \sin(\pi x)$ produce un problema cuya solución analítica es particularmente simple.

Solución analítica de referencia. El problema con $f(x) = \pi^2 \sin(\pi x)$ y CFs homogéneas se resuelve por inspección directa. Postulando $u(x) = A \sin(\pi x)$, se tiene $-u''(x) = A\pi^2 \sin(\pi x)$, lo que iguala $f(x) = \pi^2 \sin(\pi x)$ cuando $A = 1$. Las condiciones de frontera $u(0) = \sin(0) = 0$ y $u(1) = \sin(\pi) = 0$ se satisfacen automáticamente. La solución exacta es por tanto

$$u(x) \;=\; \sin(\pi x). \tag{5.46}$$

Más generalmente, para un término fuente expresado como combinación lineal de modos

$$f(x) \;=\; \sum_{k=1}^{K} (k\pi)^2 \sin(k\pi x), \tag{5.47}$$

la solución analítica correspondiente es

$$u(x) \;=\; \sum_{k=1}^{K} \sin(k\pi x), \tag{5.48}$$

Propiedades estructurales del problema elíptico. La ecuación de Poisson presenta tres propiedades cualitativas relevantes que la distinguen de los problemas evolutivos:

1. *Existencia y unicidad.* Bajo condiciones de regularidad muy generales sobre f, el problema admite una única solución, lo que descarta el modo de falla por convergencia a soluciones falsas. Si la PINN converge a una pérdida pequeña, lo hace necesariamente a la solución verdadera.

2. *Principio del máximo.* Para términos fuente $f \geq 0$, la solución satisface $u(x) \geq \min(u_L, u_R)$ en todo el dominio. Para el caso particular con CFs nulas y fuente positiva como el considerado aquí, la solución es no negativa en todo $[0, L]$.

3. *Suavidad de la solución.* Si f es de clase C^k, entonces u es de clase C^{k+2}. Para el caso con f analítica, u resulta también analítica. Esta regularidad supera la de los problemas evolutivos, donde la suavidad temporal podía estar limitada por el orden del operador.

Las tres propiedades anticipan que la formulación PINN, aplicada a este problema, debería arrojar los resultados **más precisos del capítulo**: el espacio de soluciones admisibles es un punto único, no existen falsos mínimos locales, y la regularidad analítica de la solución coincide con la del espacio funcional natural de las redes neuronales con activaciones suaves.

5.4.2. Formulación PINN para la ecuación de Poisson

La aplicación del marco PINN al problema se simplifica respecto a los casos espacio-temporales en **cuatro aspectos estructurales**: dominio unidimensional, residuo con una única derivada parcial (de segundo orden), ausencia de condiciones iniciales, y reducción de la pérdida a sólo **dos componentes**. Estas simplificaciones convierten al problema de Poisson en el *caso límite mínimamente complejo* de la metodología PINN.

La red neuronal aproximadora se denota

$$u_\theta : [0, L] \to \mathbb{R}, \qquad x \mapsto u_\theta(x), \tag{5.49}$$

con una única neurona de entrada que recibe el escalar x y una única neurona de salida con activación lineal. Las capas ocultas mantienen la configuración base de cinco capas con veinte neuronas por capa, empleada en las secciones anteriores.

Residuo de la ecuación. El residuo asociado a la ecuación de Poisson se define como

$$r_\theta(x) \;:=\; -\frac{d^2 u_\theta}{dx^2}(x) \;-\; f(x). \tag{5.50}$$

La evaluación numérica de $u''_\theta(x)$ requiere dos llamadas anidadas a `torch.autograd.grad`. El término fuente $f(x)$ se evalúa directamente sobre los puntos de colocación a partir de su expresión analítica, sin involucrar al modelo neuronal.

Estructura dual de la pérdida. La pérdida se descompone en sólo dos contribuciones:

$$\mathcal{L}(\theta) \;=\; \lambda_r \, \mathcal{L}_r(\theta) \;+\; \lambda_{BC} \, \mathcal{L}_{BC}(\theta), \tag{5.51}$$

donde

$$\mathcal{L}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| r_\theta(x_r^{(i)}) \right|^2, \tag{5.52}$$

$$\mathcal{L}_{BC}(\theta) = \frac{1}{2} \Big(|u_\theta(0) - u_L|^2 + |u_\theta(L) - u_R|^2 \Big). \tag{5.53}$$

El conjunto $\{x_r^{(i)}\}_{i=1}^{N_r}$ se distribuye uniformemente sobre $[0, L]$, mientras que la componente $\mathcal{L}_{BC}$ se evalúa en **exactamente dos puntos** —los extremos del dominio—. Esta es una diferencia notable respecto a los problemas espacio-temporales, donde la condición de frontera se evaluaba sobre líneas enteras del dominio bidimensional con doscientos puntos de muestreo redundante. Aquí, los dos puntos $x = 0$ y $x = L$ son suficientes porque las CFs son puntuales.

Ponderación de los términos. Se adoptan los pesos

$$\lambda_r = 1, \qquad \lambda_{BC} = 100, \tag{5.54}$$

manteniendo la misma asimetría empleada en las secciones anteriores. La justificación de la asimetría es, sin embargo, diferente. En los problemas espacio-temporales, $\lambda_{\text{data}} = 100$ compensaba la menor dimensión geométrica de las restricciones de datos frente al residuo. En el problema de Poisson, ambas componentes tienen dimensión geométrica formalmente comparable —el residuo se evalúa sobre un intervalo y la CF en dos puntos—, pero $\mathcal{L}_{BC}$ promedia sobre sólo dos sumandos frente a los miles del residuo. El peso $\lambda_{BC} = 100$ continúa fijando las CFs con fuerza relativa apropiada.

Embedding de Fourier 1D. Se emplea el embedding

$$\boldsymbol{\varphi}(x) = \left(\tfrac{x}{L},\ \{\sin(\omega_k x), \cos(\omega_k x)\}_{k=1}^{K_x}\right)^{\top}, \tag{5.55}$$

con frecuencias $\omega_k = k\pi/L$ y $K_x = 4$. La justificación es ahora aun más directa: las frecuencias $\omega_k = k\pi/L$ coinciden exactamente con las del primer eigenmodo de la solución analítica (para $k = 1$), proporcionando a la red una base estructuralmente perfecta para representar el caso base. La dimensión de salida del embedding es $2K_x + 1 = 9$.

Muestreo y optimización. Se emplean $N_r = 2{,}000$ puntos interiores muestreados uniformemente. El presupuesto de optimización mantiene la prescripción del capítulo: 5000 iteraciones de Adam ($\eta = 10^{-3}$) seguidas de hasta 2500 iteraciones de L-BFGS con búsqueda de línea de Wolfe.

5.4.3. Implementación en PyTorch

La implementación reutiliza la estructura modular de las secciones anteriores con dos modificaciones específicas del problema: el embedding es ahora unidimensional, y la función de pérdida incluye únicamente las dos componentes (5.52)–(5.53). El código del modelo se omite por brevedad. Los bloques nuevos son la función de pérdida y el muestreo de puntos.

```python
import torch
import torch.nn as nn
import numpy as np


class FourierEmbedding1D(nn.Module):
    """phi(x) = [x/L, sin(omega_k x), cos(omega_k x)] para k=1..K_x."""
    def __init__(self, n_freqs=4, L=1.0):
        super().__init__()
        freqs = torch.arange(1, n_freqs + 1, dtype=torch.float32)
        self.register_buffer('omega', np.pi * freqs / L)
        self.L = L
        self.n_freqs = n_freqs

    @property
    def output_dim(self):
        return 1 + 2 * self.n_freqs
```

```
    def forward(self, x):
        xx = x * self.omega.view(1, -1)
        return torch.cat([x / self.L, torch.sin(xx), torch.cos(xx)], dim=1)


def source_torch(x, k_modes=1):
    """f(x) = sum_{k=1..K} (k pi)^2 sin(k pi x)."""
    f = torch.zeros_like(x)
    for k in range(1, k_modes + 1):
        f += (k * np.pi)**2 * torch.sin(k * np.pi * x)
    return f
```

Listing 5.10: Embedding de Fourier 1D y término fuente.

```
def pinn_loss(model, x_int, x_bc, u_bc, k_modes=1,
              lambda_r=1.0, lambda_bc=100.0):
    # ----- Residuo PDE: -u''(x) - f(x) -----
    x = x_int.clone().requires_grad_(True)
    u = model(x)
    u_x  = torch.autograd.grad(u,   x, torch.ones_like(u),   create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)[0]
    f = source_torch(x, k_modes)
    r = -u_xx - f
    L_r = torch.mean(r**2)

    # ----- CF: u(0) = u_L, u(L) = u_R -----
    u_pred_bc = model(x_bc)
    L_bc = torch.mean((u_pred_bc - u_bc)**2)

    L_total = lambda_r * L_r + lambda_bc * L_bc
    return L_total, L_r, L_bc
```

Listing 5.11: Función de pérdida con dos componentes.

```
def sample_points(n_interior=2000, L=1.0, seed=42,
                  u_L=0.0, u_R=0.0):
    torch.manual_seed(seed); np.random.seed(seed)
    x_int = torch.rand(n_interior, 1, dtype=torch.float32) * L
    x_bc = torch.tensor([[0.0], [float(L)]], dtype=torch.float32)
    u_bc = torch.tensor([[u_L], [u_R]], dtype=torch.float32)
    return x_int, x_bc, u_bc


def train(model, k_modes=1, L=1.0, n_adam=5000, n_lbfgs=2500,
          n_interior=2000, lr_adam=1e-3, lambda_bc=100.0):
    x_int, x_bc, u_bc = sample_points(n_interior=n_interior, L=L)

    # Fase 1: Adam
    opt = torch.optim.Adam(model.parameters(), lr=lr_adam)
    for i in range(n_adam):
        opt.zero_grad()
        L_total, *_ = pinn_loss(model, x_int, x_bc, u_bc, k_modes=k_modes,
                                lambda_bc=lambda_bc)
        L_total.backward()
        opt.step()

    # Fase 2: L-BFGS
    opt = torch.optim.LBFGS(
        model.parameters(), lr=1.0, max_iter=n_lbfgs,
```

```
        tolerance_grad=1e-11, tolerance_change=1e-13,
        history_size=100, line_search_fn='strong_wolfe',
    )
    def closure():
        opt.zero_grad()
        L_total, *_ = pinn_loss(model, x_int, x_bc, u_bc, k_modes=k_modes,
                                lambda_bc=lambda_bc)
        L_total.backward()
        return L_total
    opt.step(closure)
    L_total, *_ = pinn_loss(model, x_int, x_bc, u_bc, k_modes=k_modes,
                            lambda_bc=lambda_bc)
    return L_total.item()
```

Listing 5.12: Muestreo y bucle de entrenamiento.

El código se ha probado en PyTorch 2.12.0 sobre el *backend* MPS de Apple Silicon. El experimento base se completó en aproximadamente 45 segundos en un chip M4. La reducción refleja simultáneamente la simplificación del problema en cuatro frentes ya enumerados: ausencia de variable temporal, residuo más sencillo, menos componentes en la pérdida y embedding de menor dimensión.

5.4.4. Resultados y comparación con la solución de referencia

La ejecución del procedimiento sobre la configuración base produce una aproximación numérica $u_\theta(x)$ que se compara con la solución exacta evaluada sobre una malla uniforme de 401 puntos en $[0, L]$. Los indicadores cuantitativos son: **error relativo** L^2 de $\mathcal{E}_{L^2} = 1.38 \times 10^{-5}$, **error puntual máximo** de $\mathcal{E}_{\text{máx}} = 1.88 \times 10^{-5}$ y **pérdida residual final** de $\mathcal{L}(\theta^\star) = 3.48 \times 10^{-5}$. El tiempo total de entrenamiento fue de 44.6 segundos en un chip Apple M4 con *backend* MPS, repartido en 37.0 s para la fase Adam y sólo 7.6 s para la fase L-BFGS.

Estos números deben leerse en el contexto comparativo del capítulo. La precisión alcanzada —error del orden 10^{-5}— es comparable a la obtenida para la ecuación de calor (4×10^{-5}) y dos órdenes de magnitud superior a la obtenida para la ecuación de onda (2.7×10^{-3}). Esta jerarquía empírica de precisiones —onda $\gg$ calor $\approx$ Poisson— es consistente con la jerarquía estructural de los operadores: hiperbólico $\to$ parabólico $\to$ elíptico, donde la pérdida progresiva de complejidad temporal coincide con la facilidad creciente del problema para una PINN con activaciones suaves.

Las componentes individuales de la pérdida final exhiben la asimetría más extrema de los problemas estudiados:

$$\mathcal{L}_r = 3.48 \times 10^{-5}, \qquad \mathcal{L}_{BC} = 2.09 \times 10^{-12}.$$

La componente $\mathcal{L}_{BC}$ es **siete órdenes de magnitud menor** que $\mathcal{L}_r$, frente a las diferencias de tres a cuatro órdenes observadas en los problemas espacio-temporales. La razón es estructural: $\mathcal{L}_{BC}$ se evalúa en sólo dos puntos exactos del dominio, y la red dispone de capacidad sobrada para satisfacer dos restricciones puntuales casi exactamente desde las primeras iteraciones del entrenamiento. El residuo $\mathcal{L}_r$ constituye el único objetivo de optimización numéricamente relevante después de unas cuantas iteraciones de Adam.

Solución aprendida. La figura 5.10 compara la salida de la PINN con la referencia analítica $u(x) = \sin(\pi x)$. La coincidencia es visualmente perfecta en la escala de la figura: las dos curvas se superponen indistinguiblemente desde $x = 0$ hasta $x = L = 1$, recorriendo el perfil sinusoidal con amplitud máxima 1 en $x = 0.5$.

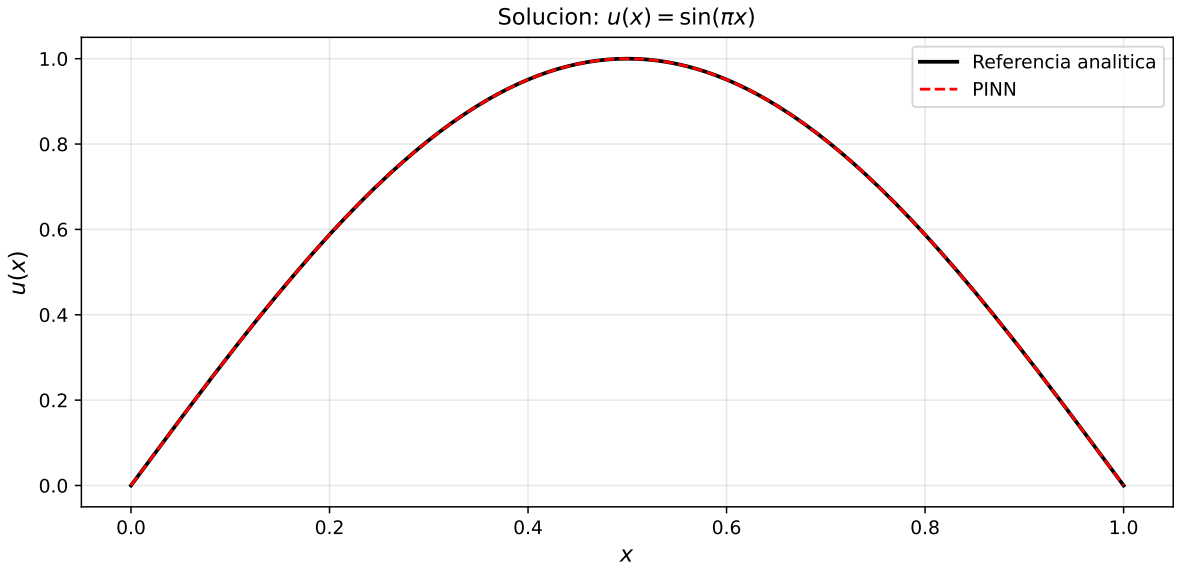


Figura 5.10: Comparación entre la solución de la PINN (línea discontinua roja) y la solución analítica $u(x) = \sin(\pi x)$ (línea continua negra).

Error puntual. La figura 5.11 muestra $|u_\theta(x) - u_{\text{ref}}(x)|$ en escala semilogarítmica. El error oscila entre 10^{-6} y 2×10^{-5} a lo largo del dominio, con un máximo cercano a $x \approx 0.3$ y un comportamiento gradualmente decreciente hacia los extremos. La curva exhibe varios valles pronunciados que corresponden a las posiciones espaciales en las que el error puntual $u_\theta - u_{\text{ref}}$ cambia de signo.

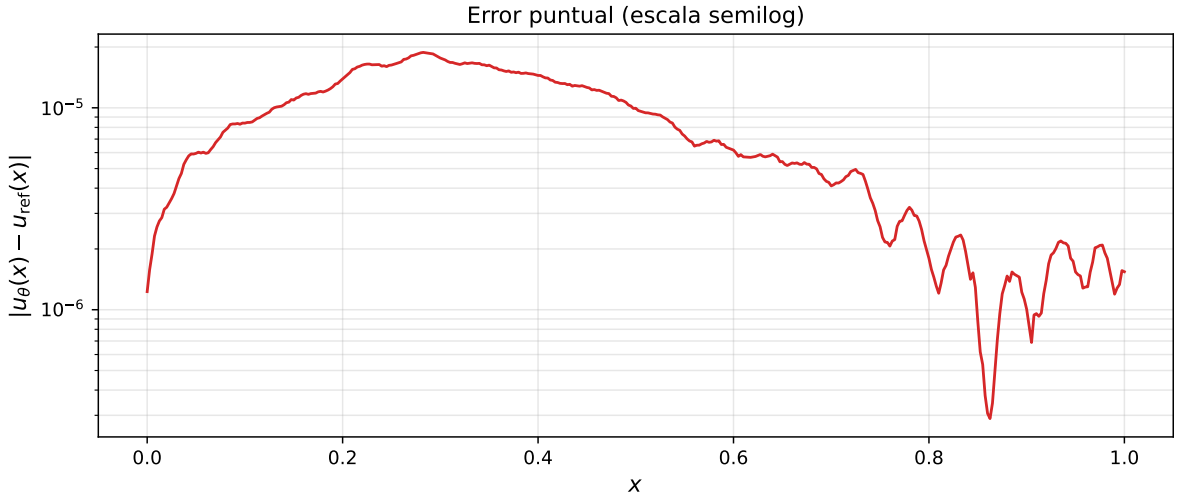


Figura 5.11: Error puntual $|u_\theta(x) - u_{\text{ref}}(x)|$ en escala semilogarítmica. Los valles del error corresponden a puntos donde la diferencia $u_\theta - u_{\text{ref}}$ cambia de signo.

Visualización del residuo. La figura 5.12 muestra una visualización directa del residuo $r_\theta(x) = -u_\theta''(x) - f(x)$ a lo largo del dominio. Esta figura no tiene análogo en las secciones anteriores: en los problemas espacio-temporales, el residuo es una función bidimensional difícil de inspeccionar gráficamente; en el problema unidimensional de Poisson, el residuo es una función escalar de x que

admite representación directa. La figura revela que el residuo oscila con magnitud típica $|r_\theta(x)| \approx 0.01$ en el interior del dominio, con picos pronunciados hacia los extremos. La estructura oscilatoria refleja el carácter aproximado de la red neuronal como solucionador local de la PDE: la solución aprendida satisface la PDE sólo *en promedio* sobre los puntos de colocación, lo que se traduce en residuos no nulos en puntos intermedios del dominio.

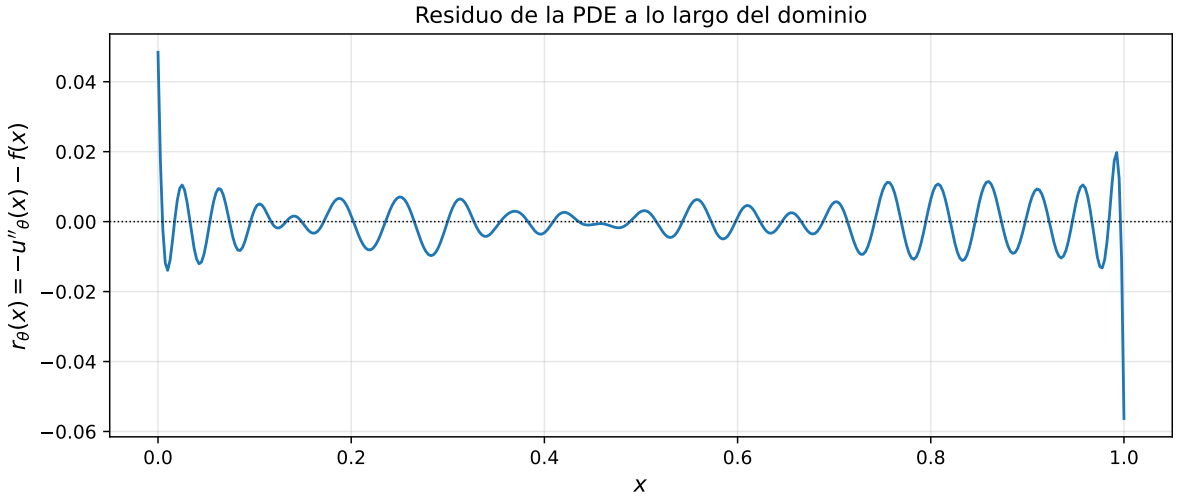


Figura 5.12: Residuo de la PDE $r_\theta(x) = -u''_\theta(x) - f(x)$ a lo largo del dominio.

Síntesis cuantitativa. La tabla 5.16 consolida los indicadores numéricos del experimento base.

Tabla 5.16: Indicadores de calidad del experimento base para la ecuación de Poisson con $L = 1$, $f(x) = \pi^2 \sin(\pi x)$, $u(0) = u(L) = 0$.

Indicador	Valor
Pérdida total final $\mathcal{L}(\theta^\star)$	3.48×10^{-5}
Componente $\mathcal{L}_r$ (residuo PDE)	3.48×10^{-5}
Componente $\mathcal{L}_{BC}$ (frontera Dirichlet)	2.09×10^{-12}
Error relativo L^2 $\mathcal{E}_{L^2}$	1.38×10^{-5}
Error puntual máximo $\mathcal{E}_{\text{máx}}$	1.88×10^{-5}
Residuo puntual máximo $\text{máx}_x \lvert r_\theta(x)\rvert$	5.63×10^{-2}
Tiempo de la fase Adam	37.0 s
Tiempo de la fase L-BFGS	7.6 s
Tiempo total	44.6 s
Hardware	Apple M4 (MPS)

El experimento confirma que la ecuación de Poisson es el problema más favorable a la metodología PINN entre los cuatro estudiados en este capítulo, con la precisión más alta y el tiempo de cómputo más bajo. Las razones estructurales —unicidad de la solución, ausencia de variable temporal, regularidad analítica, menor número de componentes en la pérdida— coinciden para producir un escenario de aplicabilidad ideal. La sección siguiente examina cómo varía este resultado al modificar los hiperparámetros principales y, en particular, cómo se degrada el método al aumentar la complejidad espectral del término fuente, lo que constituye el análogo natural —en un problema estacionario— de la variación de los parámetros de rigidez μ, c y α estudiados en las secciones precedentes.

5.4.5. Estudios de sensibilidad

Análogamente a los estudios realizados en las tres secciones precedentes, se examina la sensibilidad de la formulación PINN para la ecuación de Poisson al variar cuatro ejes de su configuración: la topología de la red, la función de activación, la ponderación de la condición de frontera y la complejidad espectral del término fuente. La cuarta variable es la novedad estructural de esta sección: dado que la ecuación de Poisson no posee un coeficiente físico análogo a μ, c o α que permita graduar la dificultad del problema, se varía en su lugar el número K de modos espaciales de la fuente $f(x) = \sum_{k=1}^{K} (k\pi)^2 \sin(k\pi x)$. Este parámetro mide directamente la *complejidad espectral* de la solución y proporciona un análogo natural al "parámetro de rigidez"de los problemas evolutivos.

Cada experimento de sensibilidad se entrena con el mismo presupuesto computacional que el caso base (5000 iteraciones de Adam y 2500 de L-BFGS) y con la misma semilla aleatoria.

Topología de la red

La tabla 5.17 contrasta cuatro configuraciones de profundidad y ancho, manteniendo fijos el resto de hiperparámetros (activación tanh, $K_x = 4$, $\lambda_{BC} = 100$, $k_{\text{modes}} = 1$).

Tabla 5.17: Sensibilidad respecto a la topología de la red. Activación tanh, $K_x = 4$, $\lambda_{BC} = 100$, $f(x) = \pi^2 \sin(\pi x)$.

Configuración	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
3 capas × 20 neuronas	1.90×10^{-5}	1.91×10^{-5}	4.66×10^{-6}
5 capas × 20 neuronas (base)	1.38×10^{-5}	1.88×10^{-5}	3.48×10^{-5}
5 capas × 50 neuronas	1.43×10^{-5}	1.49×10^{-5}	2.17×10^{-5}
7 capas × 80 neuronas	4.32×10^{-6}	6.04×10^{-6}	3.49×10^{-5}

Para este problema, **la configuración más grande es la mejor**: 7 capas × 80 neuronas reduce el error en aproximadamente un factor 3.2 respecto a la base, y todas las demás configuraciones también convergen al mismo orden de magnitud sin signos de sobreajuste.

La explicación estructural es directa. En los problemas espacio-temporales, la red dispone del producto $[0, L] \times [0, T]$ del dominio espacio-temporal como espacio de aproximación, lo que admite múltiples superficies de respuesta —algunas falsas— que satisfacen el residuo y las condiciones sobre el conjunto finito de puntos de muestreo. La sobreparametrización facilita encontrar tales soluciones ilegítimas. En el problema estacionario unidimensional, el dominio es simplemente el intervalo $[0, L]$ y la solución es *única*; no existen, por construcción matemática, "soluciones ilegítimas"suficientemente complejas para que la sobreparametrización las memorice. La capacidad adicional se traduce entonces en mejor aproximación sin penalización.

La conclusión metodológica es contrastante: **el sobreajuste sobre puntos de colocación, observado claramente en problemas espacio-temporales, está estructuralmente atenuado en problemas estacionarios de baja dimensión**. La elección de una red modesta sigue siendo razonable por consideraciones de costo computacional, pero no por razones de generalización.

Función de activación

La tabla 5.18 compara las tres funciones de activación sobre la arquitectura base de cinco capas por veinte neuronas.

Este estudio aporta dos hallazgos significativos. El primero es la **cuarta confirmación consecutiva de la superioridad de Swish sobre** tanh a través de los cuatro problemas examinados en este capítulo. El factor de mejora es modesto —Swish es 1.7 veces mejor que tanh en este problema—,

Tabla 5.18: Sensibilidad respecto a la función de activación. Arquitectura 5×20, $K_x = 4$, $\lambda_{BC} = 100$, $f(x) = \pi^2 \sin(\pi x)$.

Activación	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
Tangente hiperbólica tanh (base)	1.38×10^{-5}	1.88×10^{-5}	3.48×10^{-5}
Sigmoide σ	9.76×10^{-6}	1.07×10^{-5}	3.00×10^{-6}
Swish	8.28×10^{-6}	8.97×10^{-6}	5.85×10^{-6}

pero la consistencia del hallazgo a través de problemas de naturaleza estructural muy distinta consolida una **recomendación metodológica robusta**: para PINNs con soluciones suaves, Swish es preferible a tanh como activación por defecto.

El segundo hallazgo es novedoso: **la sigmoide supera a** tanh **por primera vez**, con un error 1.4 veces menor. Esta observación vuelve a poner sobre la mesa una hipótesis previamente descartada. Anteriormente se especuló que la sigmoide podría beneficiarse del rango natural $(0, 1)$ —ya que la solución de la ecuación de calor con la condición inicial considerada es no negativa— , pero el experimento de aquella sección no confirmó la hipótesis: la sigmoide era competitiva pero no superior. El presente experimento sugiere una reinterpretación más matizada: **la ventaja estructural de la sigmoide ante soluciones no negativas existe, pero es de magnitud modesta y queda enmascarada por otras fuentes de variabilidad cuando el problema posee complejidad espacio-temporal sustancial**. En el problema de Poisson, estacionario y unidimensional, donde la variabilidad residual es mínima, la diferencia estructural emerge como medible.

Una observación sobre la pérdida final: con sigmoide, $\mathcal{L}(\theta^\star) = 3.00 \times 10^{-6}$ es aproximadamente un orden de magnitud menor que con tanh, pese a que el error de la solución es sólo 1.4 veces menor. Esto refleja una vez más la observación metodológica: *el residuo y el error de la solución son cantidades distintas con magnitudes potencialmente desacopladas.*

Ponderación de la condición de frontera

La tabla 5.19 contrasta tres elecciones del peso λ_{BC}, manteniendo $\lambda_r = 1$ y el resto de hiperparámetros en sus valores base.

Tabla 5.19: Sensibilidad respecto a la ponderación de la condición de frontera. Arquitectura 5×20, tanh, $K_x = 4$, $\lambda_r = 1$, $f(x) = \pi^2 \sin(\pi x)$.

λ_{BC}	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
10	6.65×10^{-6}	8.72×10^{-6}	5.51×10^{-5}
100 (base)	1.38×10^{-5}	1.88×10^{-5}	3.48×10^{-5}
1000	4.99×10^{-6}	7.20×10^{-6}	2.10×10^{-5}

A simple vista no podemos ver un patrón definido y la configuración base $\lambda_{BC} = 100$ es la *peor* de las tres. Tanto $\lambda_{BC} = 10$ (2.1 veces mejor que la base) como $\lambda_{BC} = 1000$ (2.8 veces mejor) la superan. Este resultado es contraintuitivo y merece un análisis cuidadoso por dos razones.

En primer lugar, contrasta con los patrones observados en las dos secciones anteriores: para la ecuación de onda, el error decrecía al aumentar λ_{data} (con $\lambda = 1000$ óptimo); para la ecuación de calor, el patrón era opuesto (con $\lambda = 1000$ degradando el error). En el problema de Poisson, ninguno de los dos patrones prevalece.

En segundo lugar, la magnitud absoluta de las diferencias entre las tres configuraciones es **modesta**: los errores oscilan en un rango entre 5×10^{-6} y 1.4×10^{-5}, dentro de un mismo orden de

magnitud. Esto sugiere que el no observar un patrón fijo es atribuible en buena parte al **ruido de optimización amplificado por la simplicidad extrema del problema**: cuando los errores son tan bajos (10^{-5}), las variaciones residuales entre semillas y rutas de optimización pueden dominar sobre el efecto sistemático del hiperparámetro. La condición de frontera es además trivial de satisfacer —sólo dos puntos exactos con valor cero—, lo que reduce la sensibilidad efectiva del entrenamiento al peso elegido.

La interpretación más conservadora del estudio es por tanto: **en problemas estacionarios con condiciones de frontera puntuales triviales, la elección de λ_{BC} en un rango amplio (10^1 a 10^3) tiene impacto modesto sobre la precisión final**. La sensibilidad documentada en problemas espacio-temporales no se traslada al régimen estacionario unidimensional.

Régimen de complejidad espectral: variación de k_{modes}

La tabla 5.20 reporta los resultados de la configuración base aplicada al problema con términos fuente de complejidad espectral creciente, desde $k_{\text{modes}} = 1$ (un solo modo) hasta $k_{\text{modes}} = 5$.

Tabla 5.20: Desempeño en función del número de modos espaciales activos en $f(x) = \sum_{k=1}^{K_{\text{modes}}} (k\pi)^2 \sin(k\pi x)$. Arquitectura 5×20, tanh, $K_x = 4$, $\lambda_{BC} = 100$.

k_{modes}	$\mathcal{E}_{L^2}$	$\mathcal{E}_{\text{máx}}$	Pérdida final
1 (base)	1.38×10^{-5}	1.88×10^{-5}	3.48×10^{-5}
2	4.88×10^{-6}	9.88×10^{-6}	2.43×10^{-5}
3	1.03×10^{-5}	2.40×10^{-5}	1.01×10^{-4}
5	2.30×10^{-5}	5.20×10^{-5}	5.50×10^{-4}

Este estudio merece un análisis comparativo con su contraparte en el problema de la ecuación de onda, donde se varió el coeficiente c del operador de onda y se observó un **fallo catastrófico** al pasar de $c = 4$ a $c = 8$ ($\mathcal{E}_{L^2}$ saltaba de 10^{-3} a 10^0). La predicción razonable, dado que el embedding empleado en ambos casos tiene $K = 4$ frecuencias, era que el problema de Poisson exhibiría un fallo análogo al pasar a $k_{\text{modes}} = 5$ —número de modos que excede formalmente la capacidad espectral del embedding—.

La predicción no se materializa. El error sí crece con k_{modes}, pero la degradación es suave: $\mathcal{E}_{L^2}$ crece sólo un factor 1.7 entre la base y $k_{\text{modes}} = 5$, manteniéndose en todo el rango dentro del orden 10^{-5}. No hay falla catastrófica. La pérdida residual sí crece de manera más pronunciada, lo que valida nuevamente nuestra observación: **el residuo de la PDE y el error de la solución son cantidades desacopladas en problemas elípticos**, y un residuo elevado puede coexistir con una solución razonablemente correcta gracias al efecto suavizante del operador inverso $(-d^2/dx^2)^{-1}$.

Conclusiones del estudio de sensibilidad

Los experimentos anteriores permiten extraer cuatro conclusiones sobre la resolución de la ecuación de Poisson mediante PINNs, varias de ellas en **contraste explícito** con los patrones observados en las secciones evolutivas anteriores:

1. La topología **no exhibe el sobreajuste pronunciado** observado para problemas espacio-temporales. La red más grande (7 capas $\times$ 80 neuronas) es la mejor, con factor 3.2 de mejora sobre la base, y todas las configuraciones convergen al mismo orden de magnitud. La explicación estructural es la unicidad de la solución en problemas estacionarios elípticos.

2. La superioridad de **Swish sobre** tanh **se confirma por cuarta vez consecutiva**, consolidando definitivamente la recomendación metodológica de adoptar Swish como activación por defecto. La novedad de la sección es la **primera victoria de la sigmoide sobre** tanh **en el capítulo**: la ventaja estructural por compatibilidad de rangos emerge como medible en el régimen estacionario simple, donde la variabilidad residual no la enmascara.

3. La ponderación de la condición de frontera exhibe un patrón **no monotónico** con la base como peor configuración, hallazgo cualitativamente distinto de los patrones monotónicos opuestos observados en onda y calor. La interpretación conservadora es que en problemas estacionarios con CFs puntuales triviales, la elección de λ_{BC} en un rango amplio tiene impacto modesto, dominado por el ruido de optimización.

4. El **modo de falla por insuficiencia del embedding predicho para** $k_{\text{modes}} = 5$ **no se materializa** como fallo catastrófico, en contraste claro con el comportamiento observado para $c = 8$ en la ecuación de onda. La degradación es suave y se localiza principalmente en el residuo de la PDE, no en el error de la solución, lo que valida el desacoplamiento entre ambas métricas en problemas elípticos.

5.5. Análisis transversal de hiperparámetros y funciones de pérdida

Las cuatro secciones anteriores aplicaron una **misma receta unificada** —red de cinco capas con veinte neuronas, activación tanh, embedding de Fourier modesto, ponderación de datos $\lambda_{\text{data}} = 100$, optimización híbrida Adam → L-BFGS— a cuatro problemas que abarcan la taxonomía relevante de las ecuaciones diferenciales: una ecuación diferencial ordinaria no lineal y rígida (Van der Pol), una ecuación en derivadas parciales hiperbólica (onda), una parabólica (calor) y una elíptica (Poisson). La decisión de mantener la receta constante a través de problemas estructuralmente tan distintos convierte al capítulo en un **experimento controlado**: el método es la variable fija y la naturaleza de la ecuación es la variable independiente. La presente sección cosecha lo que esa decisión de diseño permite observar —qué propiedades del método se mantienen estables a través de las clases de problema y cuáles dependen críticamente de la estructura específica de la ecuación—.

El análisis se organiza en cuatro partes: una síntesis comparativa de los casos base, los hallazgos transversales sobre los hiperparámetros principales, una taxonomía de los modos de falla observados y un conjunto de recomendaciones metodológicas accionables.

5.5.1. Síntesis comparativa de los casos base

La tabla 5.21 consolida los indicadores de los cuatro experimentos base en una sola vista comparativa.

La figura 5.13 representa gráficamente la jerarquía de precisión en escala logarítmica. El ordenamiento empírico de los errores —de mayor a menor— es: **onda** (2.7×10^{-3}), **Van der Pol** (2.4×10^{-4}), **calor** (4.0×10^{-5}) y **Poisson** (1.4×10^{-5}). El rango total abarca aproximadamente **dos órdenes de magnitud**, pese a que todos los problemas se resolvieron con idéntica arquitectura, presupuesto computacional y receta de optimización.

Esta jerarquía admite una interpretación estructural coherente. La dificultad de un problema para la formulación PINN básica crece con dos factores acoplados: la **complejidad oscilatoria de la solución** y el **número de componentes en competencia** dentro de la pérdida. La ecuación de onda combina ambos en su contra —solución oscilatoria en el tiempo y cuatro componentes de

Tabla 5.21: Síntesis comparativa de los cuatro problemas resueltos con la receta unificada. Para Van der Pol se reporta el error de la componente de posición y_1. Los estudios de sensibilidad de cada sección se ejecutaron como un lote independiente del experimento base; la reproducción de la configuración base puede diferir ligeramente del valor aquí reportado por variabilidad numérica entre ejecuciones en el *backend* MPS.

Problema	Tipo	Comp. pérdida	$\mathcal{E}_{L^2}$	Tiempo (s)
Van der Pol	EDO no lineal rígida	2	2.43×10^{-4}	193
Onda 1D	EDP hiperbólica	4	2.66×10^{-3}	300
Calor 1D	EDP parabólica	3	3.99×10^{-5}	170
Poisson 1D	EDP elíptica	2	1.38×10^{-5}	45

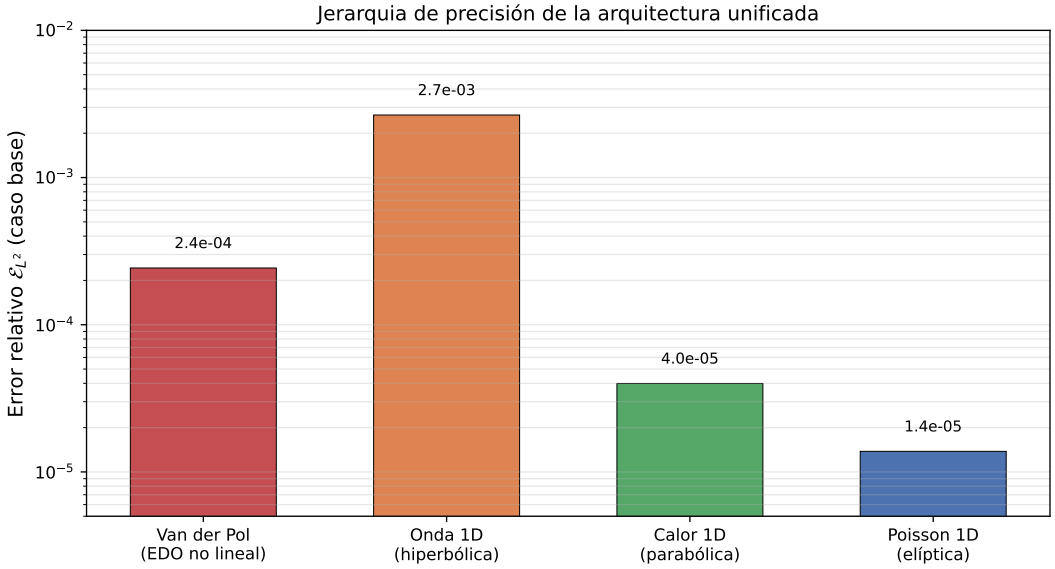


Figura 5.13: Jerarquía de precisión de la receta unificada a través de los cuatro problemas, en escala logarítmica. El error abarca dos órdenes de magnitud, con la ecuación hiperbólica como el caso más exigente y la elíptica como el más favorable.

pérdida que compiten por la atención del optimizador—, lo que explica su posición como el problema más exigente. El oscilador de Van der Pol presenta dinámica oscilatoria pero sólo dos componentes de pérdida, situándose en segundo lugar. La ecuación del calor, parabólica y disipativa, posee una solución monótonamente decreciente sin oscilación y tres componentes de pérdida, alcanzando alta precisión. Finalmente, la ecuación de Poisson —estacionaria, sin variable temporal, con solución de regularidad analítica y sólo dos componentes de pérdida— representa el escenario ideal y alcanza la máxima precisión con el menor tiempo de cómputo. La conclusión transversal es clara: **las PINNs son particularmente efectivas para problemas cuyas soluciones poseen alta regularidad y baja oscilación**, propiedades que coinciden con el sesgo inductivo natural de las redes con activaciones suaves.

5.5.2. Hallazgos transversales sobre hiperparámetros

Los cuatro problemas se estudiaron a lo largo de los mismos ejes de sensibilidad, lo que permite extraer conclusiones que trascienden cada problema individual.

Función de activación

La figura 5.14 compara las tres activaciones a través de los cuatro problemas en escala logarítmica. El hallazgo es inequívoco: **Swish supera a la tangente hiperbólica en los cuatro problemas sin excepción**. La consistencia del resultado a través de problemas de naturaleza estructuralmente distinta eleva la observación de un hallazgo casual a una **recomendación metodológica robusta**: para problemas con soluciones suaves, Swish es preferible a tanh como activación por defecto. La explicación reside en las propiedades de Swish ($x \cdot \sigma(x)$): su gradiente no saturado para entradas positivas grandes y su no monotonía permiten una optimización más eficiente que la tangente hiperbólica, cuyo gradiente se satura en ambos extremos.

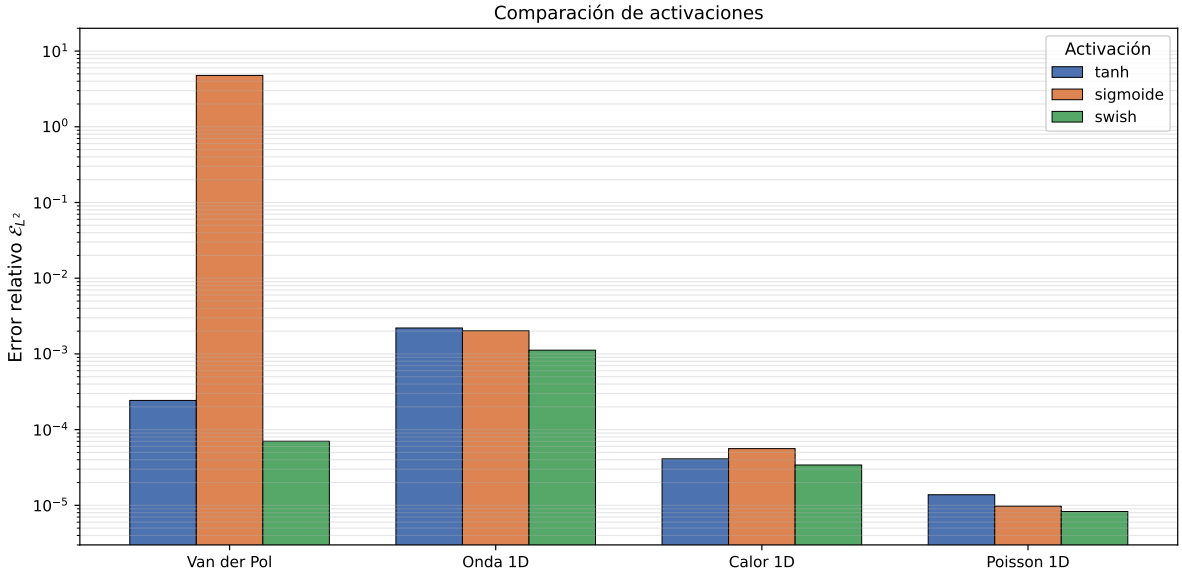


Figura 5.14: Comparación del error relativo L^2 entre las tres funciones de activación a través de los cuatro problemas, en escala logarítmica. Swish (verde) produce el menor error en todos los casos. La sigmoide falla catastróficamente en Van der Pol (barra que supera la unidad).

El comportamiento de la **sigmoide** cuenta una historia más matizada y dependiente del problema. En el oscilador de Van der Pol falla catastróficamente debido a la incompatibilidad entre su rango asimétrico $(0, 1)$ y una solución que oscila simétricamente alrededor de cero. En la ecuación de onda es competitiva con tanh, pues la simetría de la onda estacionaria atenúa la penalización del rango. En la ecuación de calor es ligeramente peor que tanh. Y en la ecuación de Poisson —estacionaria, con solución estrictamente no negativa— la sigmoide **supera a** tanh por primera vez, materializando la ventaja estructural por compatibilidad de rangos que en problemas más complejos quedaba enmascarada. La conclusión transversal es que la sigmoide es viable únicamente en problemas de solución no negativa y baja complejidad, mientras Swish ofrece una ventaja universal.

Ponderación de la pérdida

En contraste directo con la robustez del hallazgo sobre la activación, la ponderación óptima de los términos de la pérdida resultó ser **fuertemente dependiente del problema**, sin patrón universal. La tabla 5.22 resume los cuatro comportamientos observados.

El oscilador de Van der Pol prefiere el peso mínimo ($\lambda = 1$) mientras la ecuación de onda prefiere el máximo ($\lambda = 1000$), con la ecuación de calor en el valor intermedio ($\lambda = 100$). En Van

Tabla 5.22: Comportamiento de la ponderación de los términos de datos a través de los cuatro problemas.

Problema	Óptimo	Patrón observado
Van der Pol	$\lambda = 1$	menor λ mejor; sobre-pesar la CI es redundante
Onda 1D	$\lambda = 1000$	monótono decreciente con λ
Calor 1D	$\lambda = 100$	monótono; $\lambda = 1000$ degrada (opuesto a onda)
Poisson 1D	no concluyente	no monótono; diferencias dominadas por ruido

der Pol, sobre-pesar la condición inicial es redundante porque el campo vectorial del sistema es muy informativo cerca de $t = 0$ y el residuo ya ancla la trayectoria; en la onda, en cambio, las restricciones aportan información estructural genuina, como se detalla a continuación.

El contraste más ilustrativo se da entre la ecuación de onda y la del calor. En la onda, aumentar el peso de las condiciones de frontera hasta $\lambda = 1000$ *mejora* el error, porque las condiciones de frontera Dirichlet imponen una restricción estructuralmente no trivial sobre la solución oscilatoria. En el calor, el mismo aumento *degrada* el error, porque las condiciones de frontera homogéneas son automáticamente compatibles con la tendencia natural del sistema parabólico a decaer a cero, y sobre-pesarlas distrae al optimizador del residuo. Esta inversión exacta del patrón óptimo entre dos problemas tratados con idéntica formulación constituye la **lección metodológica central** sobre la ponderación: el peso óptimo depende de si las restricciones impuestas aportan información estructural genuina o son meramente compatibles con la dinámica espontánea del problema. No existe un valor universal de λ.

Topología

El estudio de la topología reveló igualmente un comportamiento dependiente del problema. En la ecuación de calor, la configuración base de cinco capas por veinte neuronas resultó óptima, y las redes más grandes *degradaron* el error por sobreajuste a los puntos de colocación. En la ecuación de Poisson, en cambio, la red más grande fue la *mejor*, sin signo alguno de sobreajuste. La explicación es estructural: el sobreajuste sobre el conjunto finito de puntos de colocación requiere la existencia de patrones que la sobreparametrización pueda memorizar, y tales patrones son más accesibles en el dominio bidimensional espacio-temporal del problema parabólico que en el dominio unidimensional estacionario del problema elíptico, cuya solución es además única. La recomendación transversal es conservadora: **una arquitectura modesta es una elección segura** —nunca fue catastrófica en ningún problema— aunque la capacidad adicional puede beneficiar a los problemas estacionarios de baja dimensión.

5.5.3. Taxonomía de modos de falla

El hallazgo conceptualmente más rico del análisis transversal emerge al forzar cada problema más allá de su régimen base, aumentando su respectivo parámetro de dificultad. La figura 5.15 reúne los cuatro barridos en una sola vista con escala vertical compartida, lo que permite comparar directamente la *forma* de la degradación.

- **Falla catastrófica (fila superior).** El oscilador de Van der Pol, al pasar de $\mu = 1$ a $\mu = 2$, sufre un salto del error de más de tres órdenes de magnitud. La red converge a una pérdida residual pequeña pero a una trayectoria cualitativamente equivocada, fenómeno característico de los problemas rígidos donde la separación de escalas temporales crea un paisaje de optimización con mínimos engañosos. La ecuación de onda, al aumentar la velocidad c, exhibe

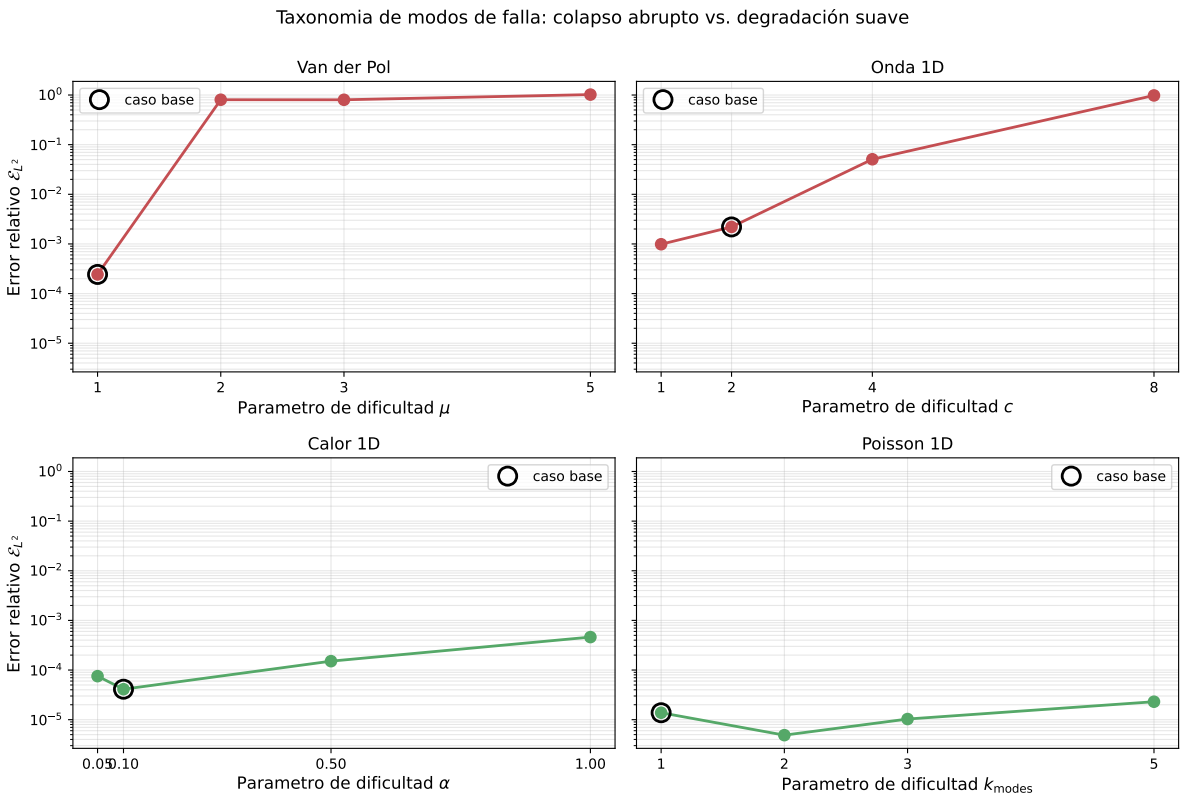


Figura 5.15: Taxonomía de modos de falla.

un salto de dos órdenes de magnitud entre $c = 4$ y $c = 8$; aquí la razón es la **insuficiencia espectral** del embedding de Fourier, que con $K_t = 4$ frecuencias temporales no cubre los modos de alta frecuencia $\omega \propto c$ requeridos por velocidades elevadas.

- **Degradación suave (fila inferior).** La ecuación de calor, al aumentar la difusividad α hasta 1.0, degrada su error de forma *monótona y suave* manteniéndose dentro de un mismo orden de magnitud. La ecuación de Poisson, al aumentar la complejidad espectral del término fuente hasta $k_{\text{modes}} = 5$, presenta igualmente una degradación suave, con el residuo de la PDE creciendo más que el error de la solución gracias al efecto suavizante del operador inverso elíptico.

En general, **los problemas evolutivos con dinámica fuerte —alta rigidez o alta velocidad de propagación— tienden a fallar de forma abrupta cuando se excede un umbral, mientras los problemas disipativos o estacionarios degradan de forma gradual y predecible**. La distinción es relevante para el diseño experimental: en la primera clase, un pequeño cambio del parámetro físico puede invalidar por completo la solución, lo que exige verificación cuidadosa; en la segunda clase, la calidad de la solución se deteriora de manera controlada y la formulación básica conserva utilidad práctica en un rango más amplio de parámetros.

5.5.4. Recomendaciones metodológicas

La síntesis de los hallazgos del capítulo permite formular un conjunto de recomendaciones accionables para la aplicación de PINNs a problemas de ecuaciones diferenciales:

1. **Activación.** Adoptar Swish como activación por defecto. Superó a la tangente hiperbólica en los cuatro problemas estudiados y nunca falló catastróficamente, a diferencia de la sigmoide. La tangente hiperbólica sigue siendo una alternativa segura y robusta cuando se requiere máxima estabilidad numérica.

2. **Ponderación de la pérdida.** No existe un valor universal del peso λ. Conviene ensayar al menos los valores $\{1, 10, 100, 1000\}$ para cada problema nuevo, o bien adoptar directamente un esquema de ponderación adaptativa cuando el problema involucra múltiples restricciones. Depende de si las restricciones aportan información estructural o son compatibles con la dinámica espontánea.

3. **Topología.** Una arquitectura modesta de cinco capas por veinte neuronas es una elección segura y eficiente; nunca fue catastrófica. La capacidad adicional sólo se justifica para problemas estacionarios de baja dimensión, donde no induce sobreajuste.

4. **Embedding de Fourier.** El número de frecuencias del embedding debe escalarse con la complejidad espectral esperada de la solución. La insuficiencia espectral fue el mecanismo de falla de la ecuación de onda a alta velocidad; un embedding dimensionado a la frecuencia máxima esperada habría evitado el colapso.

5. **Verificación frente al régimen de dificultad.** Antes de confiar en una solución PINN, conviene caracterizar la sensibilidad del problema a su parámetro de dificultad. Los problemas evolutivos con dinámica fuerte pueden fallar abruptamente, produciendo soluciones con pérdida pequeña pero cualitativamente incorrectas; en estos casos, la comparación con una solución de referencia —numérica o analítica— es indispensable.

En conjunto, los experimentos de este capítulo confirman que la formulación PINN básica es una herramienta efectiva y versátil para una amplia clase de problemas de ecuaciones diferenciales, con un régimen de aplicabilidad bien definido. Sus modos de falla —documentados aquí de forma sistemática— no son defectos arbitrarios sino manifestaciones predecibles de las propiedades estructurales de cada problema, y motivan empíricamente las técnicas adaptativas avanzadas.

Conclusiones Generales

El presente trabajo recorrió el camino que conduce desde los fundamentos del aprendizaje profundo hasta la aplicación de las redes neuronales informadas por la física a la resolución de ecuaciones diferenciales. La exposición partió de la configuración de un entorno de desarrollo reproducible y de la definición matemática de las redes neuronales, continuó con el desglose de su mecánica interna mediante la propagación directa y la retropropagación, profundizó en las técnicas de aprendizaje profundo orientadas a sistemas dinámicos y culminó en el núcleo experimental del último capítulo, donde una misma metodología se aplicó sistemáticamente a cuatro problemas.

5.6. Cumplimiento de los objetivos

El primer objetivo académico —realizar una investigación estructurada sobre la aplicación de redes neuronales en la resolución de ecuaciones diferenciales ordinarias y parciales— se cumplió a través de la articulación de los capítulos teórico y experimental. El capítulo 4 estableció el marco conceptual, situando a las PINNs dentro del panorama más amplio del aprendizaje profundo para sistemas dinámicos, con las variantes auto-adaptativas y con las *Neural ODEs*. El capítulo 5 materializó ese marco en experimentos controlados sobre el oscilador de Van der Pol y las ecuaciones de onda, calor y Poisson en una dimensión espacial, cubriendo así el espectro de problemas no lineales rígidos, hiperbólicos, parabólicos y elípticos comprometido en el plan de trabajo.

El segundo objetivo —identificar y mostrar elementos teóricos y prácticos con valor didáctico aplicable a los temarios de Matemáticas para Ciencias Aplicadas— se atendió mediante el diseño de una secuencia de ejemplos de complejidad ascendente, íntegramente reproducible y construida sobre ecuaciones que el estudiante ya conoce de su formación. La decisión metodológica de aplicar una misma receta a los cuatro problemas convirtió al conjunto en un experimento comparativo cuyos hallazgos transversales poseen en sí mismos un valor pedagógico: ilustran cómo una técnica numérica responde a la estructura matemática del problema al que se aplica.

En cuanto al propósito dual del trabajo, ambas vertientes encontraron respuesta. Respecto a la exploración de los **alcances y las limitaciones de las PINNs frente a los métodos numéricos clásicos**, los experimentos permitieron una valoración matizada y honesta. Las PINNs demostraron ser una herramienta versátil —libre de malla, capaz de incorporar restricciones físicas directamente en la función objetivo y naturalmente extensible a problemas inversos—, pero también revelaron sus límites: para los problemas directos sencillos abordados, su precisión y su costo computacional no compiten con los de un método clásico bien establecido, como la integración mediante esquemas de Runge-Kutta implícitos o las diferencias finitas, que alcanzan precisiones superiores en fracciones del tiempo. El valor de las PINNs no reside, por tanto, en desplazar a los métodos clásicos en su terreno, sino en su flexibilidad estructural y en su capacidad de abordar problemas donde los métodos tradicionales encuentran dificultades. Respecto a la segunda vertiente —mostrar que el *machine learning* puede integrarse de manera natural en los cursos de Matemáticas para Ciencias Aplicadas—, el material desarrollado, con su progresión cuidada y su código abierto, constituye una

propuesta concreta y lista para su adaptación al aula.

5.7. Limitaciones del trabajo

Los resultados de este trabajo deben interpretarse dentro de un alcance bien delimitado, acorde con su finalidad didáctica. Todos los problemas abordados son unidimensionales en espacio, con geometrías sencillas, coeficientes constantes y soluciones de referencia conocidas en forma analítica o mediante integración numérica de alta precisión. El estudio se concentró en la formulación PINN básica, sin instrumentar empíricamente las variantes avanzadas que se presentaron en el plano teórico. Asimismo, el trabajo no constituye un *benchmark* riguroso frente a los métodos numéricos clásicos —no se midieron sistemáticamente los tiempos comparativos ni se exploraron los regímenes de alta dimensión donde las ventajas de los métodos basados en redes podrían manifestarse—. Estas restricciones, apropiadas para los fines perseguidos, acotan la generalidad de las conclusiones a la clase de problemas efectivamente estudiada.

5.8. Trabajo futuro

Las limitaciones anteriores definen direcciones naturales de continuación. En el plano metodológico, la extensión a problemas bidimensionales y tridimensionales pondría a prueba la escalabilidad del muestreo de puntos de colocación y motivaría las técnicas de descomposición de dominio. Los modos de falla documentados constituyen puntos de entrada concretos para evaluar empíricamente las PINNs auto-adaptativas y los esquemas de entrenamiento causal discutidos teóricamente en el capítulo 4, cerrando el ciclo entre la formulación básica, sus limitaciones y sus refinamientos. El estudio de los problemas inversos —donde las PINNs exhiben ventajas más claras sobre los métodos clásicos— es otra extensión de alto valor. En una dirección complementaria, el aprendizaje de operadores mediante arquitecturas como DeepONet o los operadores neuronales de Fourier abre la posibilidad de aproximar familias enteras de soluciones en lugar de una solución individual, lo que representa un cambio cualitativo en el alcance del enfoque.

En el plano pedagógico, la continuación más inmediata es la transformación del material desarrollado en prácticas y ejercicios concretos para el curso de Matemáticas para Ciencias Aplicadas, con guías graduadas que acompañen al estudiante desde la reproducción de los experimentos base hasta la exploración autónoma de sus variantes.

5.9. Reflexión final

Más allá de sus resultados técnicos, este trabajo ilustra una convergencia que define buena parte de la matemática aplicada contemporánea: la que une los métodos analíticos y numéricos clásicos con las herramientas del aprendizaje automático. Para el estudiante de Ciencias de la Computación, comprender esta convergencia —no como una sustitución de lo viejo por lo nuevo, sino como una ampliación del repertorio de herramientas disponibles, cada una con su régimen de aplicabilidad— es una competencia formativa de creciente importancia. Si el presente trabajo logra que dicha convergencia resulte accesible y motivante para quien se inicia en el tema, habrá cumplido su cometido más profundo, en consonancia con el espíritu del Servicio Social que le dio origen.

Apéndice A

Repositorio del Proyecto

El código fuente completo en Python, implementando los algoritmos descritos se encuentra en GitHub (@santiago861).

Bibliografía

[1] A. Kaveh, *Applications of Artificial Neural Networks and Machine Learning in Civil Engineering*, Studies in Computational Intelligence, vol. 1168, Springer Nature Switzerland, 2024. ISBN: 978-3-031-66051-1. DOI: 10.1007/978-3-031-66051-1.

[2] H. Nelson, *Essential Math for AI: Next-Level Mathematics for Efficient and Successful AI Systems*, O'Reilly Media, 2023. ISBN: 978-1-098-10763-5.

[3] K. D. Toennies, *An Introduction to Image Classification: From Designed Models to End-to-End Learning*, Springer Nature Singapore, 2024. ISBN: 978-981-99-7882-3. DOI: 10.1007/978-981-99-7882-3.

[4] M. Raissi, P. Perdikaris, G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, vol. 378, pp. 686–707, 2019. DOI: 10.1016/j.jcp.2018.10.045.

[5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, *Neural Ordinary Differential Equations*, Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018.

[6] L. D. McClenny, U. Braga-Neto, *Self-Adaptive Physics-Informed Neural Networks using a Soft Attention Mechanism*, Journal of Computational Physics, vol. 474, art. 111722, 2023. DOI: 10.1016/j.jcp.2022.111722.

[7] J. S. Butler, *Numerical Methods and Machine Learning for Differential Equations with Applications in Python*, Technological University Dublin, 2021. Disponible en: https://john-s-butler-dit.github.io/NM_ML_DE_source/

[8] W. E, B. Yu, *The Deep Ritz Method: A Deep Learning-Based Numerical Method for Solving Variational Problems*, Communications in Mathematics and Statistics, vol. 6, no. 1, pp. 1–12, 2018. DOI: 10.1007/s40304-018-0127-z.

[9] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*, Nature Machine Intelligence, vol. 3, pp. 218–229, 2021. DOI: 10.1038/s42256-021-00302-5.

[10] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhatt, A. Stuart, A. Anandkumar, *Fourier Neural Operator for Parametric Partial Differential Equations*, International Conference on Learning Representations (ICLR), 2021.

[11] W. A. Strauss, *Partial Differential Equations: An Introduction*, 2.ª ed., John Wiley & Sons, 2007. ISBN: 978-0-470-05456-7.

[12] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, Society for Industrial and Applied Mathematics (SIAM), 2007. ISBN: 978-0-898716-29-0. DOI: 10.1137/1.9780898717839.

[13] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2.ª ed., Springer Series in Computational Mathematics, vol. 14, Springer-Verlag, 1996. DOI: 10.1007/978-3-642-05221-7.

[14] L. C. Evans, *Partial Differential Equations*, 2.ª ed., Graduate Studies in Mathematics, vol. 19, American Mathematical Society, 2010. ISBN: 978-0-8218-4974-3.

[15] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016. ISBN: 978-0-262-03561-3. Disponible en: https://www.deeplearningbook.org.

[16] D. P. Kingma, J. Ba, *Adam: A Method for Stochastic Optimization*, International Conference on Learning Representations (ICLR), 2015. arXiv: 1412.6980.

[17] D. C. Liu, J. Nocedal, *On the Limited Memory BFGS Method for Large Scale Optimization*, Mathematical Programming, vol. 45, pp. 503–528, 1989. DOI: 10.1007/BF01589116.

[18] X. Glorot, Y. Bengio, *Understanding the Difficulty of Training Deep Feedforward Neural Networks*, Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR, vol. 9, pp. 249–256, 2010.

[19] P. Ramachandran, B. Zoph, Q. V. Le, *Searching for Activation Functions*, arXiv preprint, 2017. arXiv: 1710.05941.

[20] A. Paszke, S. Gross, F. Massa, et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019. arXiv: 1912.01703.

[21] I. E. Lagaris, A. Likas, D. I. Fotiadis, *Artificial Neural Networks for Solving Ordinary and Partial Differential Equations*, IEEE Transactions on Neural Networks, vol. 9, no. 5, pp. 987–1000, 1998. DOI: 10.1109/72.712178.

[22] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, M. W. Mahoney, *Characterizing Possible Failure Modes in Physics-Informed Neural Networks*, Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021. arXiv: 2109.01050.

[23] S. Wang, X. Yu, P. Perdikaris, *When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective*, Journal of Computational Physics, vol. 449, art. 110768, 2022. DOI: 10.1016/j.jcp.2021.110768.

[24] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, *Physics-Informed Machine Learning*, Nature Reviews Physics, vol. 3, no. 6, pp. 422–440, 2021. DOI: 10.1038/s42254-021-00314-5.

[25] S. Cuomo, V. Schiano Di Cola, F. Giampaolo, G. Rozza, M. Raissi, F. Piccialli, *Scientific Machine Learning through Physics-Informed Neural Networks: Where We Are and What's Next*, Journal of Scientific Computing, vol. 92, no. 3, art. 88, 2022. DOI: 10.1007/s10915-022-01939-z.

[26] M. Tancik, P. P. Srinivasan, B. Mildenhall, et al., *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains*, Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 7537–7547, 2020. arXiv: 2006.10739.

[27] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis, *DeepXDE: A Deep Learning Library for Solving Differential Equations*, SIAM Review, vol. 63, no. 1, pp. 208–228, 2021. DOI: 10.1137/19M1274067.